Universitat Oberta de Catalunya

How to use MVM

User guide

Juan Antonio Gómez Gutiérrez

Content

Introduction

MVM is the tool that supports our detection, validation and repair strategy that we propose in the works:

- 'Tool_for_Debugging_Unsatisfiable_Integrity_Constraints_in_UMLOCL_Class_Diagrams (EMMSAD 2020)',
- 'Interactive_repair_of_inconsistencies (04 ER2025)'
- 'Interactive_repair_in_conceptual_models_using_LLM'.

As a strategy, it could have been implemented in various programming languages, however, having decided to base ourselves on the USE tool as a starting point, MVM is developed in Java and as an extension of it. In this way, MVM uses many of the functionalities that USE provides as standard and extends it by taking advantage of its magnificent environment.

Strategy

Our strategy includes the following sections:

1. **Consistency Check**: Determine if a UML/OCL diagram is consistent.
2. **Diagnosis**: Identify the unsatisfactory core, the minimum subsets of constraints involved in the inconsistency, as well as example instances that satisfy the maximum number of constraints in the model.
3. **Validation**: Create, visualize, and modify instances using a graphical user interface, and evaluate the validity of model constraints in the context of that instance.
4. **Guided Interactive Repair**: Propose possible solutions to identified inconsistencies, both in terms of graphical constraints in the class diagram (such as multiplicities) and textual constraints (such as OCL invariants); evaluate the suitability of such candidate solutions; and reverse if a previous decision in the repair process is deemed inadequate.

In addition to determining whether a model is satisfactory or not, the intention is to indicate which are the elements that cause unsatisfactoriness and propose alternatives for their repair, having the possibility of creating instances that demonstrate the viability of the model.

MVM TOOL

Introduction

MVM is the tool that supports our strategy and has been implemented in Java as an extension of the USE (UML-based Specification Environment) tool.

If you want to do tests, you can download it from our **GitHub** page in a simple way:

https://github.com/juanto2021/MVM?tab=readme-ov-file#readme

It is a zip that, once unzipped, contains everything you need to run **USE**+MVM. The only thing you need to be able to run it is **Java 11** or higher.

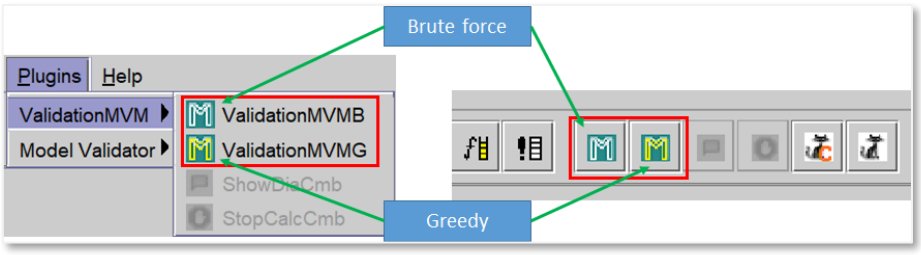
Search for MUS/MSS

The first functionality that MVM provided was the search for **MUS** (**Minimum Unsatisfiable Core**) and **MSS** (**Maximum Satisfiable Subset**). To do this, it constructs all possible combinations between the invariants of the model and, relying on the **Solver** used by **USE** (**kodkodSolver**), manufactures lists with groups of satisfactory and unsatisfactory invariants.

The calculation of **MUS/MSS** can be done in 2 ways:

1. **Brute force method**: Searches for all combinations before presenting any results.
2. **Greedy method**: it looks for a first group of invariants that are not related to each other and gives a result so that the user can start working while continuing to work in the background to complete all possible combinations.

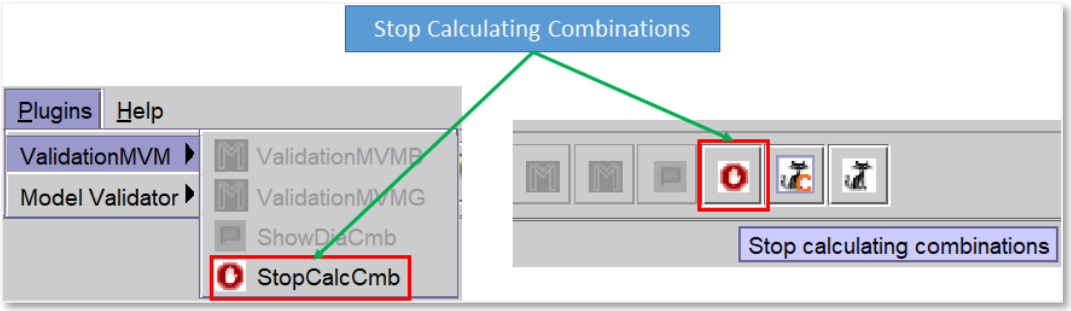
Both methods are available in the menu or toolbar:



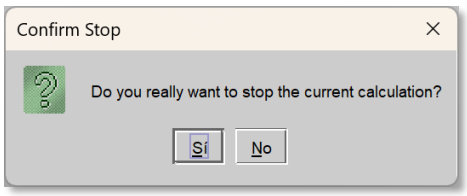
Brute force

When we run this option, the search for all combinations between invariants is launched to internally manufacture the lists of groups of satisfactory and unsatisfactory combinations. Depending on the number of existing

invariants, this search can take a considerable amount of time. In this case, if the user wants, they can stop this search by clicking on the **Stop calculating combinations icon**:



If you click on this option, a message will appear requesting confirmation:

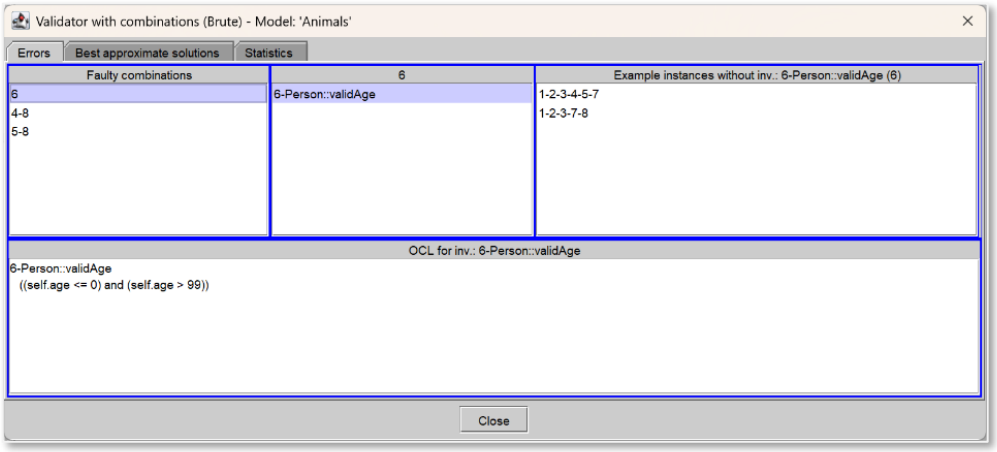


When the search for combinations is complete, the following dialog box appears with the following tabs:

- **Errors**: Displays groups of combinations that fail when active. That is, any set of joins that includes any of the groups that appear on this screen will produce an unsatisfactory instance.
- **Best approximate solutions**: Shows the groups of combinations that can be active simultaneously and that would produce a satisfactory instance.
- **Statistics**: indicates certain statistical information such as the time it has taken the process to find the combinations, the number of calls to the **Solver** or the number of satisfactory and unsatisfactory combinations.

In the title of the dialog box, you can see the selected method (**Brute**) and the name of the model being analyzed (**Animals**)

Errors



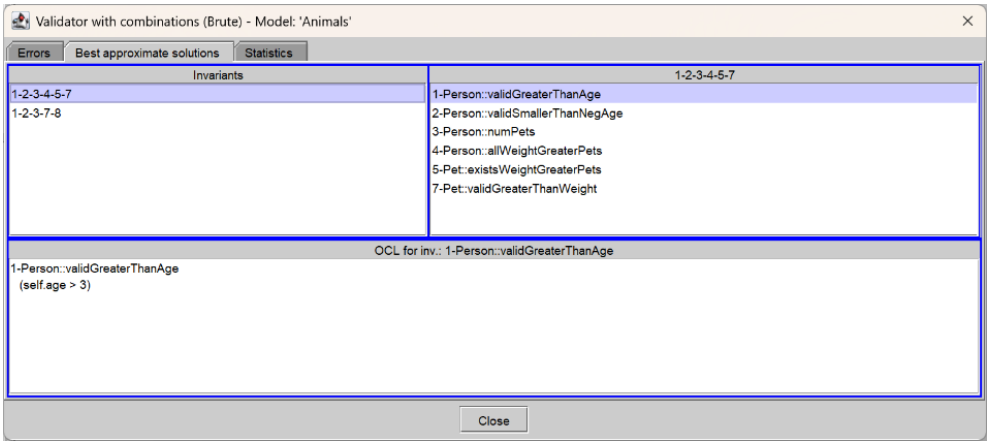
On this screen we can see different blocks that interact with each other so that, when we click on a row in the **Faulty combinations panel**, the rest of the blocks are synchronized and show the detail associated with the selected group.

Panels

- **Faulty combinations**: In this example, we can see that 3 groups of **MUS** (**6**, **4-8** and **5-8**) have been detected. If we click on the first group (it contains the invariant '**6**'), we will see that the panel on the right shows the selected combination '6' as the title and inside it a line for each invariant of that combination.
- '**6**': panel showing all the invariants that make up the selected MUS group ('**6**'). When you select a line from this block, the **instances without inv** and **OCL for inv Example panes** synchronize by displaying information associated with the invariant of the selected line.
- **Example of instances without inv: '6'**: proposes satisfactory combinations that do not contain the selected **MUS** group and that can generate a satisfactory instance by simply double-clicking on any of its lines (e.g. '**1-2-3-4-5-7**').
- **OCL for inv: '6'**: Displays the definition of the selected invariant to have a view of the possible problem to be solved. In this example it is clear that age cannot be simultaneously **<=0** and **>99**.

The **Close** button closes the dialog box and returns to the previous screen.

Best approximate solutions

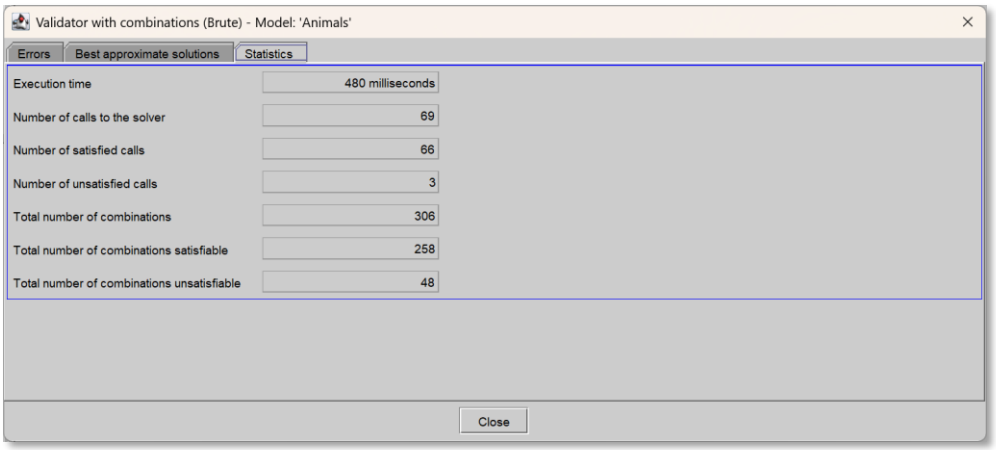


In this tab, we can see the groups of joins that can generate satisfactory instances as long as the rest of the invariants that do not appear are disabled. Note that the groups are ordered from the highest number of satisfactory invariants to the least.

The blocks that make up this tab are:

- **Invariants**: Groups of invariants that generate a satisfactory instance. Similar to the one described in **Errors**, if we double-click on any line of the **Invariants** block, an instance is created and an object diagram opens showing it.
- '**1-2-3-4-5-7**': shows the invariants that make up the group with their name.
- **OCL for inv: 1-Person::valildGreaterThanAge**: Displays the bodyexpression of the invariant selected in the previous block.

Statistics



It displays the following information:

- **Execution time**: The time it takes for the process to complete.
- **Number of calls to the solver**: number of calls that are actually made to the solver.

- **Number of satisfied calls**: solver calls that are satisfactory.
- **Number of unsatisfied calls**: Calls to the solver that are unsatisfactory.
- **Total number of combinations**: Total number of combinations.
- **Total number of satisfiable combinations**: number of total satisfactory combinations.
- **Total number of combinations unsatisfiable**: number of total unsatisfactory combinations.

Greedy

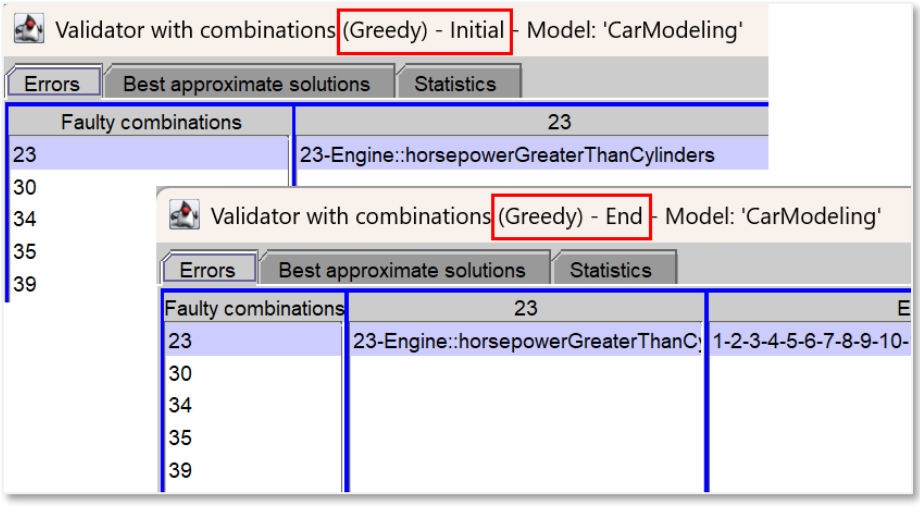
Similar to **Brute force**, the **Greedy method** also searches for **MUS** and **MSS,** but it does so in 2 phases:

- **Look for a first result with satisfactory invariants.**
- **Look for the rest of the pending combinations.**

In the first phase, **Greedy** determines the dependence of each invariant on the others by analyzing attributes and classes involved in it and fabricates a collection of invariants that do not interfere with each other. In this way, almost instantaneously we obtain a first result with which to produce a satisfactory instance.

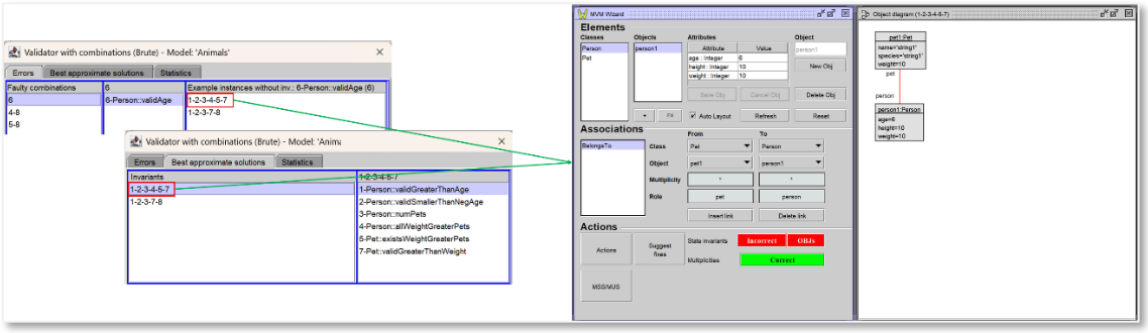
The second phase looks for the rest of the pending combinations until all the possible ones are completed.

Regarding the dialog box shown during the search, it should be noted that when **Greedy** gives the first result, in the title of the dialog, in addition to the **Greedy method,** the word Initial is also shown indicating that it is the first result. When the Greedy combination search is complete, End is displayed.



Creating a diagram from the MUS/MSS dialog

To create an object diagram, simply double-click on one of the combinations shown in the **Errors tab** or in **Best approximate solutions**:

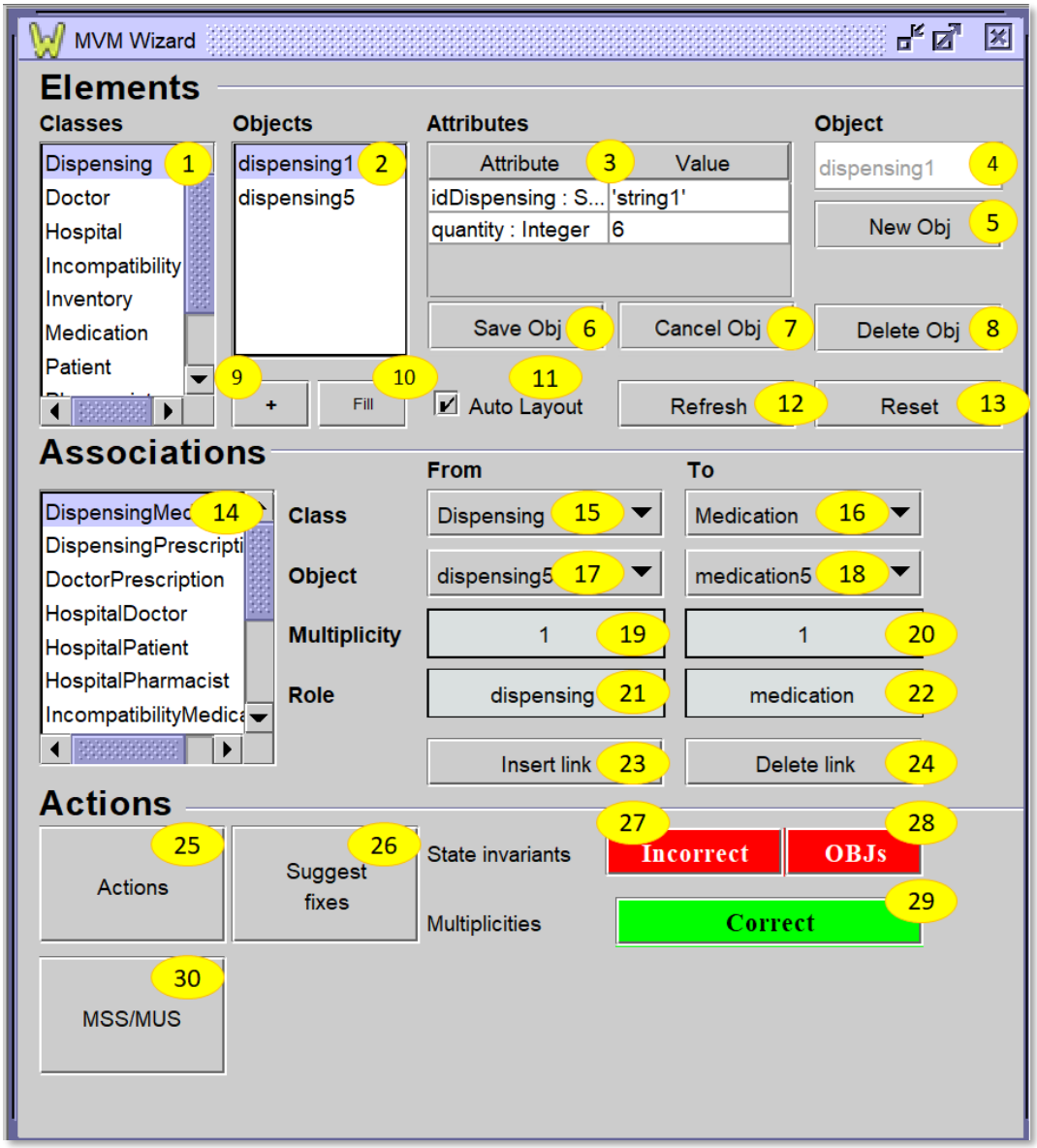


MVM Wizard

This screen is the main screen from which the vast majority of the functionalities contained in MVM can be used or accessed.

It is basically divided into 3 blocks:

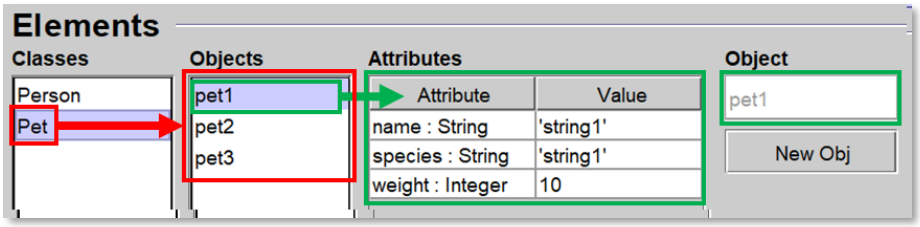
- **Elements**: Manages existing objects in the current instance (additions, deletions, modifications, and queries)
- **Associations**: Manage existing links between objects
- **Actions**: access the utilities for repairing invariants, multiplicities, log of actions performed and consult **OpenAI.**



Below, we detail the purpose of each graphic element.

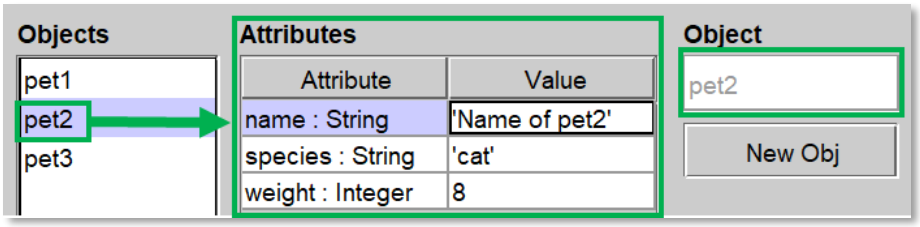
1: Classes

Displays the existing classes in the model. When you click on a class, the **Objects** and **Attributes** blocks synchronize to show the existing objects of the selected class and the attributes and their values of the first object of that class.



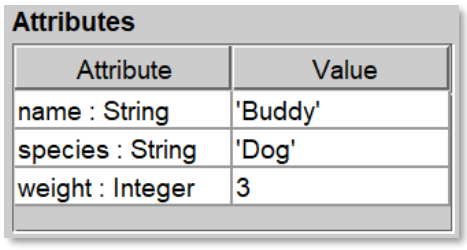
2: Objects

Displays the existing objects in the current instance of the class selected in the **Classes** block. Each time an object is selected, the **Attributes table** displays its corresponding attributes and values.



3: Attributes

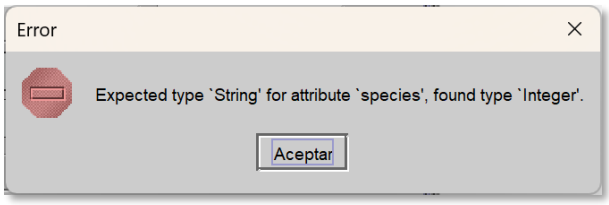
It allows you to visualize and modify the attributes and values of an object. In the case of an existing object, you can modify a value by selecting it and then clicking on the value you want to modify.



The values assigned to the attributes of type **String** must be enclosed in single quotation marks (**'Example String'**).

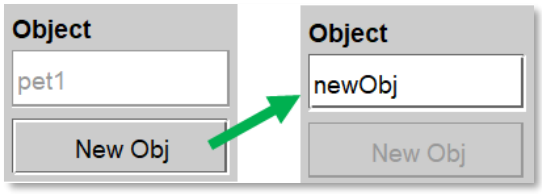
When a value is modified, the **Save Obj** and **Cancel Obj** buttons are enabled to make the changes permanent or leave the object as it was without changing anything respectively.

If you enter a value that does not correspond to the expected type, an error message appears:

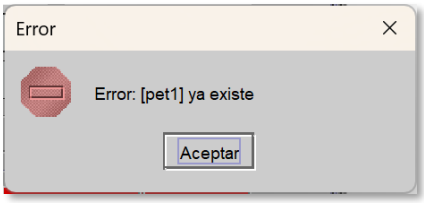


4: Object

This text box is used to display the **ID** of the object that has the focus and is disabled when the treated object already exists. However, when we click on the **New Obj** button, it is enabled so that we can enter a new **ID:**



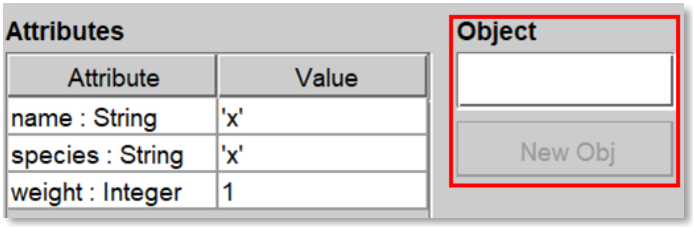
If we enter an **ID** that already exists and click on **Save Obj**, an error message appears:



If we want to leave the object as it was before changing anything, we will simply press **Cancel Obj**.

5: New object

Pressing this button enables the Object text box to enter an **ID** and leaves the values in the attributes so that we can modify only those that are necessary.



6: Save object

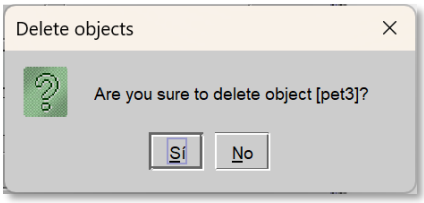
Applies to the instance the modifications made to the object.

7: Cancel object

Leaves the object with the values it had before you modified it and clicked **Save Obj**.

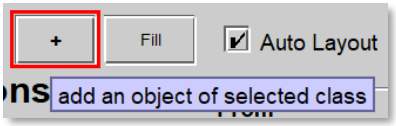
8: Delete object

Allows you to delete the selected object, but first requests confirmation with this message:



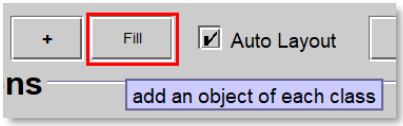
9: +

Allows you to create an object by copying the selected object. The **object ID** will be the name of the class followed by a number that it will get from a sequential number within the class.



10: Fill

Create one object of each class. The **object ID** will be the name of the class followed by a number that it will get from a sequential number within the class.



Initialize attributes depending on their type:

- **String**: 'x'.
- **Boolean**: true
- **Integer**: 1
- **Real**: 1.0.

11: Auto Layout

Enables/disables the feature that automatically places objects on the diagram in a manner spaced apart.

12: Refresh

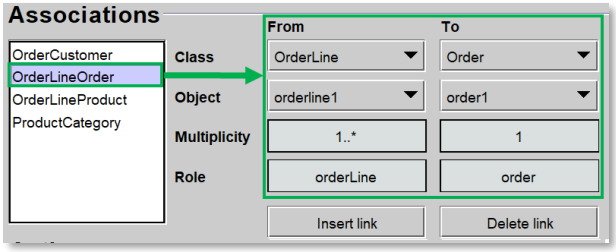
Recreate the elements of the diagram.

13: Reset

Deletes all objects from the instance and flushes the diagram.

14: Associations

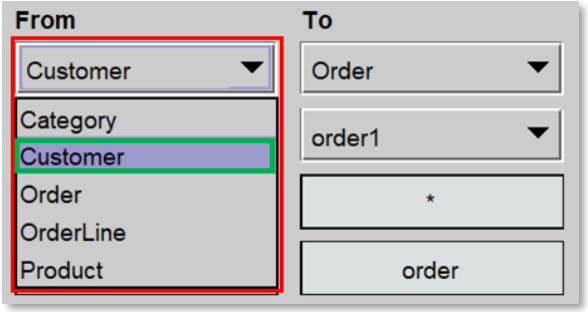
Lists the associations defined in the model.



Selecting an association synchronizes the '**From**' and '**To**' information on the right.

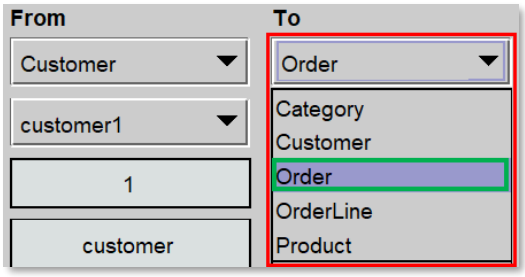
15: From Class

Displays the **participating class** at the origin endpoint.



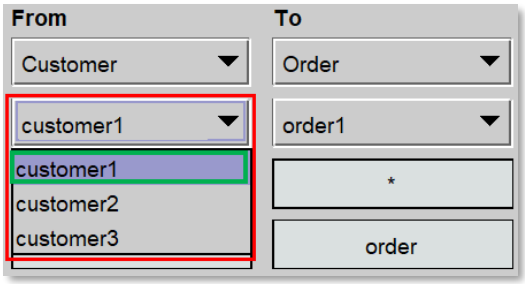
16: To Class

Displays the **participating class** at the end endpoint.



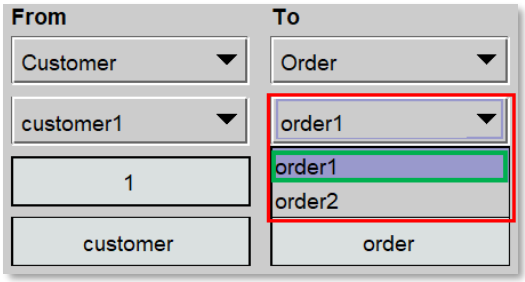
17: From Object

Displays the **participating object** at the source endpoint.



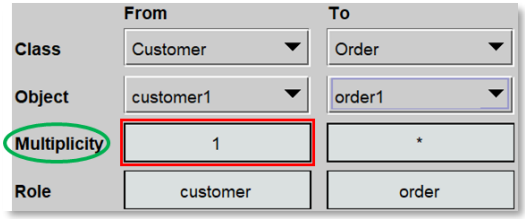
18: To Object

Displays the **participating object** at the **end endpoint**.



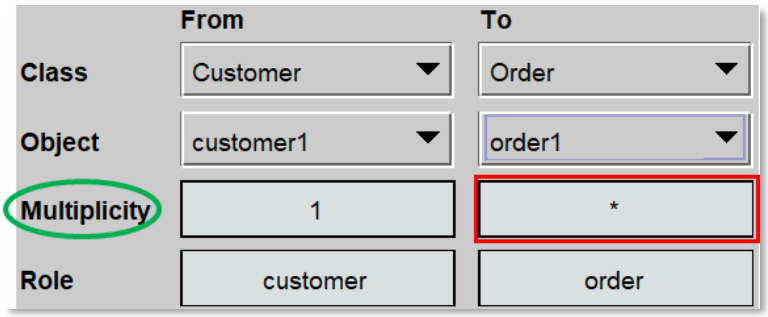
19: From multiplicity

Multiplicity at the **origin** extreme.



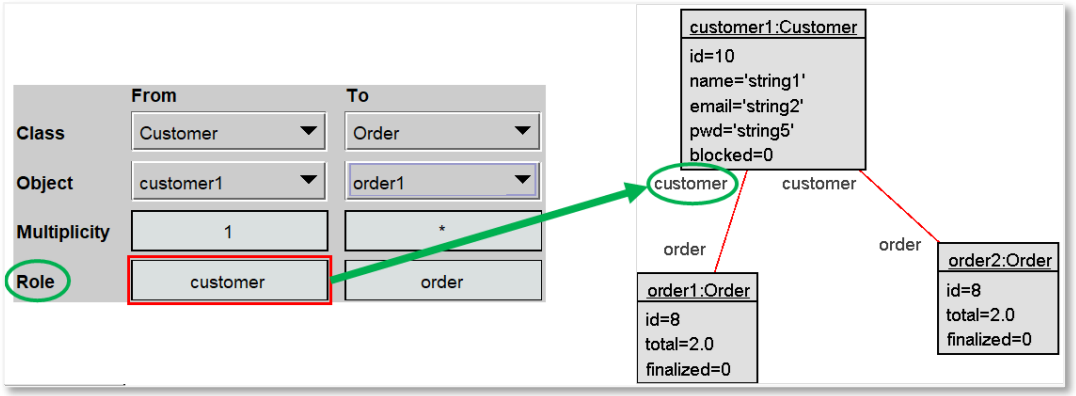
20: To multiplicity

Multiplicity at the **endpoint**



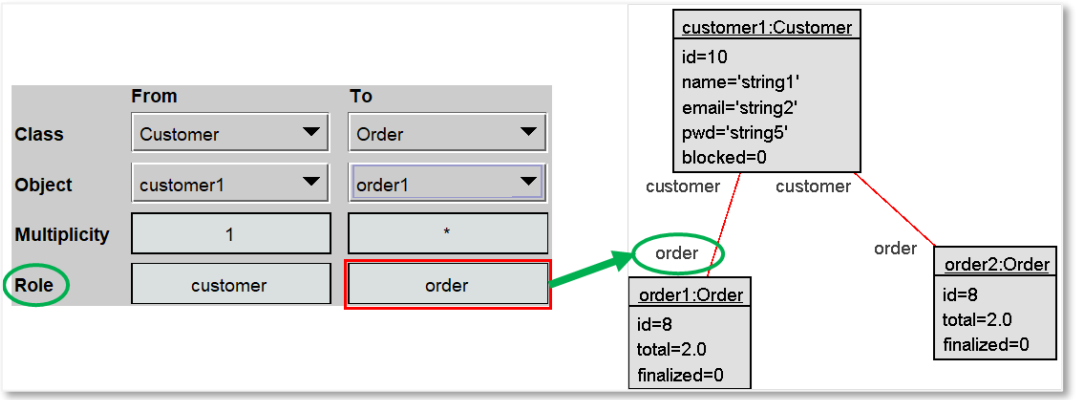
21: From Role

Role on the **origin** endpoint.



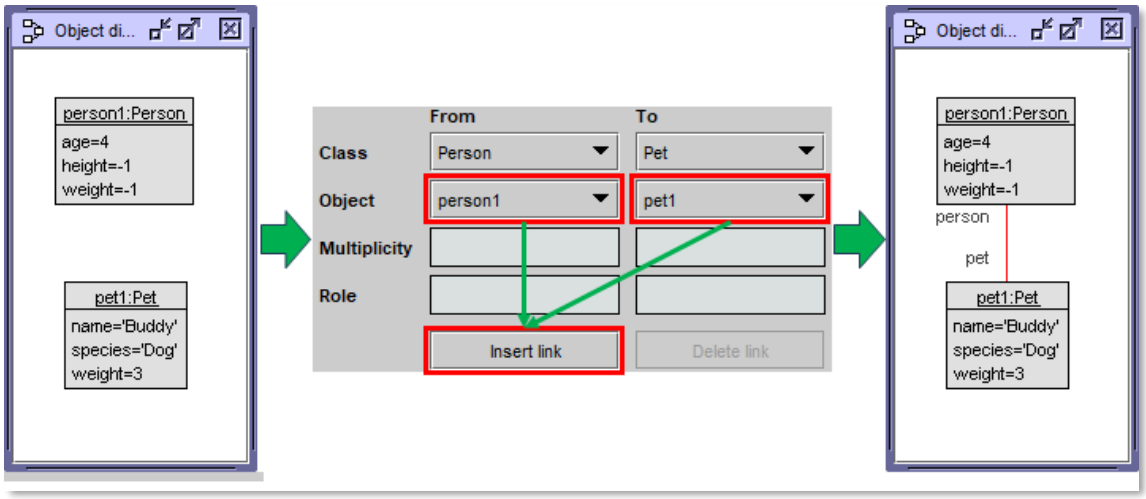
22: To Role

Role at the **end endpoint**.



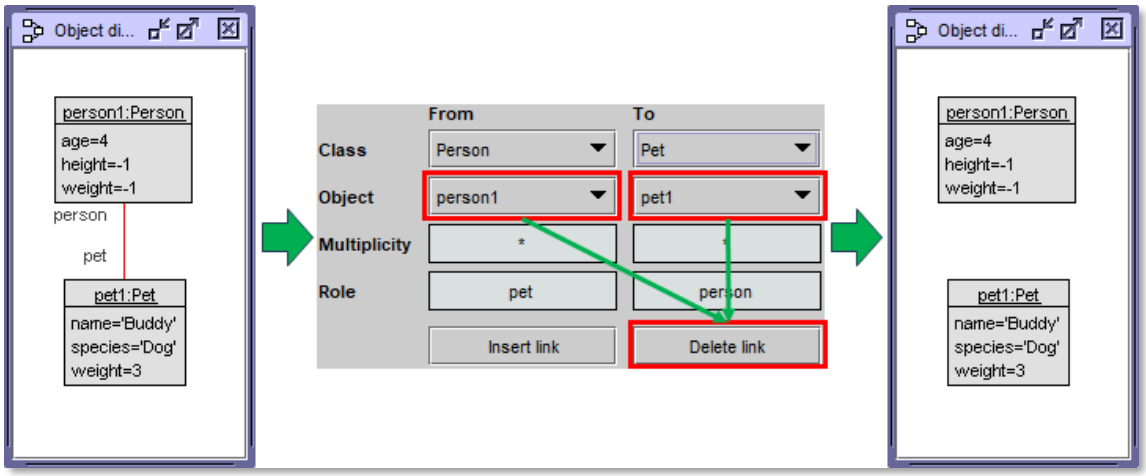
23: Insert link

It allows you to insert a link between 2 objects. To do this, we have to select the source object and the final object and click on the **Insert Link button**:



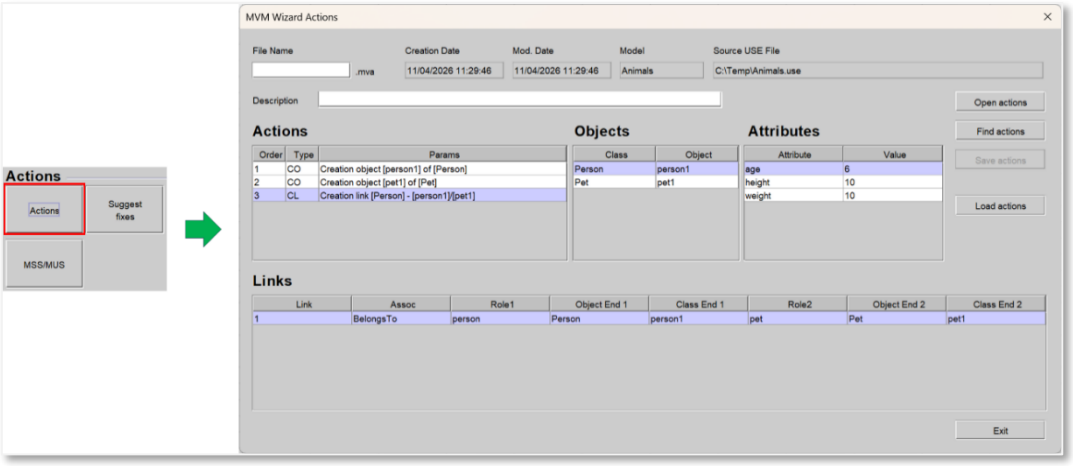
24: Delete link

Allows you to delete a link between 2 objects. To select a link, we can select the objects that are at each end or click on it in the diagram.



25: Actions

This button allows access to **MVM Wizard Actions** , which is the functionality in charge of managing the record of the actions carried out on a given instance.



With this functionality, the user will be able to record a set of actions and retrieve them later to reproduce a given situation on the instance. You can even go back to a specific situation without needing to record any files beforehand.

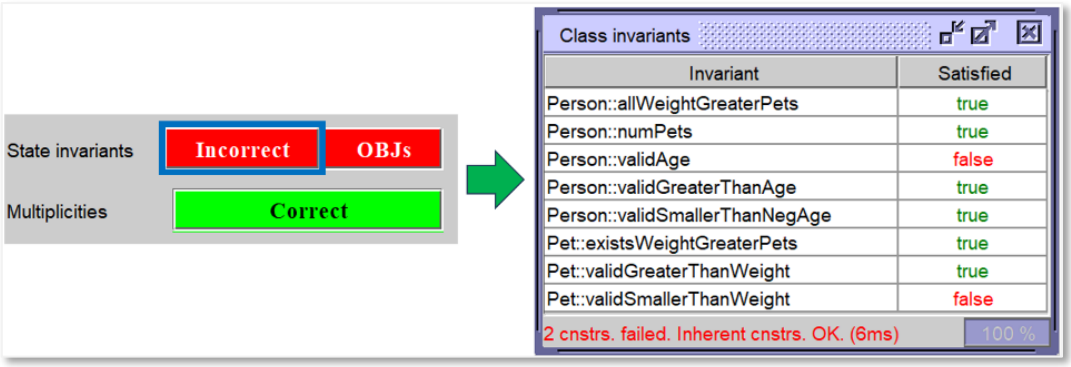
26: Suggest fixes

This button allows **OpenAI** to be invoked to perform a query that allows us to get suggestions to detect errors in the model and correct them.

27: State invariants

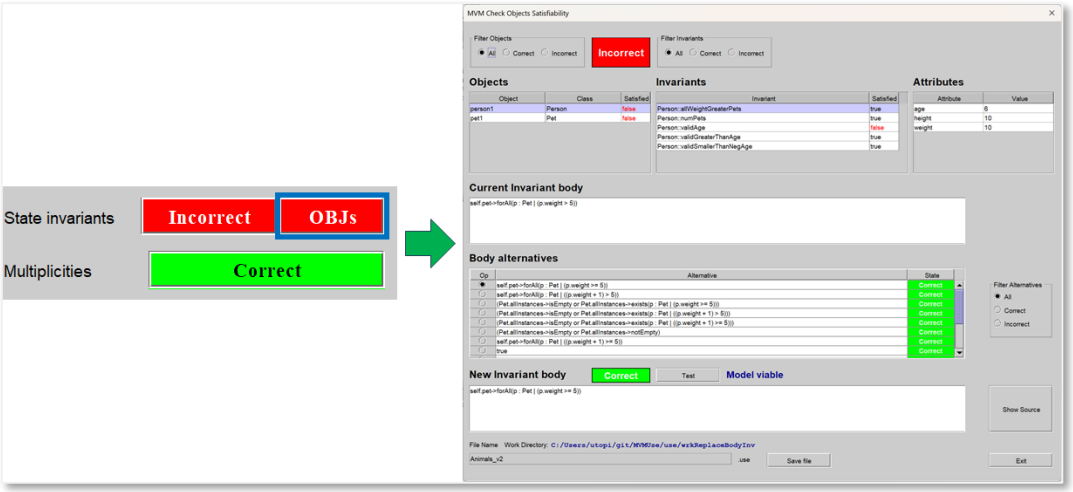
This section allows you to review the status of invariants. The color of the button indicates whether all the invariants are met or not so that, at a glance, we can check the status of the instance as far as invariants are concerned.

If we click on it, a screen appears where we can see which invariants are satisfied and which are not.



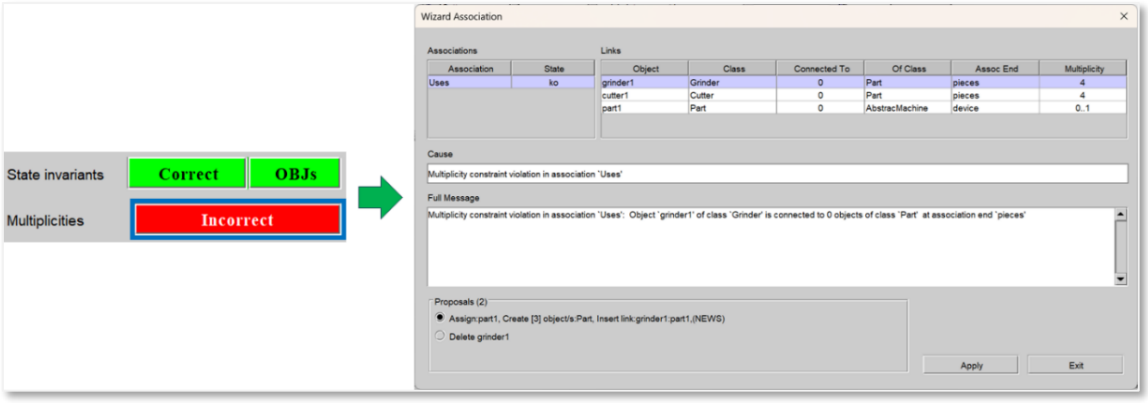
28: OBJs

This button allows access to the functionality that tells us which invariants are failing according to the existing objects in the instance and which are the alternatives proposed for their solution. The color of the button will depend on whether the invariants are satisfied or not.



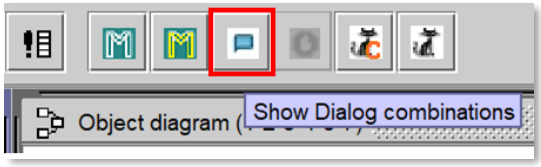
29: Multiplicities

This button gives access to the screen that determines the multiplicity problems encountered in the current instance and helps to solve them by proposing the **creation/deletion** of objects and the creation of links between them.

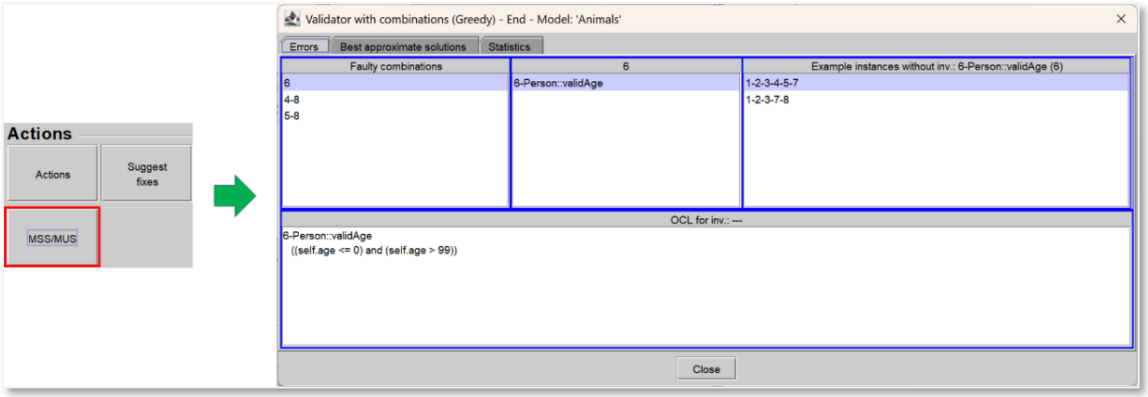


30: MSS/MUS

Using this button, if accessible, we can display the dialog box that shows the previously obtained **MUS/MSS**. If it is not accessible, it means that they have not yet been calculated. It is equivalent to clicking on the icon that we have in the toolbar:



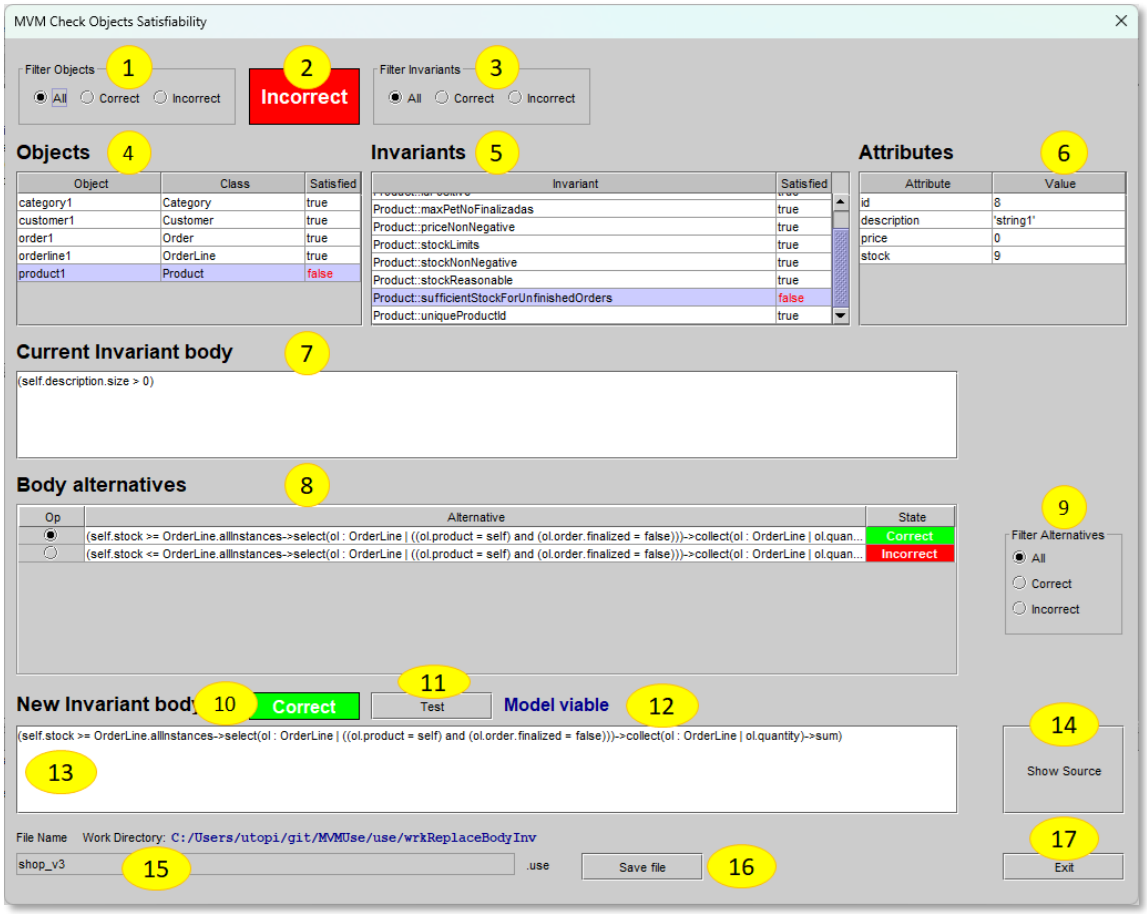
When you click on this option, you will see the dialog associated with the **MUS/MSS:**



MVM Check Objects Satisfiability

This button also changes color depending on whether the invariants are satisfied or not. Clicking on it gives access to the following MVM **Check Objects**

Satisfiability screen:

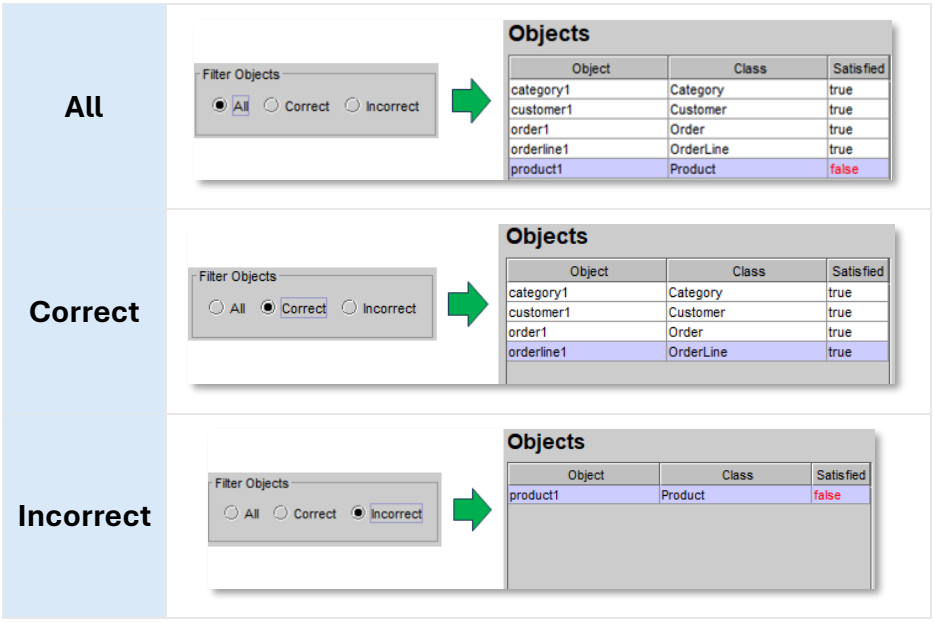


This screen shows the existing objects in the instance and for each object, the invariants in which they participate and also the associated attributes and values. Each time an object is selected, the other blocks are synchronized to display the information associated with it. Therefore, if an object appears as false in the **Satisfied** column, there is surely at least one invariant that is not satisfied for it.

Below, we comment on the graphic elements that make it up.

1: Filter Objects

This group of options allows you to display objects according to the selected option:

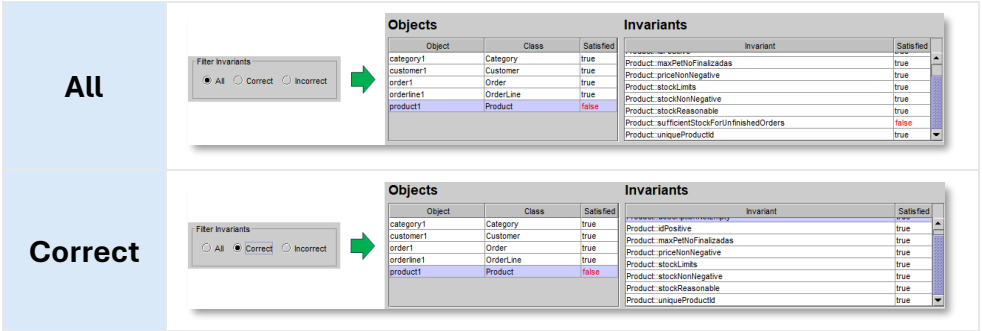


2: Incorrect/Correct

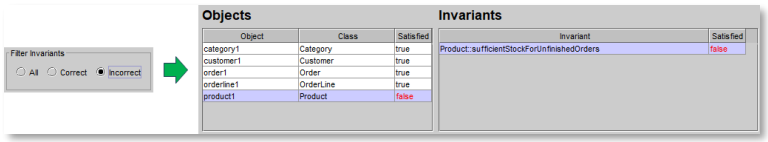
This label shows the general state of the instance so that at a glance it can be easily cataloged:

3: Filter Invariants

When we select an object, the invariant block shows all the invariants in which it participates. In this case, it may be interesting to show them all, or only the correct or incorrect ones.

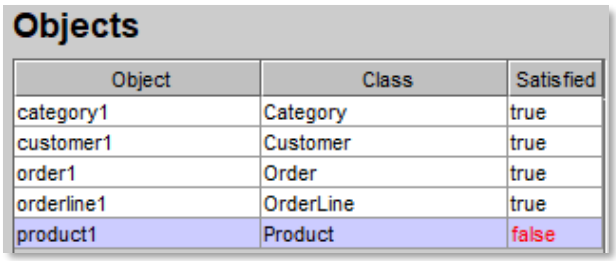


Incorrect



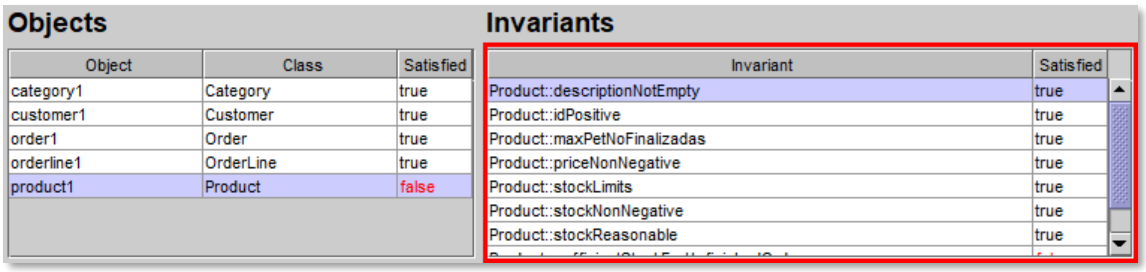
4: Objects

Displays the existing objects on the instance. When clicked, the rest of the blocks are synchronized to show the information associated with it.



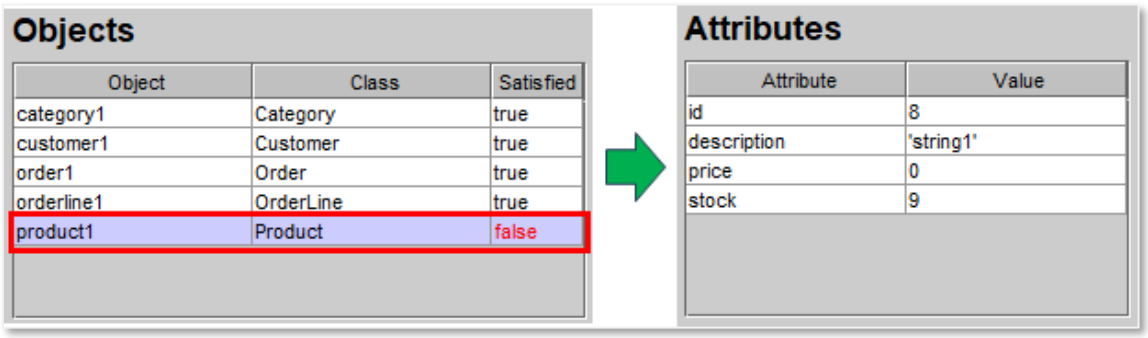
5: Invariants

Displays the invariants associated with the selected object.



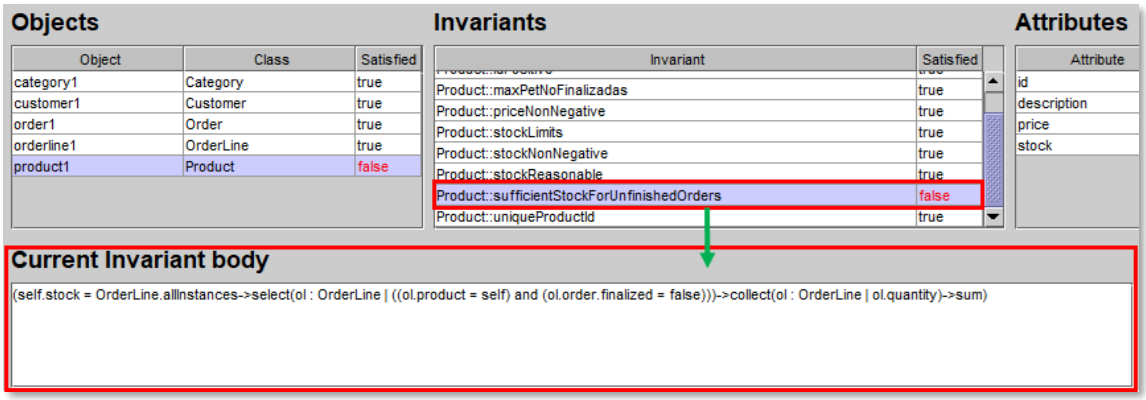
6: Attributes

Displays the attributes associated with the selected object.



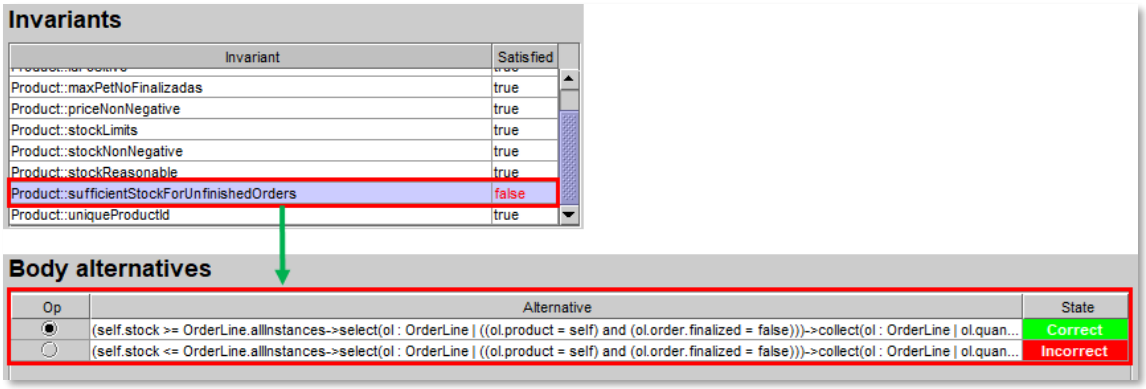
7: Current invariant body

Displays the **bodyexpression** currently held by the selected invariant.



8: Body alternatives

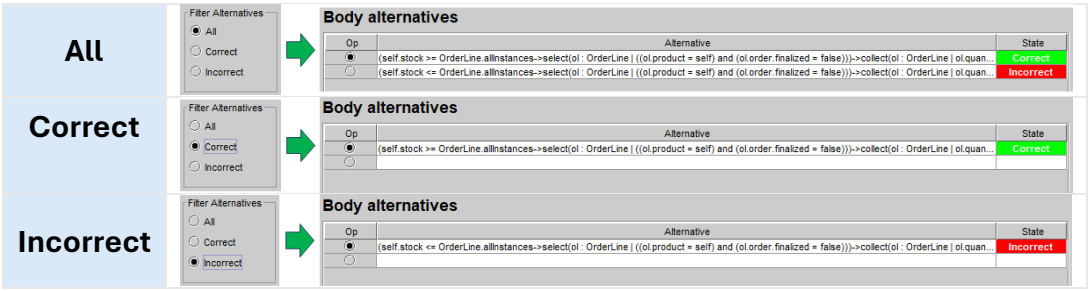
Displays the alternatives that have been calculated for the selected invariant.



For each of the proposed alternatives, the system calculates its satisfactibility.

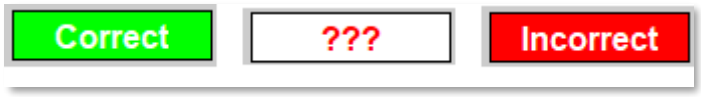
9: Filter Alternatives

It allows you to visualize all the alternatives, only the correct ones or only the wrong ones.



10: New Invariant body - Incorrect/Correct

This label shows the result of the test of the alternative whose definition is found in the text box just below (**13: Body expression**). It can have any of the following values:



The 'questions' are displayed when the text box containing the definition of the bodyexpression receives focus and disappear when it loses it.

11: Test

This button allows you to run the test to check that the **bodyexpression** of the alternative in the text box is correct.

12: Model viable

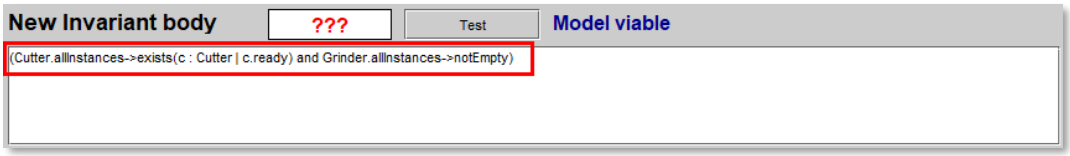
This text allows you to check whether the model definition has a good syntax or not after the replacement of the new **bodyexpression** over the old one.

Examples:

Correct	Incorrect
Model viable	Non-viable model !!! [line 30:0 no viable alternative at input '<EOF>']

13: Body expression

This text box contains the bodyexpression that will replace the current one in the treated invariant in the new model. When we click on an alternative, the associated expression is placed in this text box. If the user wants, they can modify it manually and that's when questions appear on the label above this text box.

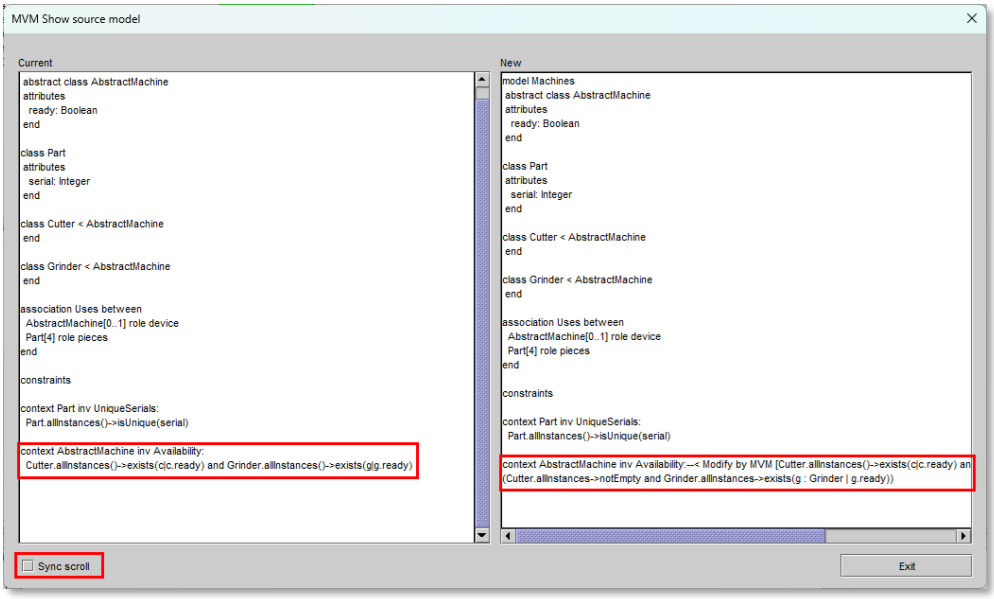


14: Show Source

It shows a screen with 2 parts:

- The **current one**
- the **new** one as it would be after replacing the invariant with the alternative.

This screen looks like this:



In it you can see the old definition of the invariant **Availability**, and the new one. Note that the old definition is converted to a comment (--) for the record, and the new one is added below.

It is possible to synchronize both panels (**Current** and New) by activating the **Sync scroll check**.

15: File Name

Indicate the name of the file to be proposed. The first proposal will be to add the suffix **_vX** with **X** being the version number that already exists. If no version exists, the suffix will be '_v1'. If a previous version exists, such as **_v4**, the proposed version will be **_v5**.

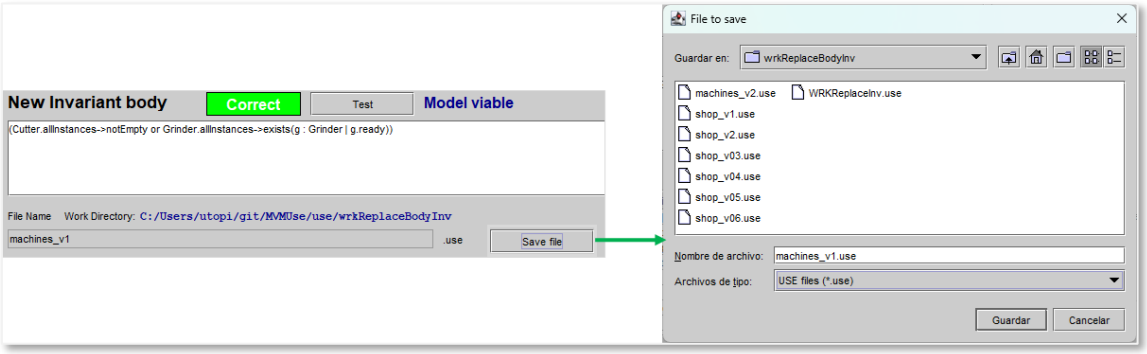
The default target directory will be **wrkReplaceBodyInv** which will be inside the working directory where the application is running.

If this directory does not exist, it will be created automatically.

16: Save file

This button allows you to save a file with the new model in which the previously selected invariant has been modified, replacing its definition with the new definition entered in the New **invariant body text box**.

Clicking on it will open the **File to Save** dialog proposing the default directory and name, but the user can modify what is necessary according to their criteria:



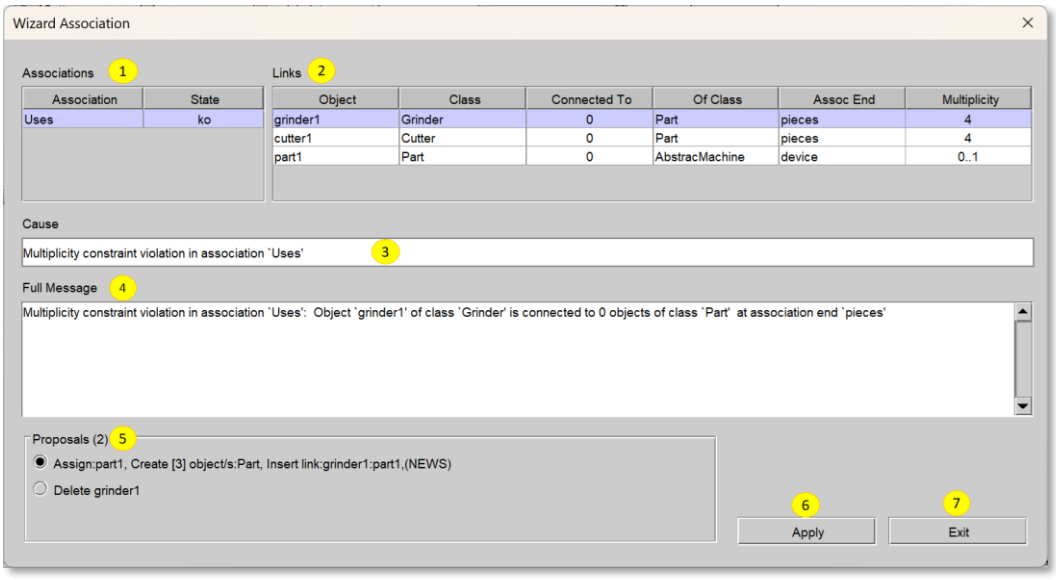
17: Exit

Allows you to close the screen in progress and return to the previous one.

Wizard Association

This button gives access to the screen that helps solve problems of multiplicity by proposing the **creation/deletion** of **objects** and the **creation** of **links** between them.

The main screen of this utility is composed of the following graphic elements:



1: Associations

Displays the list of associations that have a problem. When you click on an association, the rest of the blocks on the screen are synchronized to show the information of the links associated with it.

2: Links

Displays objects that are potentially related to the selected association. When you click on a link, you will see that the rest of the blocks (except **Associations**) synchronize to show information related to the selected link.

3: Cause

It shows the **cause** why the link is failing.

4: Full Message

Displays the **full error message** associated with the selected link.

5: Proposals

It shows the proposals envisaged to try to solve the problem of multiplicity.

The proposals that are made are based on the following actions:

- **Create elements** that help respect the cardinality of links
- **Assign elements** to create links
- **Crear links**
- **Delete Items**

6: Apply

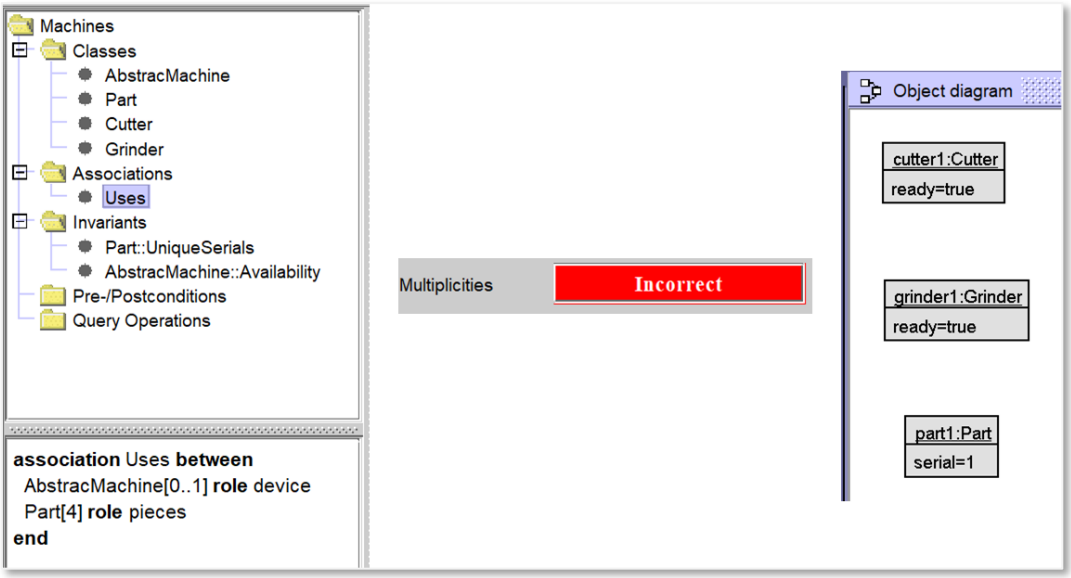
This button is responsible for applying the selected proposal. Once we click on **Apply**, the dialog closes and the proposed actions are executed.

7: Exit

Closes the dialog box without taking any action.

Example multiplicities

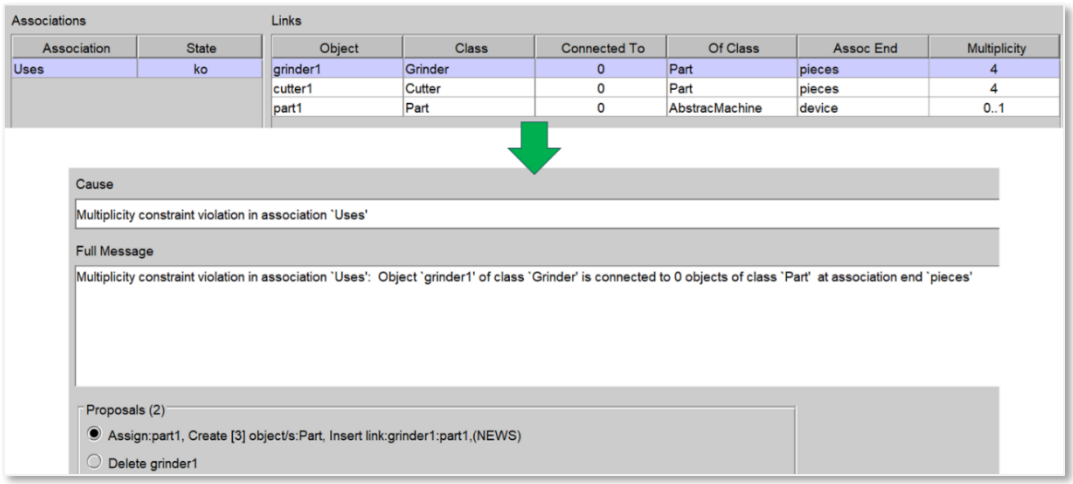
Suppose we have an instance with an object of each class:



The **AbstractMachine** class is an abstract class that cannot be created directly by an object.

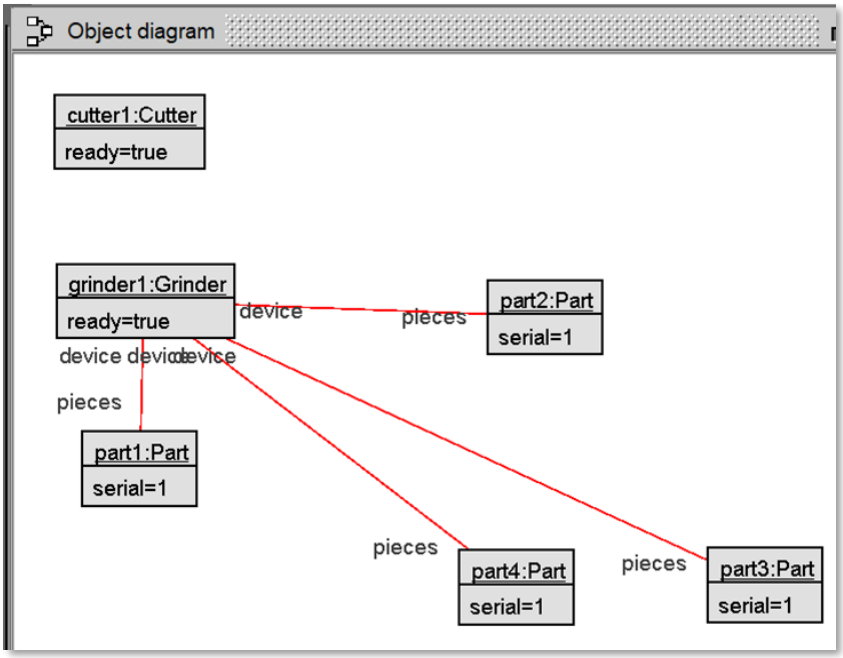
We note that **Multiplicities** is **Incorrect**.

If we click on the **Incorrect** button to access the **Wizard Association screen**, we will see that it indicates that the **grinder1** object is not connected to any **Part** object and therefore the **Uses** association fails.



We also see that one of the proposals is to assign **Part1** (since it is not associated), create 3 objects of type **Part** and insert a link between **grinder1**, the **part1** object and the new (**NEWS**) objects that it creates.

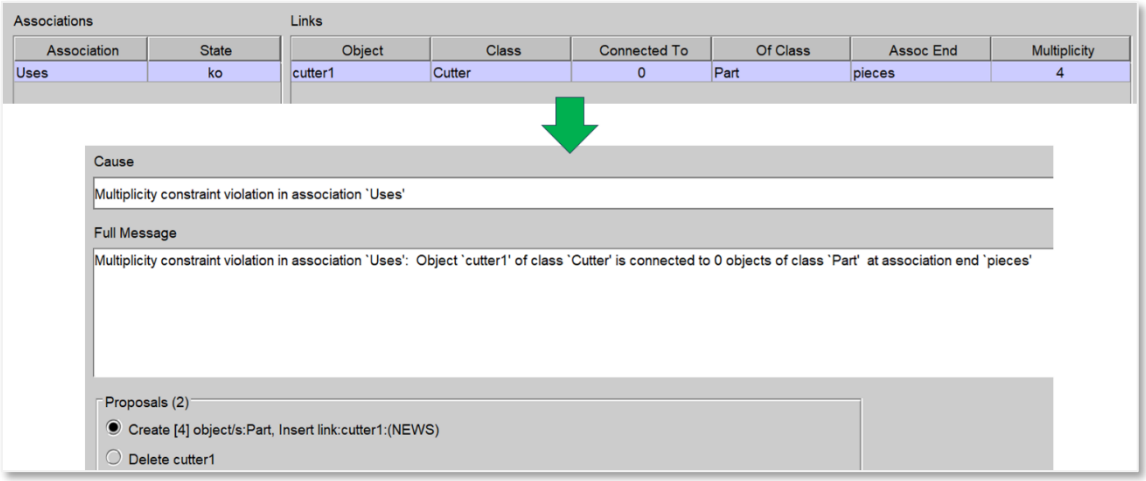
If we select this option and click on **Apply**, we will get the following diagram of objects:



We can see that the topic of multiplicities is still in an Incorrect state.

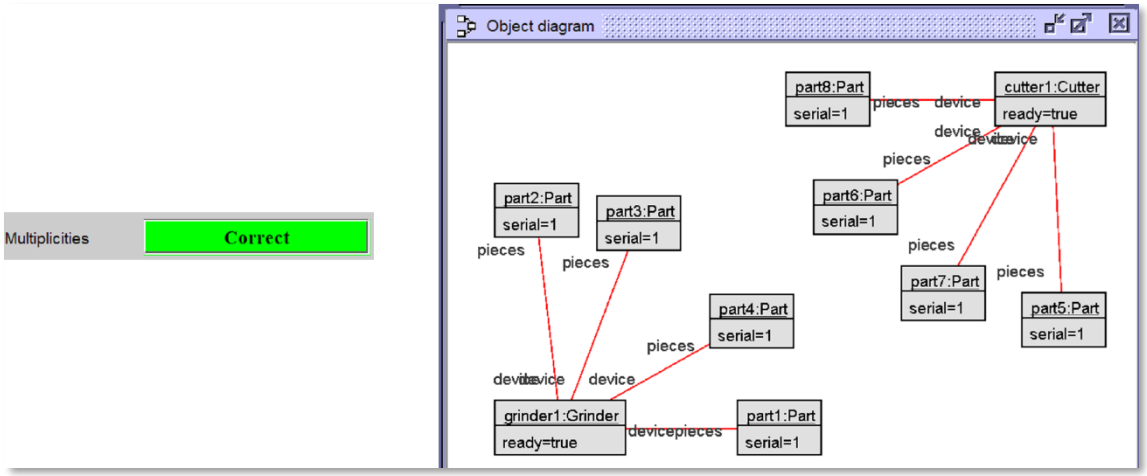
We can click on that button again to continue with the solution of these multiplicities and we see that there are fewer to solve.

This time, multiplicity fails because the object **cutter1** is not connected properly.



Similar to the previous case, we will accept the creation of **4 objects** of type **Part** and the creation of the link of the same (**NEWS)** with **cutter1**.

Click on **Apply** and we will get the following diagram of objects:

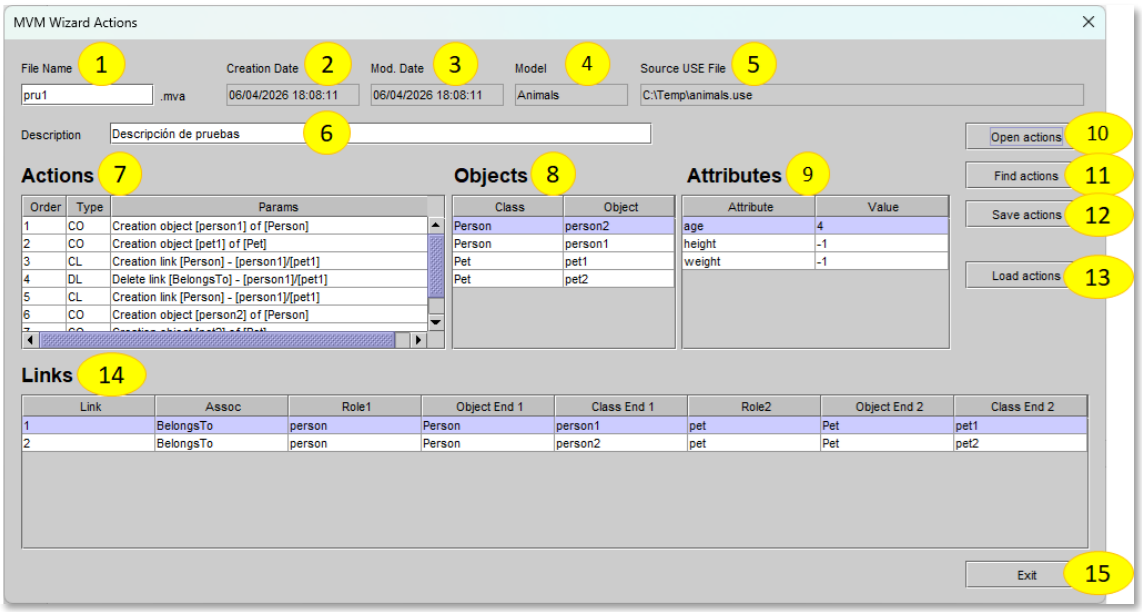


We can see how the problem of multiplicities has already been solved.

Actions

This button allows access to **MVM Wizard Actions**. This utility is responsible for managing the record of the actions performed on a given instance. In this way, we can record each action and return to a certain point in the sequence in case we want to undo any of them. It is also possible to store all actions with a name so that you can retrieve it later and replay them all at once instantly.

Next, the main screen is shown and its graphic elements are commented on.



1: File Name

Name of the file that will store our sequence of actions. This type of file is stored with the **.mva extension**.

2: Creation Date

Date and **time** of **file creation** .

3: Mod. Date

Date and **time** of the last **modification** of the file.

4: Model

Model on which the actions have been carried out.

5: Source USE File

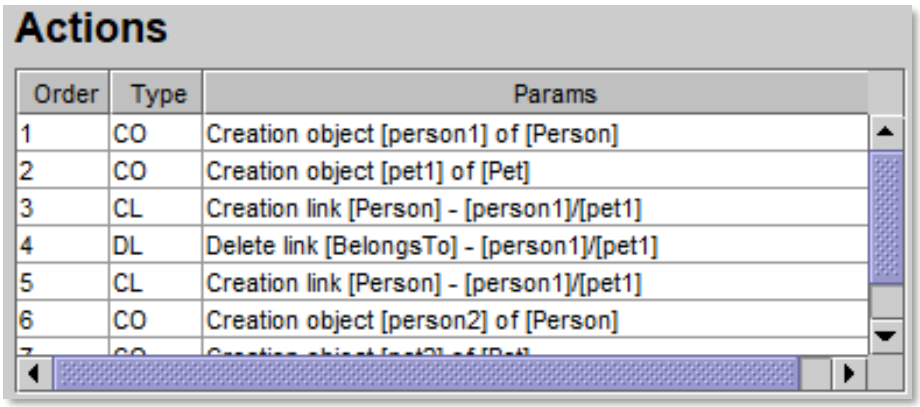
File used in the MVM session and containing the model on which the actions have been performed.

6: Description

Description of the sequence of actions.

7: Actions

Actions performed on the instance listed in the order they were performed.

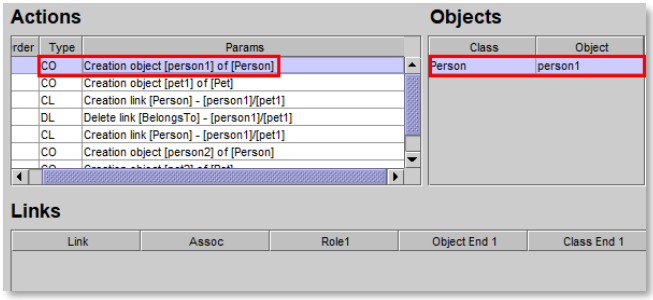
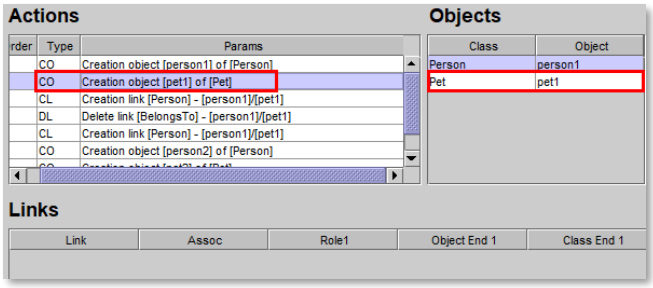
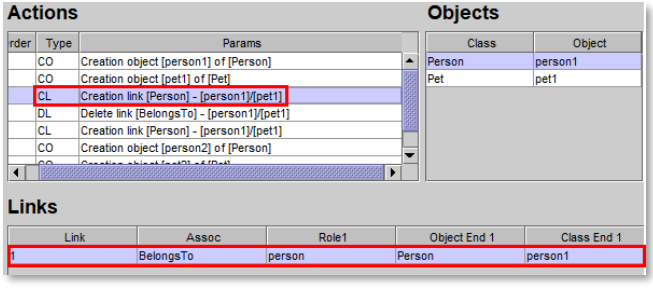


The types of actions are as follows:

CO	Create object
CL	Create link
DO	Delete object
DL	Delete link
MA	Modification object

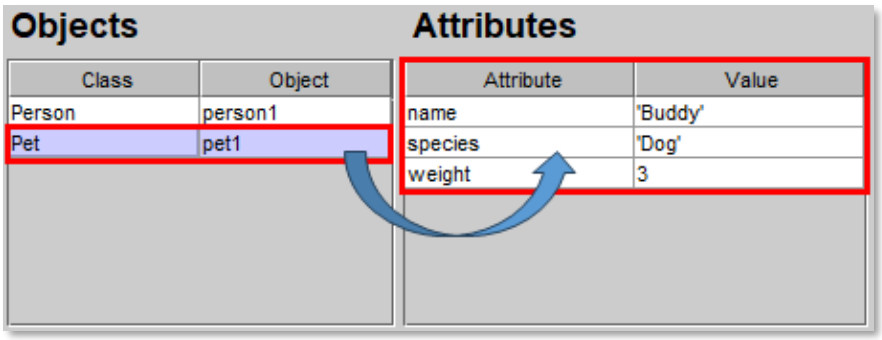
When an action is selected, the **Objects** and Links blocks are synchronized to show the objects and links that exist after the action n is performed.

We can check the elements that are created by clicking on the first 3 actions:

Action	Objects & Links
1	
2	
3	

8: Objects

Displays existing objects after the selected action has been performed. When you select an object, the **Attributes block** displays the values associated with each attribute of the object:

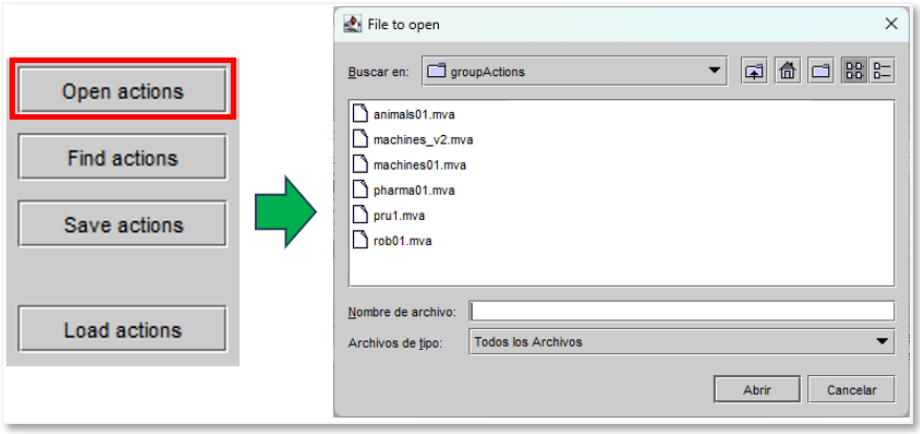


9: Attributes

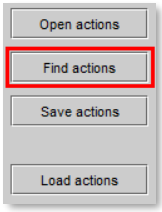
Displays the value of each of the attributes of the object that is selected. See figure shown in the previous section (**8: Objects**).

10: Open actions

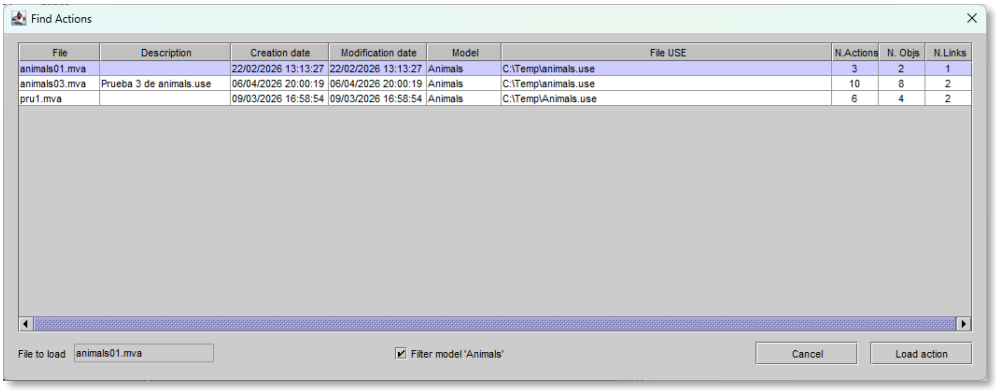
This button allows you to open a file of sequences of actions.



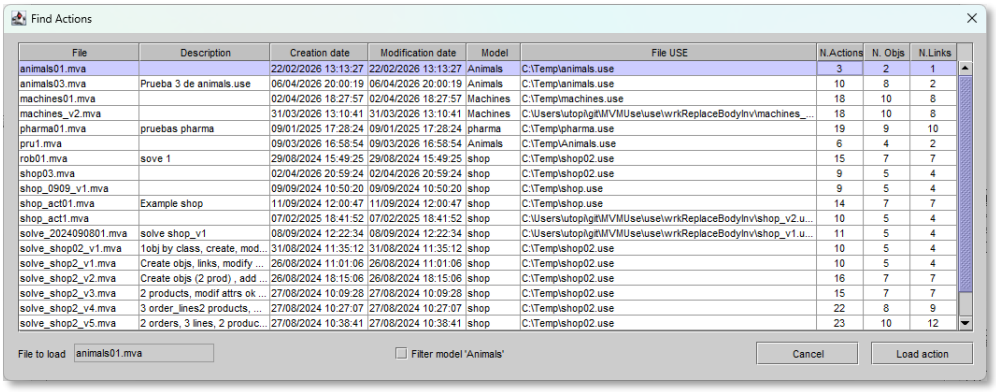
11: Find Actions



This button allows you to search for files with sequences of actions:



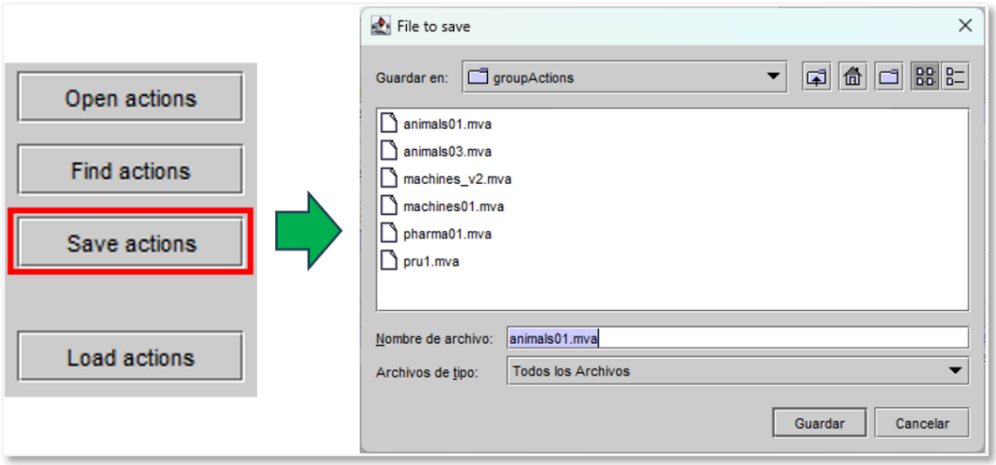
By default, it uses the model's name to filter the files to be searched, but if we uncheck the **Filter model** '**Animals**' check, it shows all the existing files:



To load a file, just click on the line that contains it and click on the **Load action** button or double-click on that line.

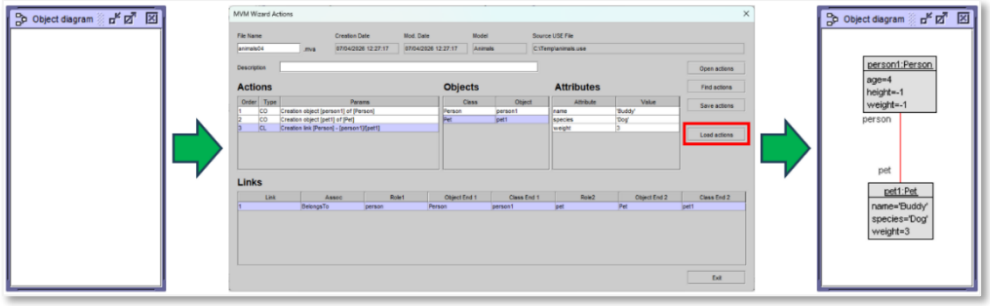
12: Save actions

It allows you to save a file with the actions carried out so far on the instance.



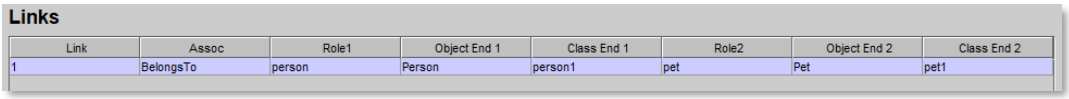
13: Load actions

This button allows you to create an instance with the elements indicated in the associated action. If a list of actions has for example 10 actions, but we select action 3, only the first 3 actions will be executed. This allows you to roll back the instance's situation to a certain point by undoing all subsequent instances.



14: Links

Displays the existing links in the instance after the execution of the selected action.



Links

Link	Assoc	Role1	Object End 1	Class End 1	Role2	Object End 2	Class End 2
1	BelongsTo	person	Person	person1	pet	Pet	pet1

15: Exit

Closes the dialog box.

Suggest fixes

Using this button, we make a query to **OpenAI** asking it mainly to help us determine what errors the model may contain and to tell us how we could solve it.

In the consultation we may provide you with certain information such as **INPUT** in case we have already calculated or not the **MUS/MSS** previously or we already have a collection of objects and links.

To provide OpenAI with the necessary context, the query will be performed with a block of text with the following structure:

- **PROFILE**: contextualize a OpenAI
- **TASK:** lists the actions to be performed
- **INPUTS:** information we provide to you
- **OUTPUTS:** information we expect to receive and the required format
- **REMARKS:** Considerations for refining your query

With this structure, we provide **OpenAI** with context, a set of tasks to perform, a set of information to analyze, and request an output to show to the designer and to perform other actions, such as creating a new instance as a result of the query. Obviously, we refined our query by adding additional instructions that improve the final answer.

Below, we detail each of the blocks that make up the query.

Profile

First, we contextualize **OpenAI** and indicate what its main role will be in this consultation.

We indicate the following:

```
1. You are a software modeling assistant, expert on
software design, development and debugging.
2. You provide concise, concrete and actionable feedback
about software design errors.
```

Task

We list the set of tasks that OpenAI must perform using the provided information to obtain an accurate answer. We indicate the following:

```
"Given a UML class diagram annotated with OCL invariants
that is inconsistent, you will:
1. Identify the invariants that cause the inconsistency.
```

```
2. Review the properties file to see if the specified
ranges are correct, and if not, suggest values to change.
3. Modify ONLY the invariants that MUST be changed to
make the model satisfiable.
4. Calculate the MUS/MSS if this prompt does not provide
it later.
5. Produce a JSON output with Properties, Objects, Links,
Comment and ChangedInvariants.
This explanation should be usable by a software engineer
to locate and correct the defects in the model."
```

Inputs

The inputs provided may vary, although we always provide information about the definition of the model to be analyzed and the contents of the property file containing the values used by the **Solver** included in the **USE** tool to describe the ranges of possible values that will be used in the proposed solution.

Therefore, the inputs we provide are as follows:

- **Model**: The contents of the model definition file
- **Properties**: The contents of the file defining the ranges to be used in the solution search
- **Invariants**: invariants of the model and its expressions
- **Others**: Optionally we can provide **MUS,** MSS**,** and a list of objects and links

We have indicated the following:

```
Model
   • Model: The UML model definition provided in the
     format of the USE tool UML-based Specification
     Environment) developed by the University of Bremen.
Model definition: <model definition in text format>
```

```
Properties
   • Properties: A textual description of the range of
     allowed values for each attribute in the model and
     the range on the number of objects per class and the
     number of links per association.
Properties definition: <properties definition in text
format>
```

```
Invariants
   • Invariants: A summary of the invariants in the
     model, extracted from the USE definition model, and
```

```
    assigning an integer index to each invariant that
    will be used to refer to the invariant.
  • List of invariants: <List of invariants of the
    model>
  • Original Invariants: A JSON array containing the
    exact original text of each invariant.
Original Invariants definition: <List of invariants with
their bodyExpressions>
```

```
Others
Optionally we can add the next groups of information:
  • MUS/MSS found so far: MUS and MSS groups that we
    know at the time of the query, since depending on
    the search method, results may still be being
    searched in the background.
  • List of MUS: <List of MUS>
  • List of MSS: <List of MSS>
  • List of objects: <List of objects existing in the
    current instance>
List of links: <List of links existing in the current
instance>
```

Outputs

This block aims to tell **OpenAI** what we should consider in the answer and how we want to obtain it in terms of the format and structure of the information.

Basically, the structure we require is:

- Comments
- Updated list of objects and links
- Modified Invariants

We have indicated the following:

```
Comments
1. Explanation:
1.1. A brief discussion of the inconsistency problems in
the model.
1.2. For each problem, the model should identify the
invariants involved and outline a potential way to solve
the inconsistency
1.3. Do not provide suggestions like "this value should
be valid"
1.4. Instead, provide informative statements like "this
value should be larger\", "this values should be within
```

the following range\", "the value of attribute X should be different to the value of attribute Y", "this value should be unique", etc.
1.5. Each sentence must be on its own line.

Updated list of objects and links
2. Updated list of objects and links:
2.1.1 If the input included an instance of the model, provide a modified list of objects and links, with minimal changes, such that the corresponding instance satisfies all invariants and graphical UML constraints.
2.1.2 Once the list of objects has been compiled, review it again in case any more need to be created and in case there are any classes without objects.
2.2. If any object has an attribute whose value violates an invariant, please suggest a new value for that attribute and include it in the list of objects you provide at the end.
2.3. Please ensure that all objects involved in any link of any association are part of the proposed object list. If an object does not exist in the object list, do not use it in any link.

Changed Invariants
3. ChangedInvariants:
A JSON array containing ONLY the invariants whose text is actually modified.
You MUST follow these rules:
3.1. You MUST compare each proposed invariant with its corresponding entry in OriginalInvariants tag.
3.2. If the proposal is the same as the original (both textually and semantically), it MUST NOT include that invariant.
3.3. You MUST modify an invariant ONLY if it is impossible to satisfy the model without modifying it.
3.4. If the MUS contains invariants that CAN be satisfied by adjusting objects or properties you MUST NOT modify those invariants.
3.5. If the MUS contains invariants that CANNOT be satisfied without modifying them, you MUST generate a corrected version.
3.6. The corrected version MUST be syntactically valid OCL.
3.7. ChangedInvariants may be empty IF AND ONLY IF no invariant requires modification.
3.8. If you mention in the Explanation that an invariant must be changed, you MUST include it in ChangedInvariants.

3.9. Retain the numbering indicated in each invariant and indicate it in the results table.
Example format:
"ChangedInvariants":[
{
"name":"1-validAge",
"original":"self.age <= 0 and self.age > 99",
"proposal":"self.age >= 0 and self.age < 99"
}
]

Remarks

Finally, we added a block with several considerations to refine the query and force certain results to make the result more accurate.

We indicate the following:

1. Please return only the JSON structure without any additional explanation, since the result must be delivered to an application. Therefore, the JSON structure must contain the following parts:
1.1 Properties: properties to be modified. The properties tag only refers to the properties that exist in <NameProperties> and they should review the range limits to see if it is necessary to modify them to satisfy the invariants.
1.2 Objects: A list of objects with their corresponding values as if the modified properties had already been applied. Please return the same field names indicated in the request corresponding to each class. Please ensure that when defining objects, you normalize the output using the tags 'class', 'name', and 'attributes'. If more objects are needed to satisfy the multiplicity of the association links, especially those multiplicities that require an endpoint, please indicate that these objects should be created and propose a sample in the list of proposed resulting objects.If you don't need to change your values, don't change them. However, if there is a value that violates an invariant, it suggests changing it.
1.3 Links: For each link, specify the fields: codeLink, end1Class, end1Object, end1Role, end2Class, end2Object, end2Role, nomAssoc.
Example: codeLink="1" | end1Class="Person" end1Object="person1" "end1Role="person" end2Class="Pet" end2Object="pet1" end2Role="person" nomAssoc="BelongsTo"
For each association, respect its multiplicity. That is, in a multiplicity of 4, it verifies that there are 4 links.

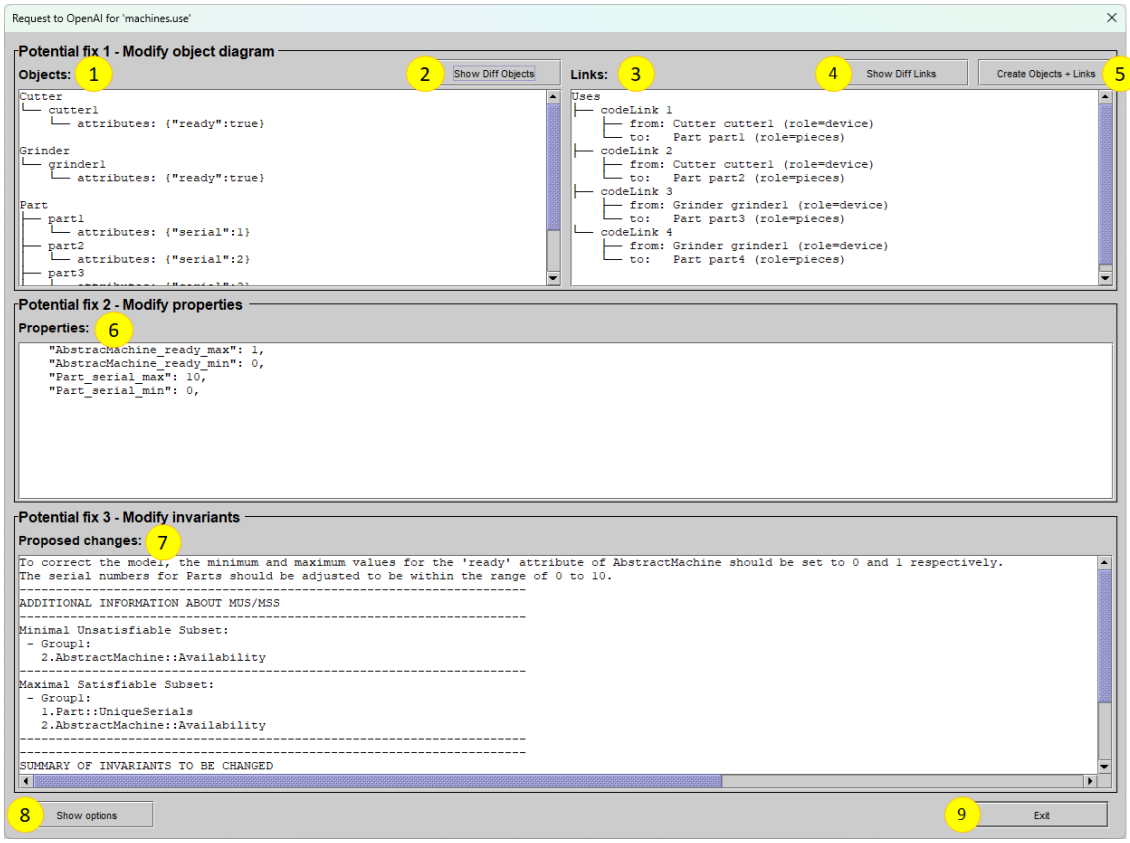
1.4 Above all, make sure that all links of associations only use objects indicated in the previous object list.
1.5 Comment: Explanation of 'how to correct the model', taking into account the properties that have been indicated for correction as if they were already corrected. Focus only on modifying invariants (not objects or properties).
The explanation should not be in the past tense but in the present or future tense. The only tags to return should be: Properties, Objects, Links, Comment and ChangedInvariants.
Do not omit the creation of objects or links.
If the ChangedInvariants tag contains modified invariants, don't forget to include the invariant number provided in the initial list.
Return ONLY valid JSON.
Do not include markdown.
If MUS exist, then ChangedInvariants must always exist.
Return each sentence on a separate line, using a line break between sentences.
For example:
Input: Sentence1. Sentence2. Sentence3.
Output:
Sentence1.
Sentence2.
Sentence3.

If not provide MSS/MUS…
1.5 Include MUS and MSS in the Comment tag using EXACTLY this format:\n

```
     ADDITIONAL INFORMATION ABOUT MUS/MSS
     Minimal Unsatisfiable Subset:
      - Group1:
        1.invariantX
      - Group2:
        2.invariantY
        3.invariantZ
     Maximal Satisfiable Subset:
      - Group1:
        4.invariantA
        5.invariantB
        6.invariantC
      - Group2:
        7.invariantD
        8.invariantE
      - Group3:
        9.invariantF
       10.invariantG
```

(UI – Suggest fixes)

The main screen of this functionality looks like this:



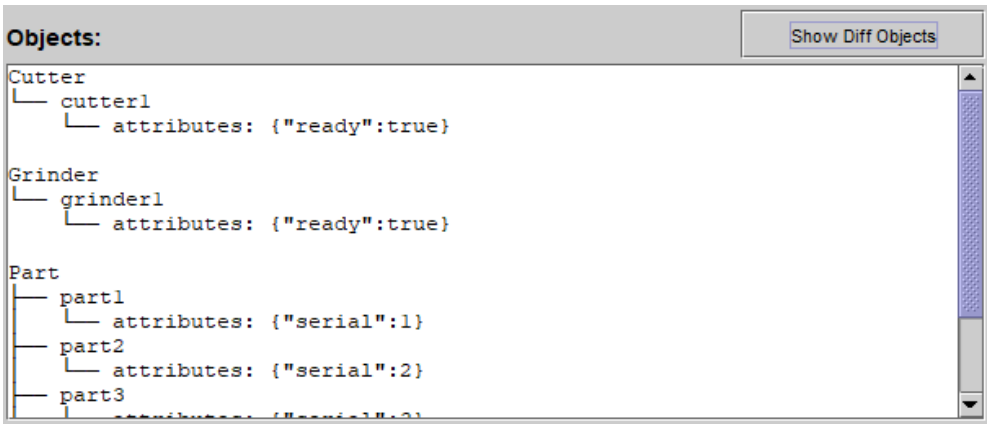
It basically has 3 blocks of suggestions:

- **Potential fix 1 – Modify object diagram**: Indicates the objects and links that should exist in the instance. If in the query we have provided a list of objects and links in the **INPUTs** block, it will make suggestions on how to modify them to improve the instance. If we haven't provided any lists, it will suggest creating a set of items to instantiate.
- **Potential fix 2 – Modify properties**: Proposes the modification of certain parameters contained in the model properties file (<ModelName>.**properties**).
- **Potential fix 2 – Modify invariants**: indicates the invariants that should be modified to improve the satisfiability of the model. If **we have not provided the** MUS/MSS **in the INPUTs** block, it will calculate them and show them to us.

Below, we detail the purpose of each graphic element.

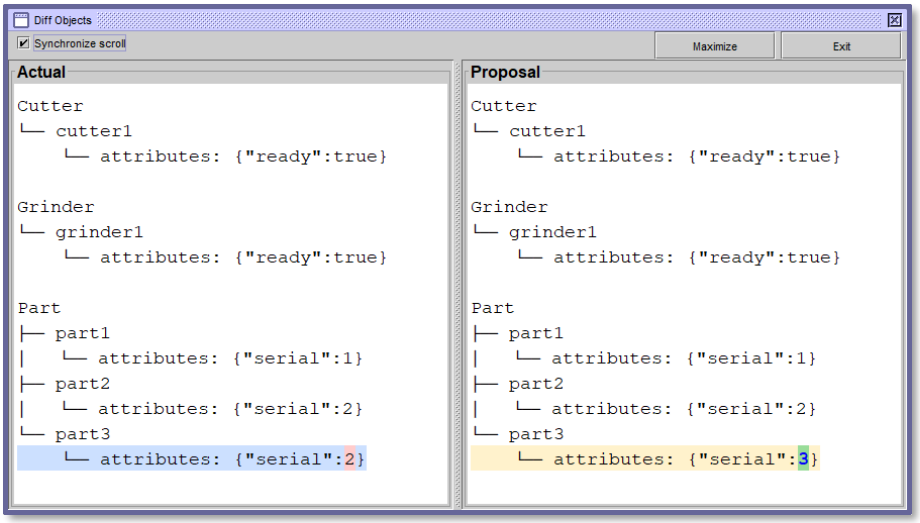
1: Objects

It shows the objects that it suggests to participate in the instance we intend to achieve by adding/removing those that may already exist previously or modifying the values it deems necessary. If there was no list of objects previously, it will suggest a new one.

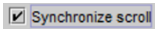


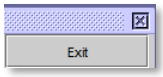
2: Show Diff Objects

It shows the differences between the currently existing objects and those proposed in the new instance:



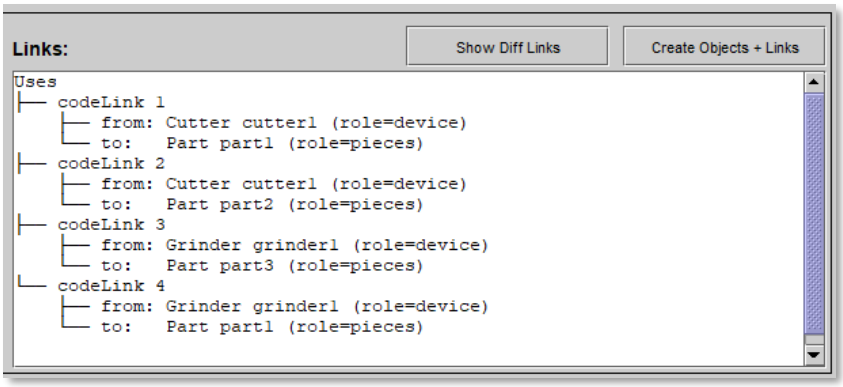
It has the following functionalities:

	Moving the scroll bar allows you to synchronize the 2 panels.

	This button changes its label when you press it and has 2 functionalities: • **Maximize**: Expand the size of the dialogue so that it takes up the entire screen. • **Restore**: Restores to its initial size.
	Close the dialog box.

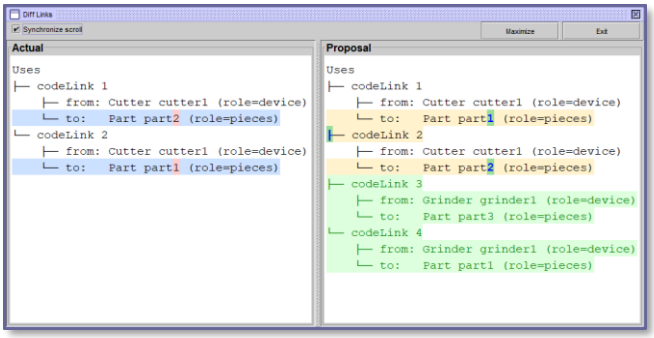
3: Links

Shows the links that it considers should exist in the instance. If links already existed previously, you can suggest modifying them. If they did not exist, it shall also propose the creation of those it deems appropriate.



4: Show Diff Links

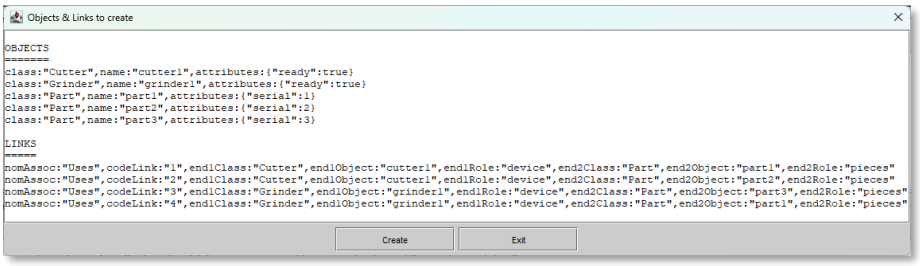
It shows the differences between existing links and the proposed new ones.



The **Synchronize scroll**, **Maximize/Restore,** and **Exit** graphics work the same as those discussed in the **Objects section**.

5: Create Objects + Links

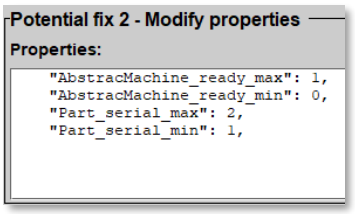
This button allows the creation of the proposed objects and links, but before its creation it shows a summary screen:



If we press **Exit**, we close the screen without taking any action. If we press **Create**, the instance is recreated using the list of proposed elements.

6: Properties

Muestra los parámetros que sugiere modificar en el fichero <**nombreModelo**>.**properties** para facilitar la búsqueda de soluciones por parte del Solver utilizado en el cálculo de **MUS**/**MSS**.



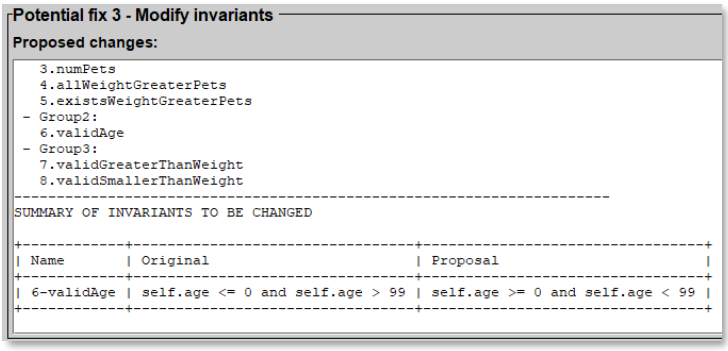
If you choose to modify this file, you have to reload the model to refresh its properties.

7: Proposed changes

This Block shows textually the changes that should be made in the model and, especially, in the invariants it contains.

If we have not provided the **MUS/MSS** as **INPUT** in the query, **OpenAI** will calculate and display them here.

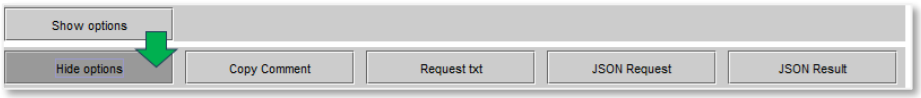
In the best of chaos, a table may be provided showing the current invariant (**Original**) and the proposal (**Proposal**):



8: Show options

This option is intended to help review the query and response blocks that occur during the call to **OpenAI**. It is an option more oriented to the debugging of the process than to its use, but sometimes, it can help to understand how the query was carried out internally.

When you click on this button, the following set of additional buttons appears:



- **Copy Comment**: Allows you to copy the text that appears in the Proposed changes block to the clipboard. (See example in **ANNEXES)**
- **Request txt**: Copy on the clipboard, the query as it has been made to **OpenAI** (See example in **ANNEXES)**
- **JSON Request**: Copy query in **JSON format**. (See example in **ANNEXES)**
- **JSON Result**: Copies the result received from **OpenAI.** (See example in **ANNEXES)**

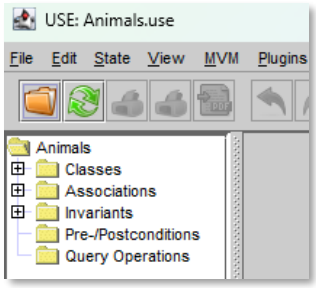
9: Exit

Leave the dialogue without taking any action.

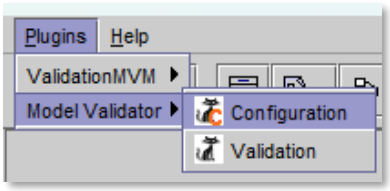
MVM Usage Examples

Example 1 – Animals.use

In our first example, we'll open the **Animals.use** specification:

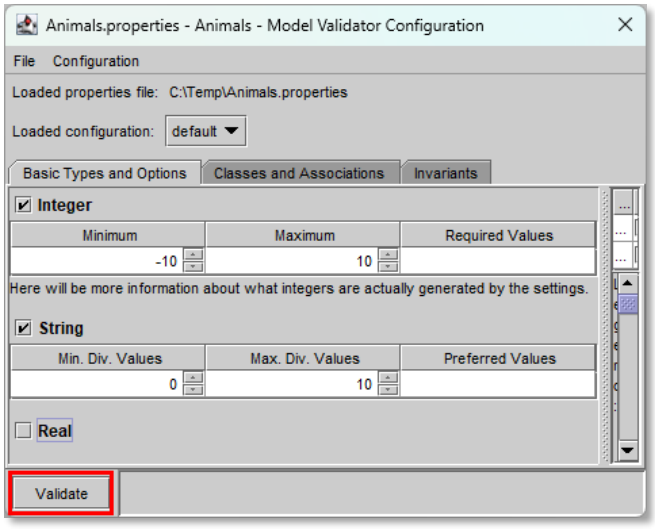


Next, we'll run the Plugins**->Model Validator->Configuration menu option**:

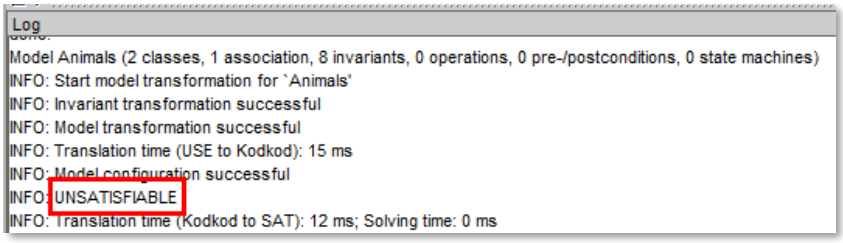


In this way, we verify that USE has certain minimum and maximum values for the **Solver** and specifically, for **Integer** and **String**.

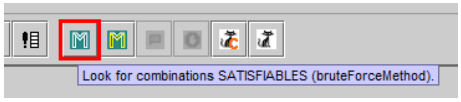
Next, click on **Validate**:



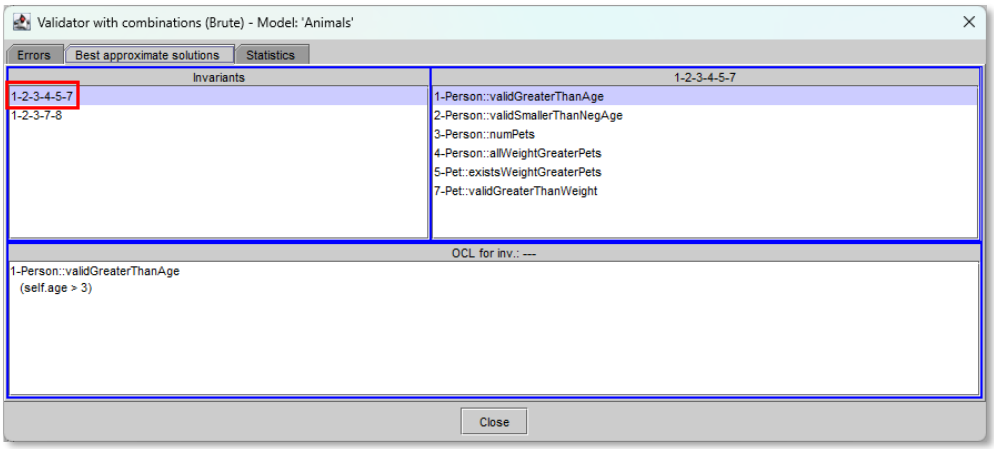
Reviewing the log, we see that the result of the validation is **UNSATISFACTORY**:



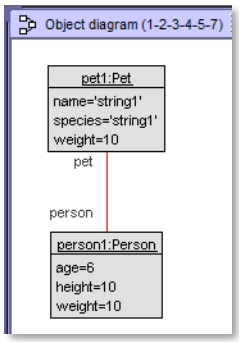
From here, we can test running **Brute force** to get the **MUS/MSS** of the model:



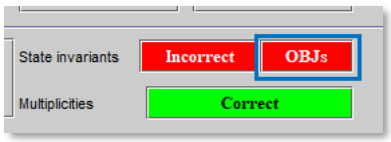
On the next screen we select the **Best approximate solutions tab**:



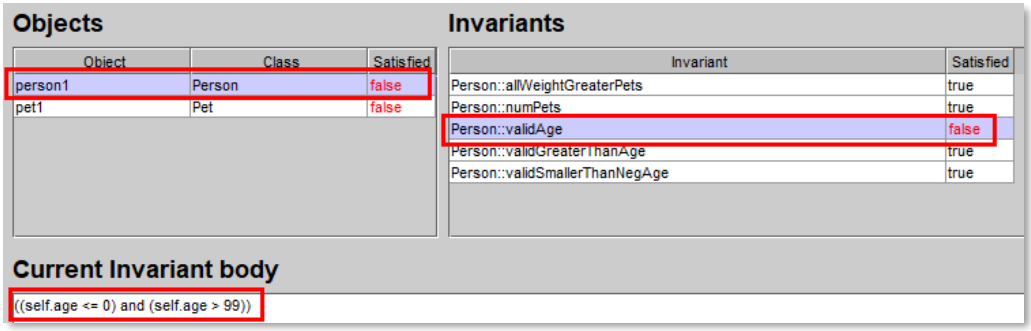
By double-clicking on the first combination of **Best approximate solutions** we will create an object diagram:



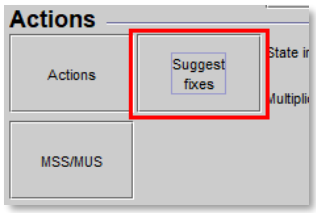
At this point, we can click on **OBJs** to get a first view of the problem:



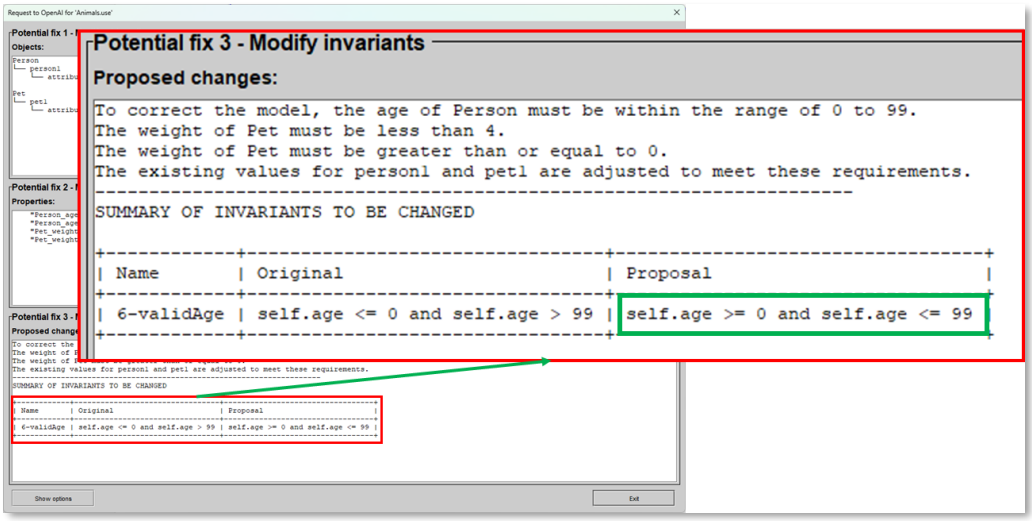
We see that the first problem we find is the following:



We exit this screen with Exit and run **Suggest fixes** to launch a query to **OpenAI:**



We observe the proposal he makes in the Proposed **changes section**:



We take note and simply exit this screen by pressing **Exit**.
Next, we copy the **Animals.use** file onto **Animals02.use** and in its definition, we will change:

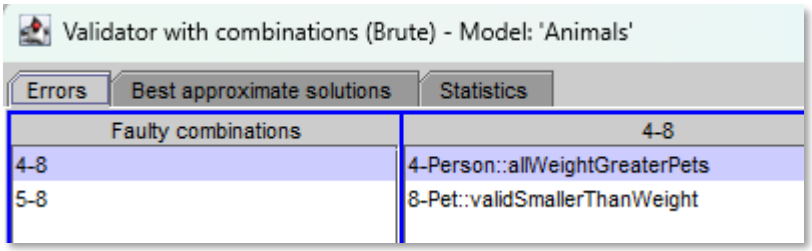
```
self.age <= 0 and self.age >= 99
```

Por

```
self.age >= 0 and self.age <= 99
```

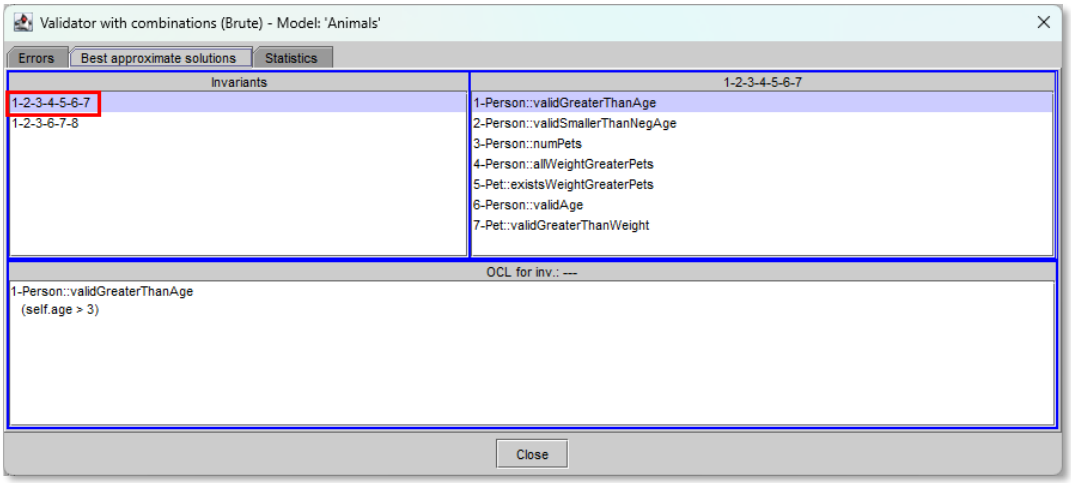
We will also copy **Animals.properties** to **Animals02.properties** so that the new version has a configuration file for **Solver**.

Next, we load the **Animals02** file, launch **Brute force** to see the MUS/**MSS** diagnosis and see that there are still errors:

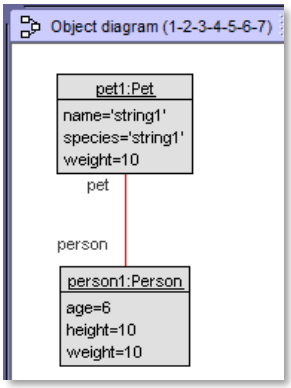


This means that invariants **4** and **8** cannot be activated together, nor can invariants **5** and **8**. These are specifically the groups of **SSM**.

Now, we select the **Best approximate solutions tab**:

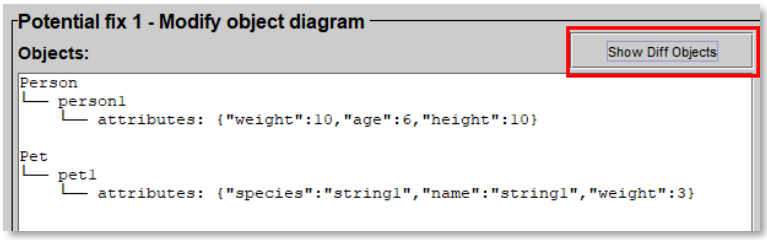


Then we will double-click on the first combination to create a diagram of objects with the maximum number of invariants that meet without problems:

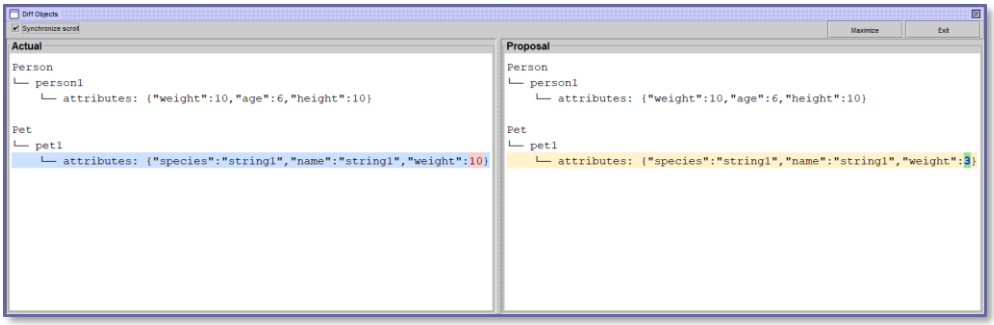


Again, click on **Suggestions fixes** (**LLMs**) and look at the list of objects it proposes.

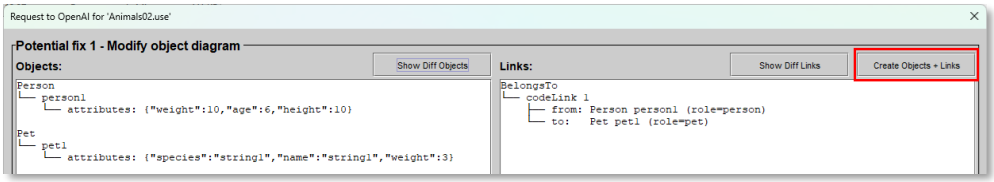
If we click on the **Show Diff Objects button**...



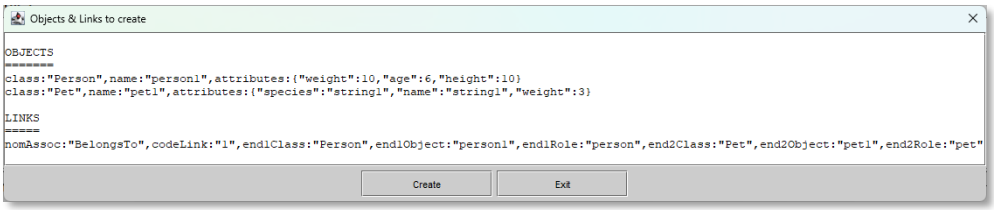
... We will see the changes it proposes:



We exit with **Exit** and click on the **Create Objects + Links button**:

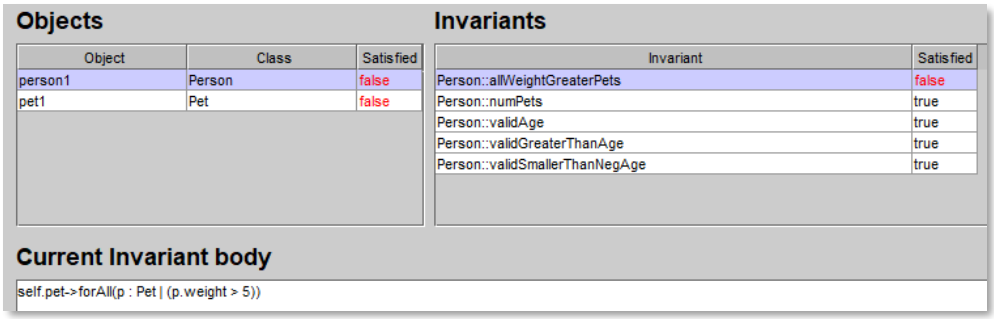


We will see that it shows the following screen indicating the objects and links that it will create:

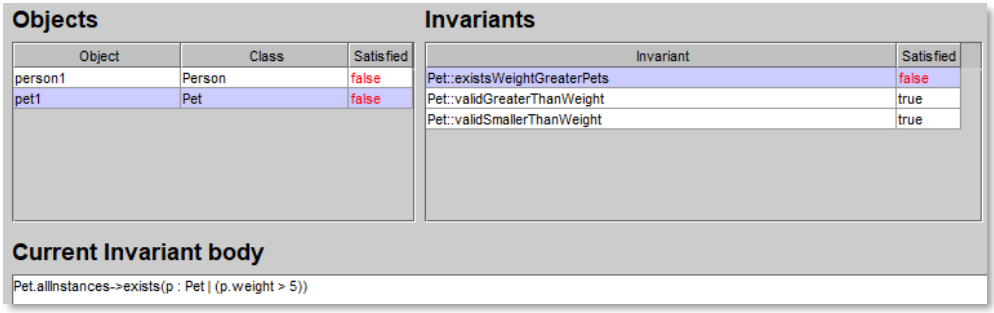


Finally, click on **Create** to create the instance.

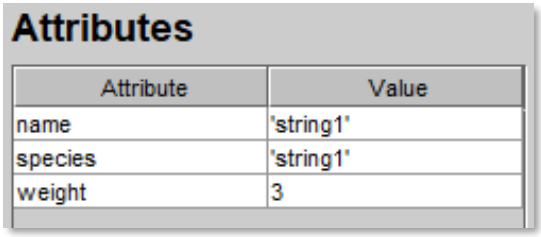
At this point, if we click on **OBJs** we see that there are 2 invariants that fail:



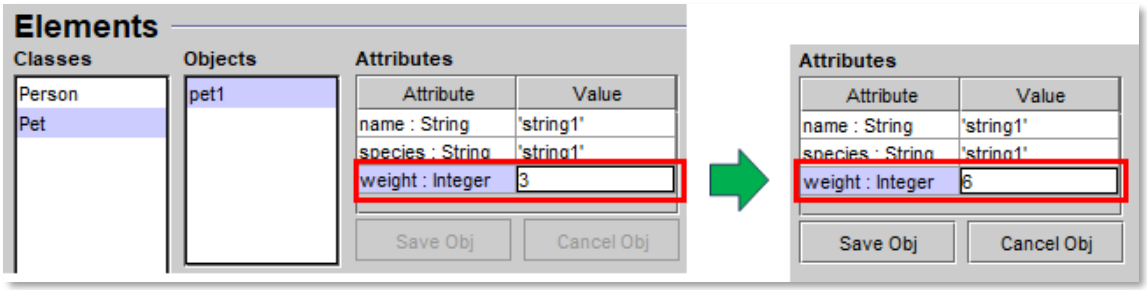
Y



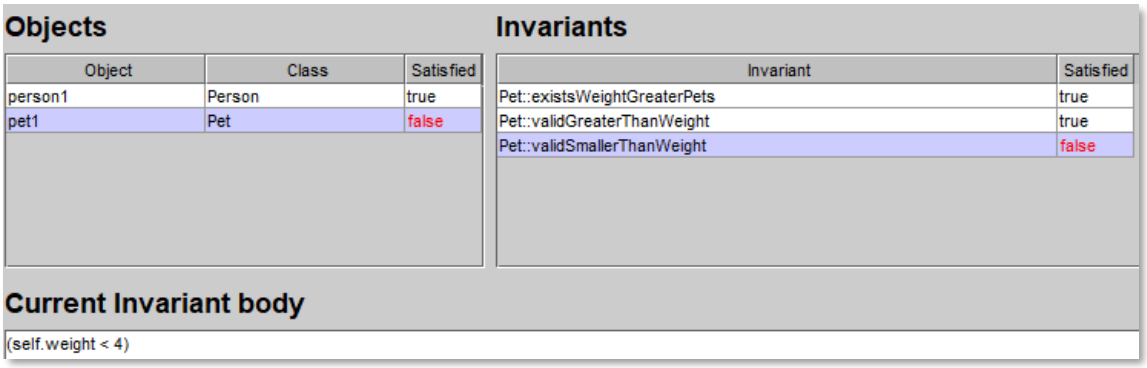
They fail because our **pet** weighs 3:



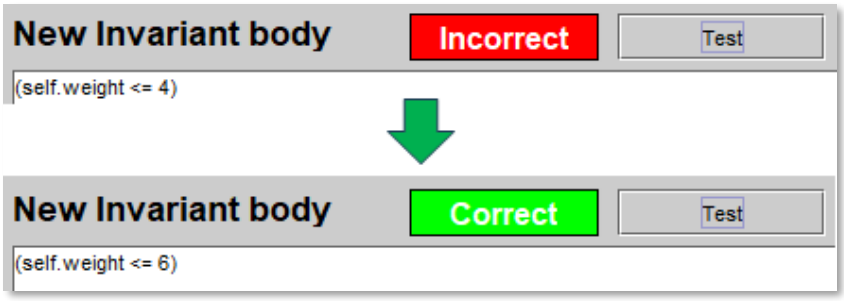
Let's change this weight by pressing **Exit** and returning to the previous screen to access the **Pet values** and replace the **3** with a **6**:



If we click on **OBJs** again, we see that now only one invariant fails:

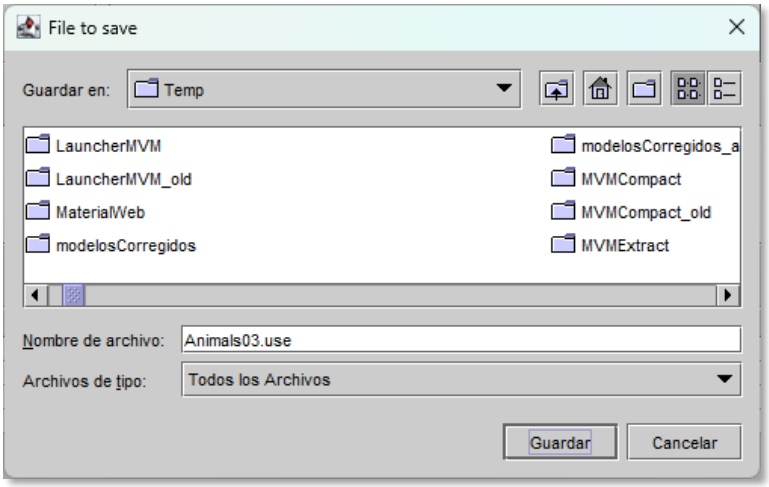


At this point, we can manually make a change on the failing invariant to adjust its bodyexpression to the following: `(self.weight <= 6)`:



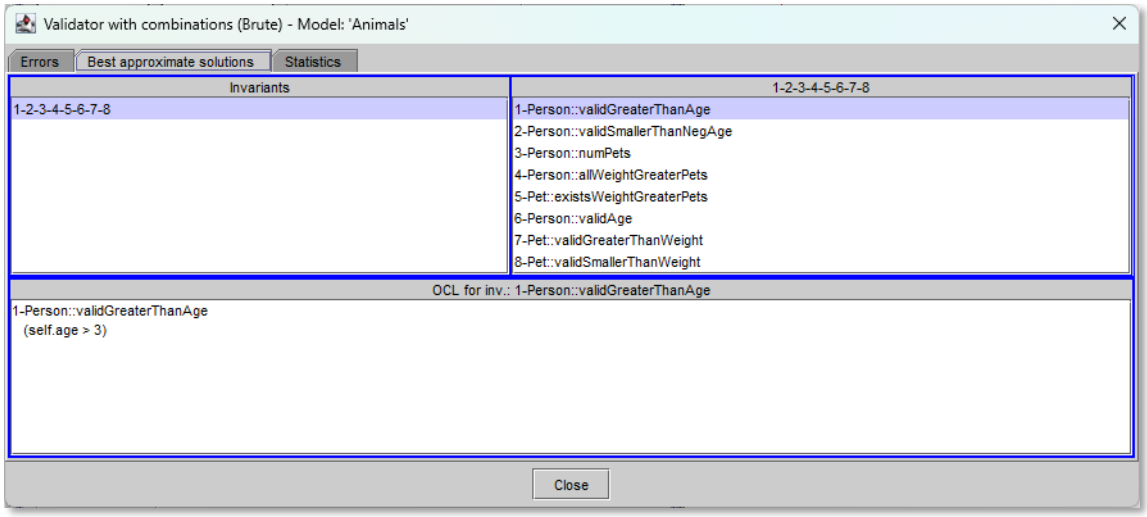
We see that now, when we click on **Test**, the invariant is shown as correct.

Next, we save a new file called **Animals03.use** (don't forget to change the directory to leave it where you prefer):

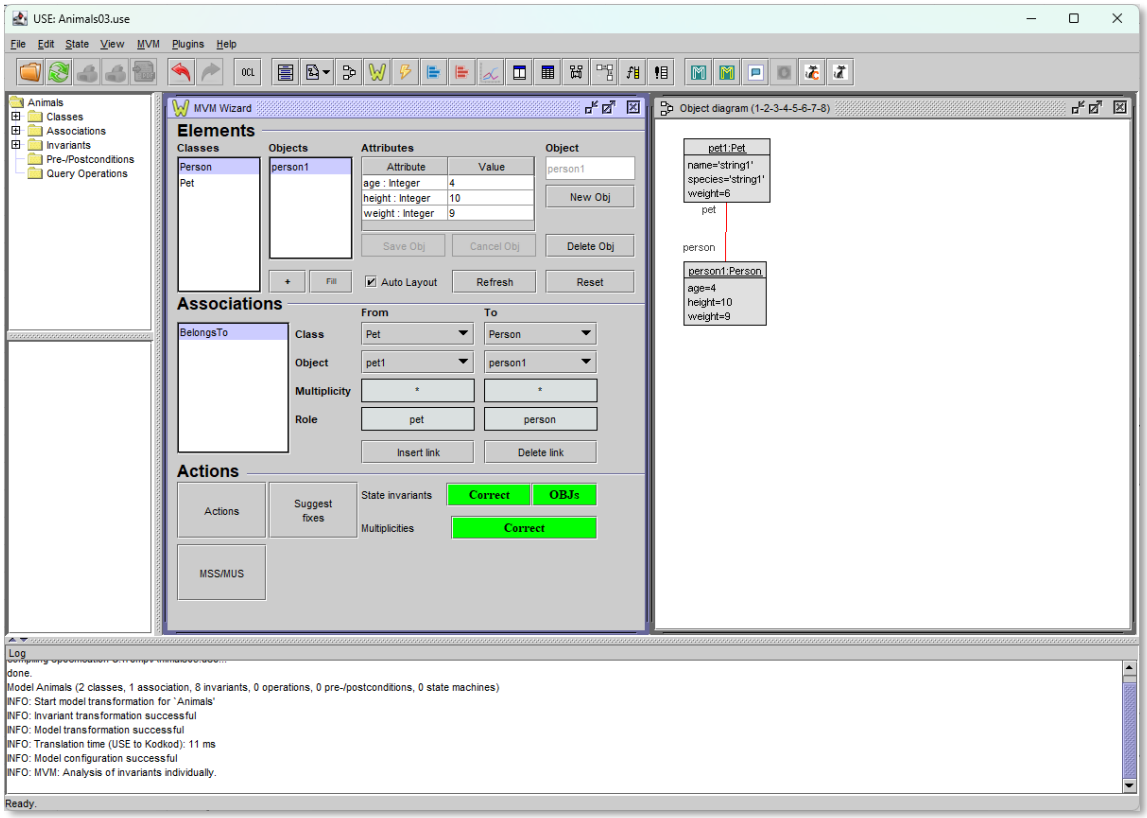


We went out with **Exit** and loaded the new **Animals03.use** into **USE.**

We run **Brute force** and create an instance with the first combination of **Best approximate solutions** by double-clicking on it:

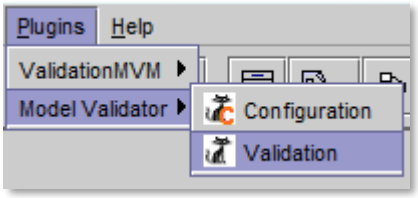


We will see that the instance is already created successfully:

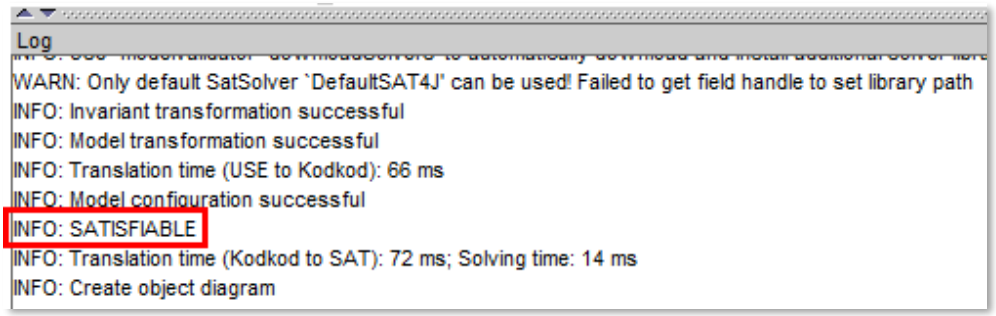


The model defined in **Animals03.use**, we can say that it is **SATISFAIABLE.**

Sure enough, if we load the **Animals03.use model** and launch the validation using **Animals03.properties**:

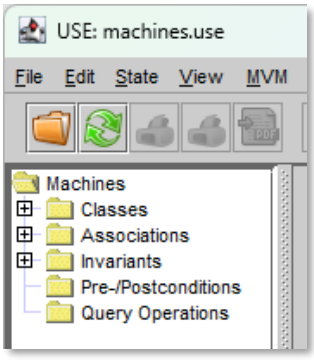


We will get:

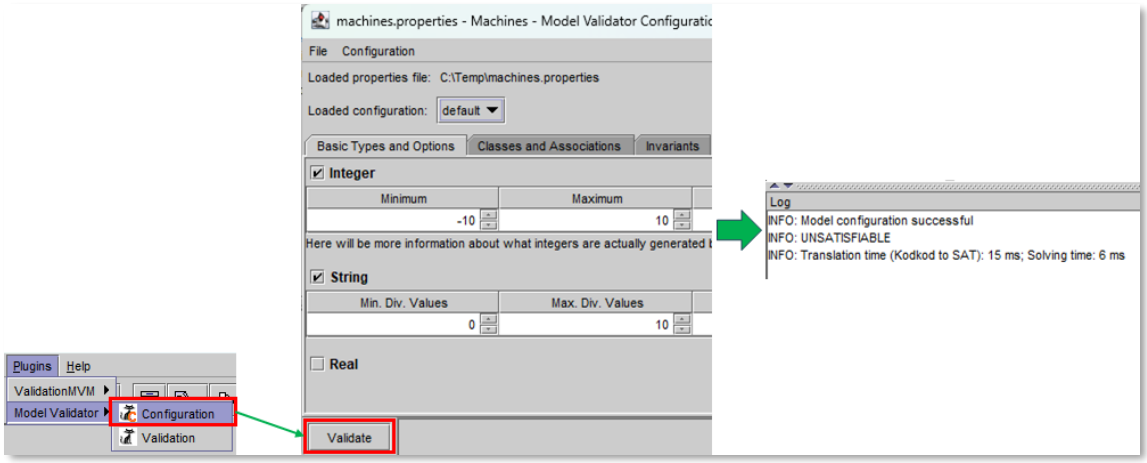


Ejemplo 2 – machines.use

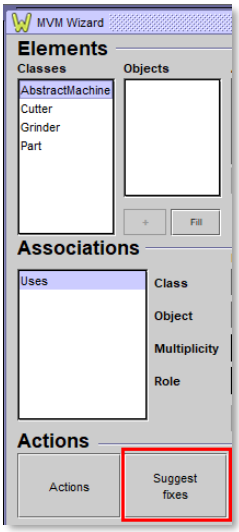
In this example, we'll open the **machines.use** specification:



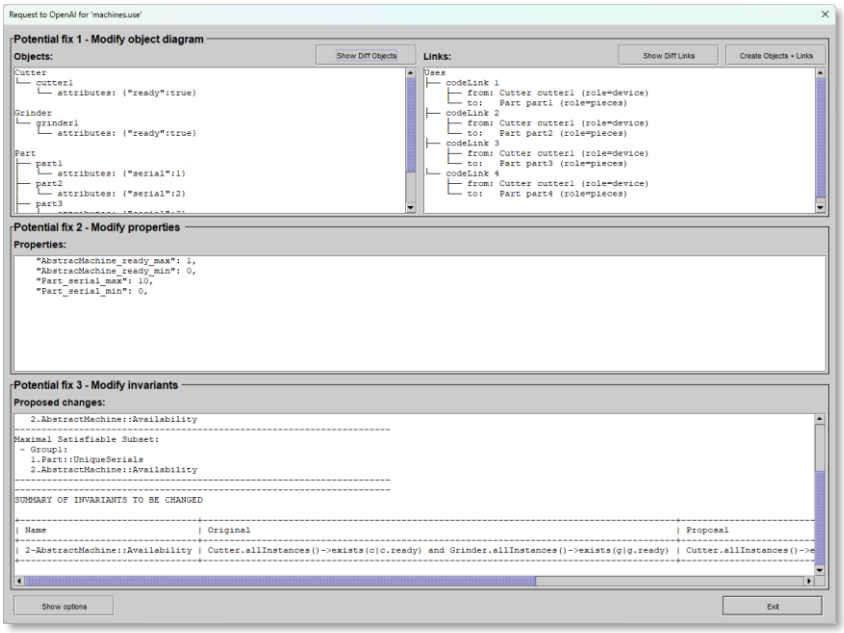
Once opened, we will run **Plugins->Model Validator->Configuration**, click on **Validate** and check that it is **UNSATISFACTORY:**



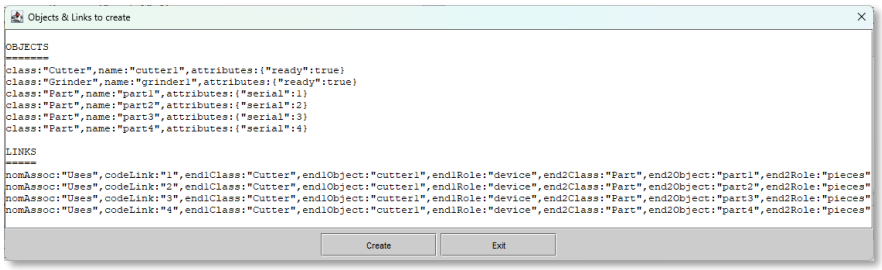
Next, we will open the **MVM Wizard wizard** (**MVM->MVM Wizard->MVM Wizard**) and click on **Suggest fixes**:



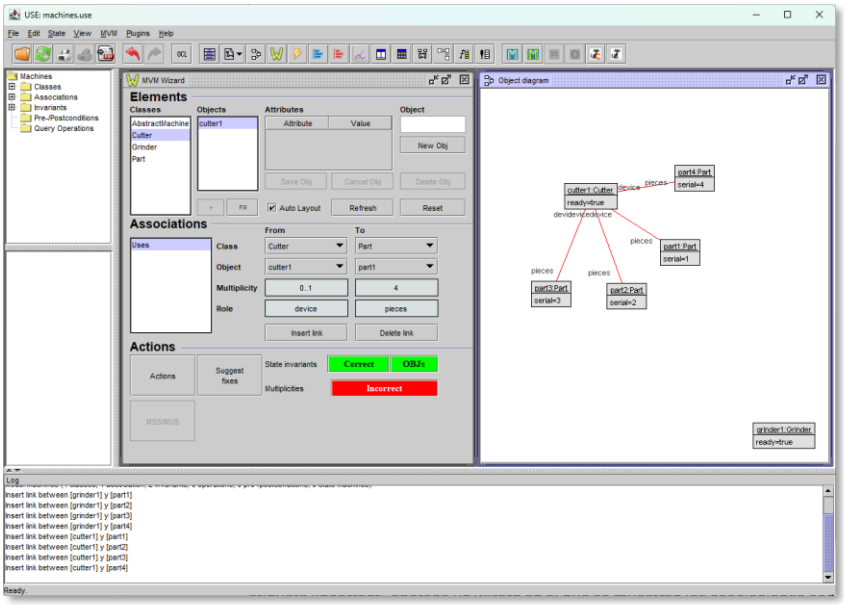
The following screen will appear:



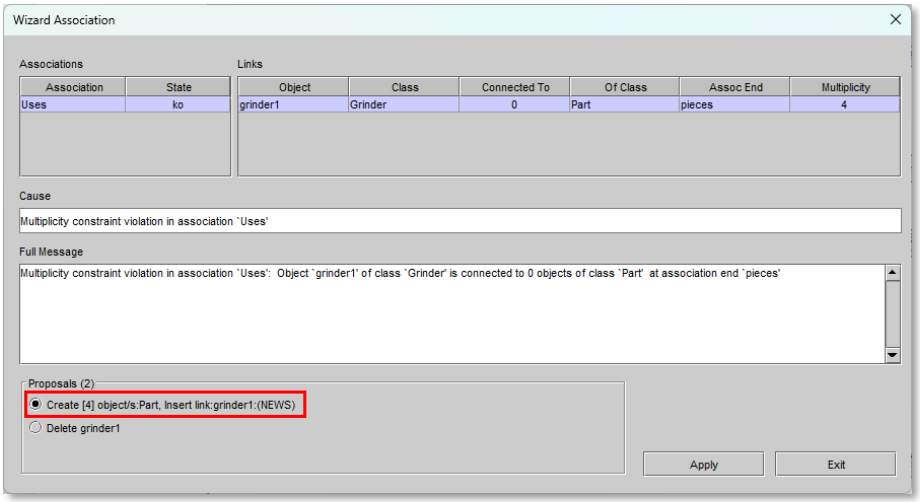
Click on **Create Objects + Links** to create a first instance. Clicking on this button displays the objects and links to be created:



If we click on **Create** we will see the creation of this first instance:

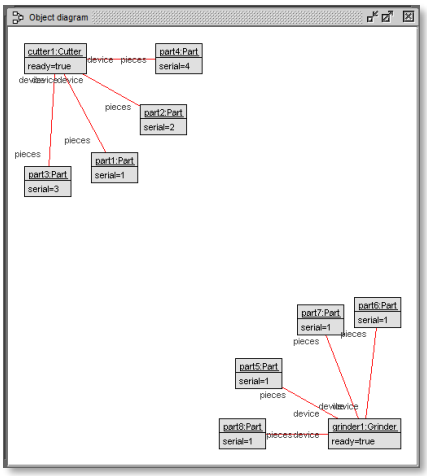


We see that multiplicities fail. If we click on the red button labeled '**Incorrect**', a wizard appears showing the associations with problems and some proposals to solve them:

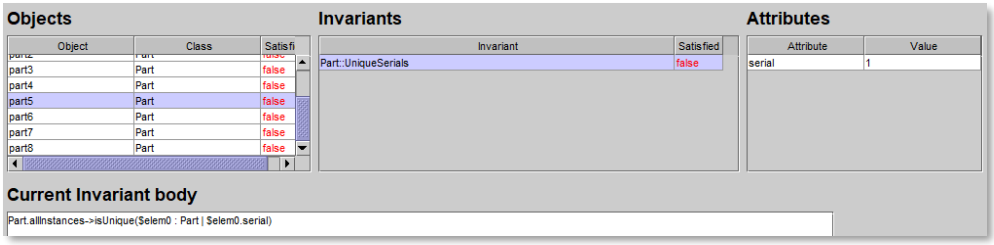


This time, we selected the creation of 4 Part type objects and the creation of a link with the **grinder1 object**.

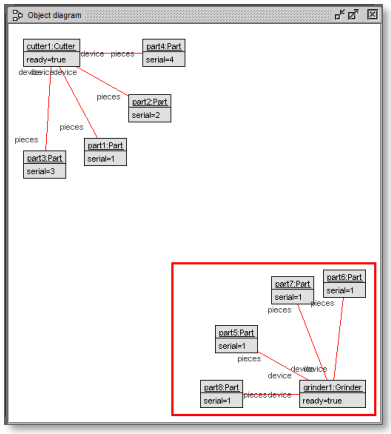
Click on **Apply** and see the new instance:



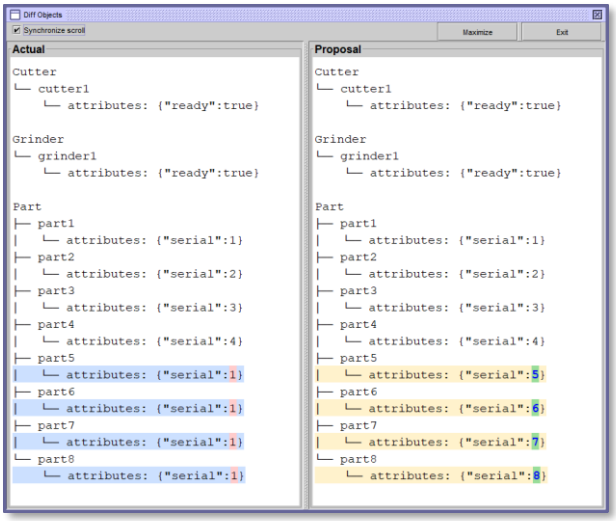
If we click on **OBJs** we will see that the only invariant that fails is the one that indicates that the serial numbers must be unique:



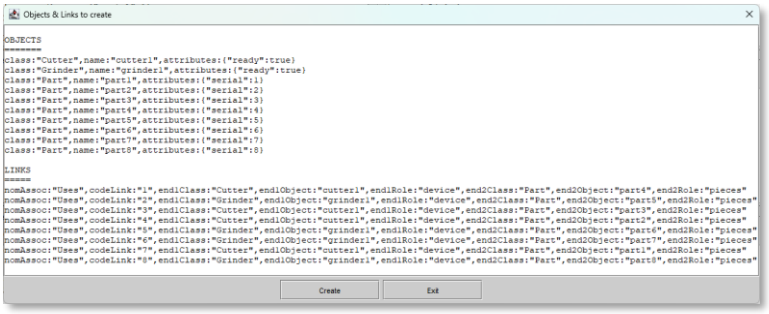
We will exit this screen with **Exit** and we will see how the new **Part** objects have the same serial number as they have been automatically created as a copy of each other:



Next, we will click on **Suggest fixes** to see the proposal for creating objects that it makes and check how it intends to solve this problem. Once the **Request to OpenAI for 'machines.use'** screen appears**, we will click on** Show Diff **Objects to see the changes it proposes to make:**

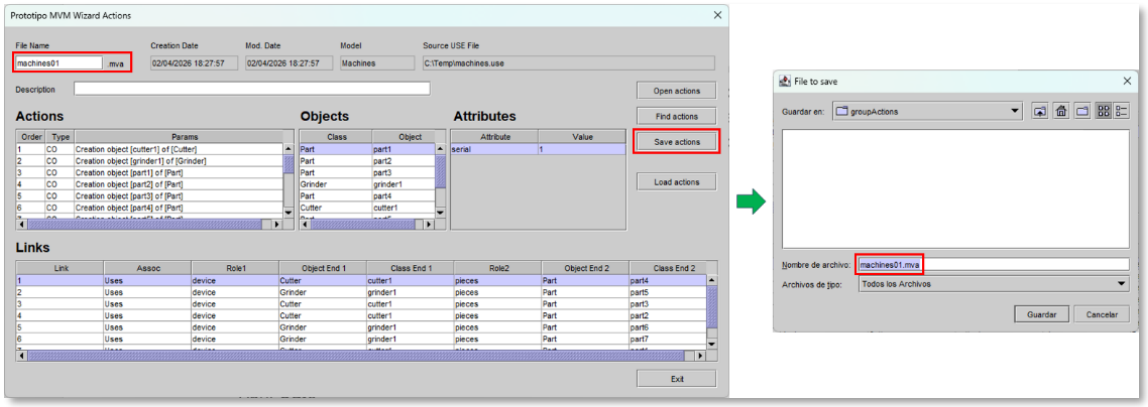


We'll exit with **Exit** and click on **Create Objects + Links**. It will show us the list of objects and links proposed to create the new instance:



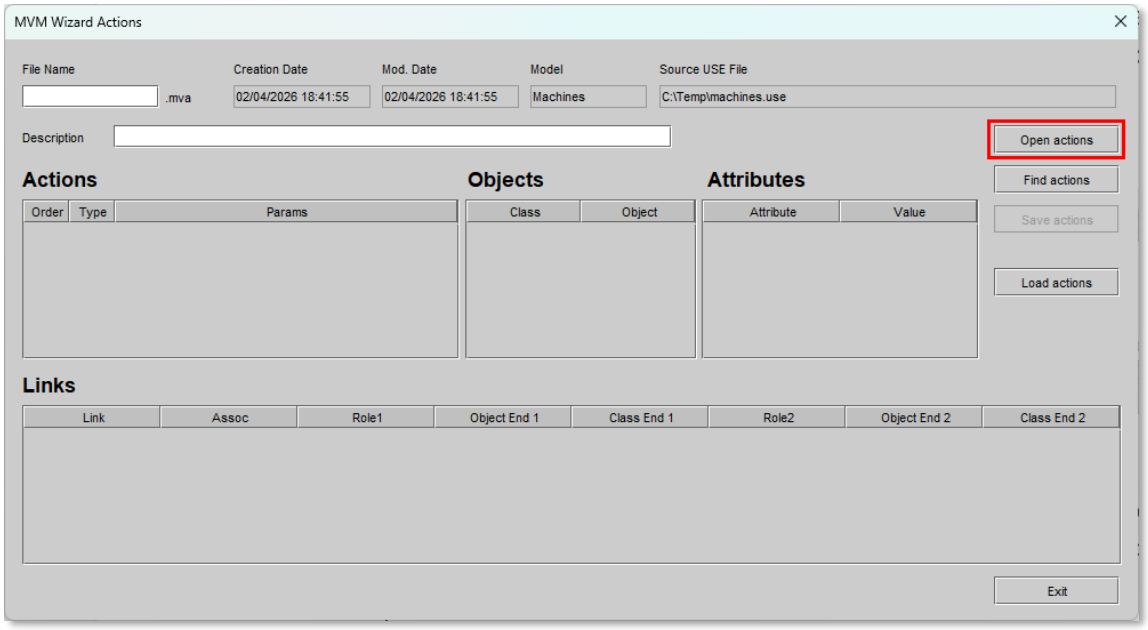
When we click on **Create** we see the new instance created correctly.

If we wanted to, at this point we could save the entire sequence of creating elements so that we could reproduce the creation of a satisfactory instance at any time and instantaneously by simply loading that sequence. To do this, click on the **Actions** button and give a name to the sequence of actions:

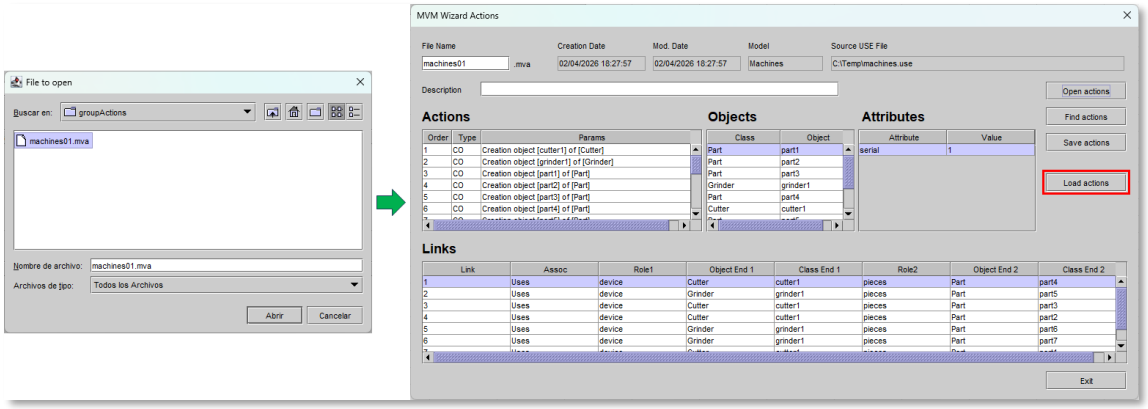


Now, we could load the machines.use file again and, to obtain a satisfactory instance, it would be enough to open the **Wizard** (**MVM->MVM Wizard->MVM Wizard**), click on **Actions** and load the sequence previously saved in the previous step.

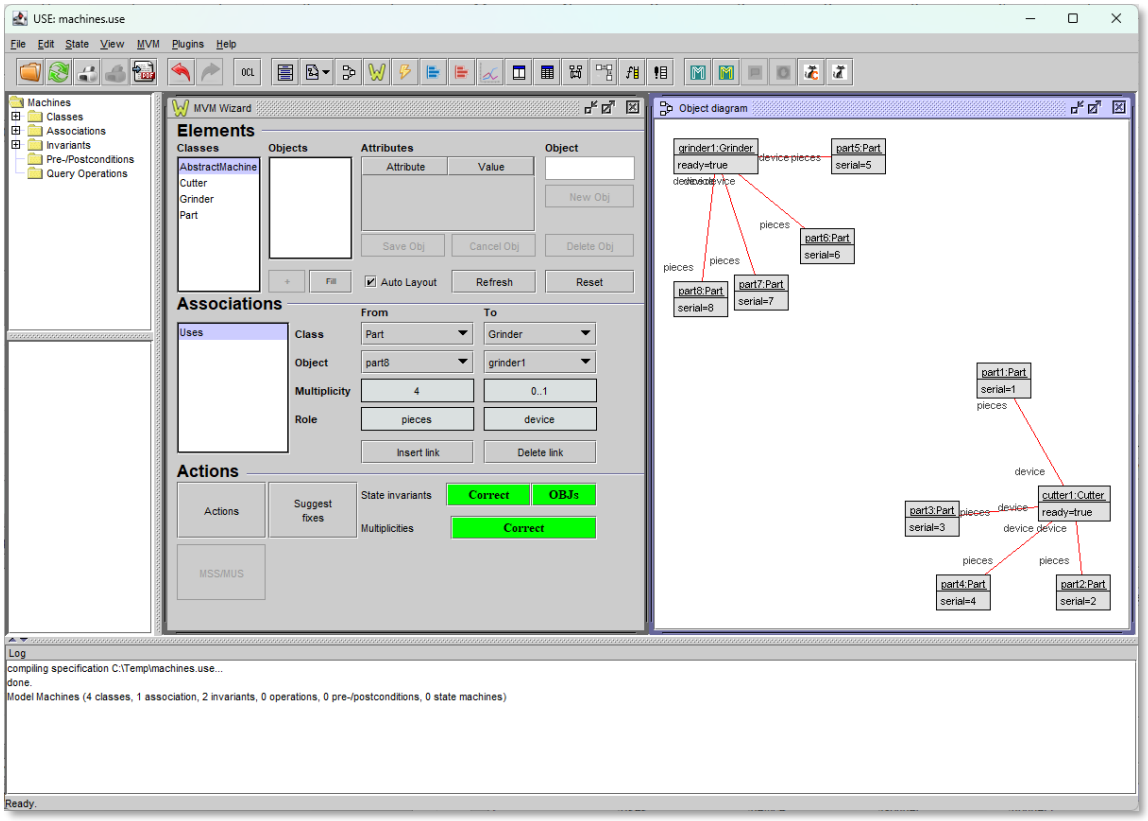
Click on **Actions** and the **MVM Wizard Actions** appears:



Click on **Open Actions** to locate our sequence of actions and once selected click on **Load actions**:

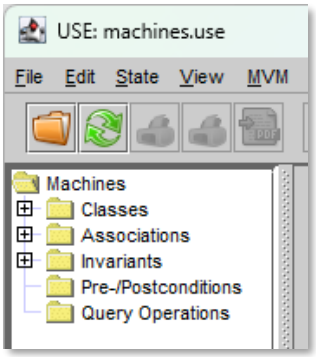


Once we have clicked on **Load actions**, we will see an instance of our model that is totally satisfactory:

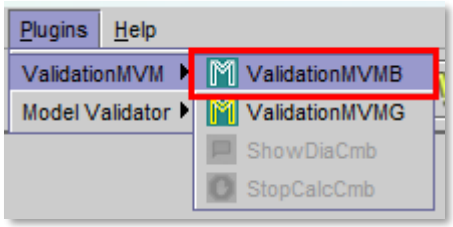


Ejemplo 3 – machines.use

Open **machines.use** specification:

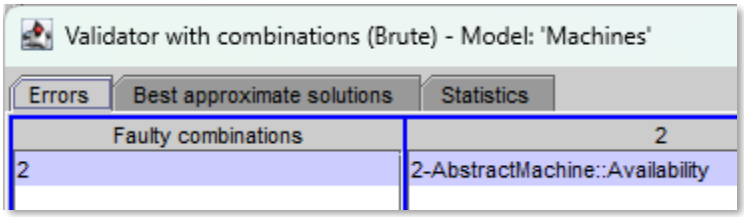


Next, we launched **Brute force** as follows:

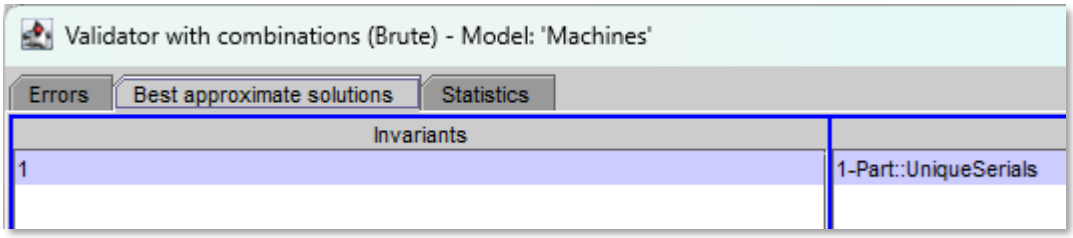


Or

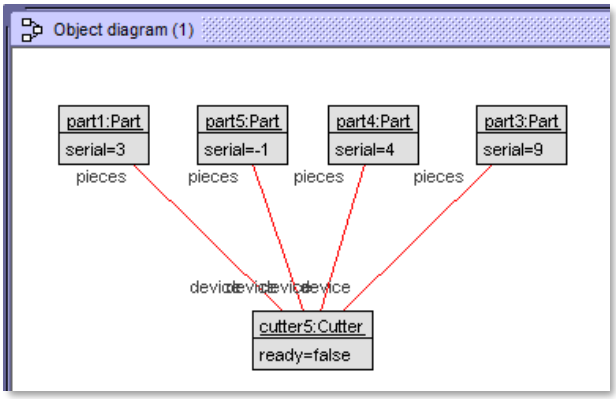
When you finish the search, you see the combinations that cause errors:



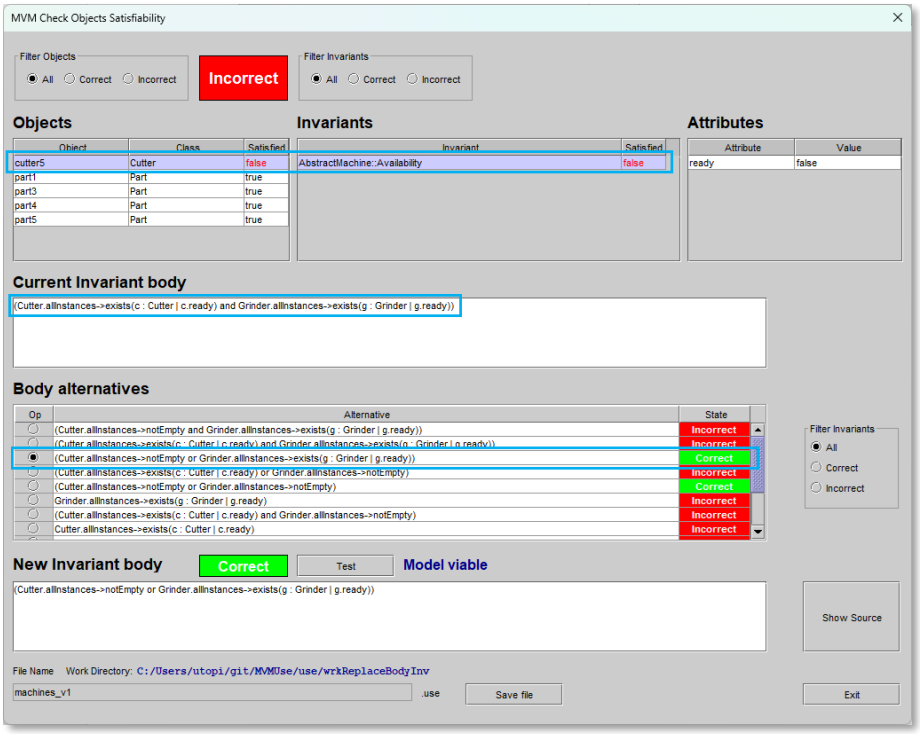
We will also look at the best solutions:



From here, we create an instance by double-clicking on the existing combination in **Best approximate solutions** and observe the following diagram:



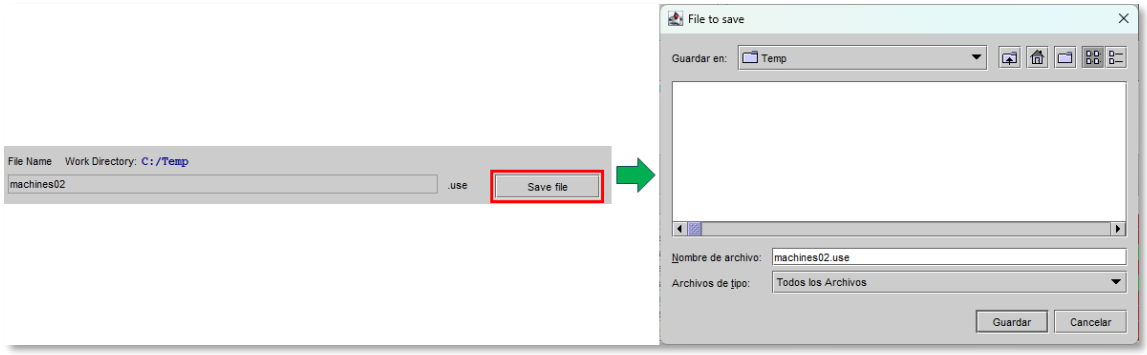
Click on **OBJs** to see alternatives and to see the invariant that fails:



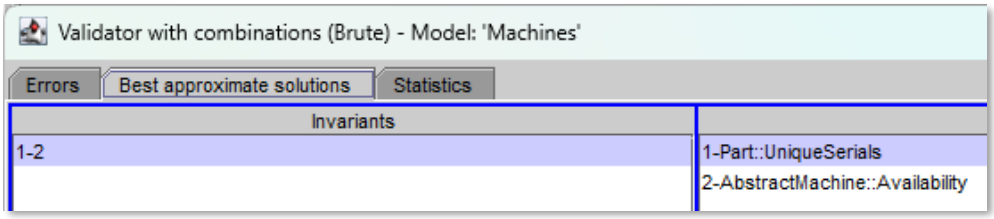
Here, we select the following alternative:

```
(Cutter.allInstances->notEmpty or Grinder.allInstances->exists(g
: Grinder | g.ready))
```

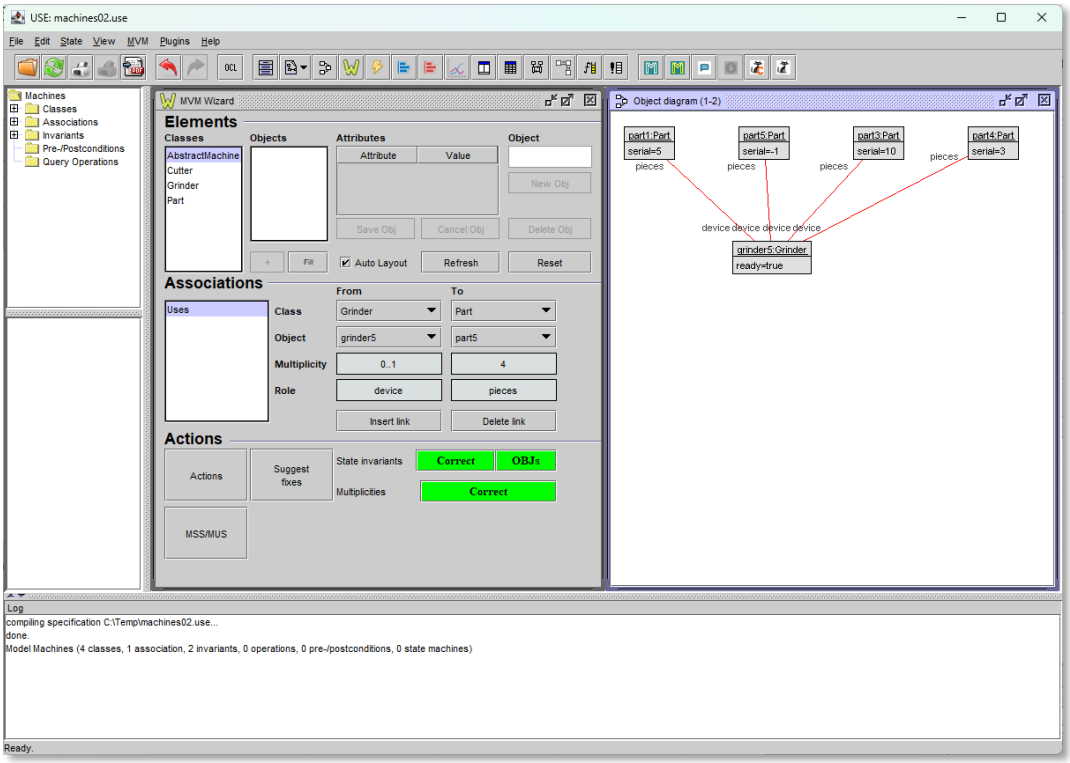
Next, we save the changes to the **machines02.use** file by clicking on the **Save file button**:



Next, we load the **machines02.use** specification and launch Brute **force again**:

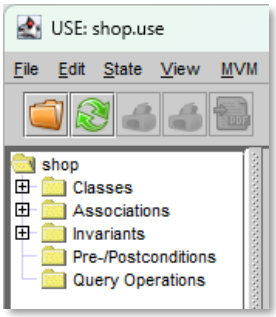


We see that there are no more errors and if we create an instance by double-clicking on the combination that appears in **Best approximate solutions** we will get a satisfactory instance:

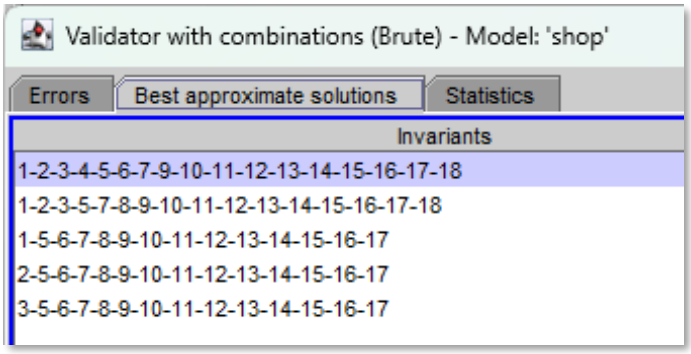


Ejemplo 4 – shop.use

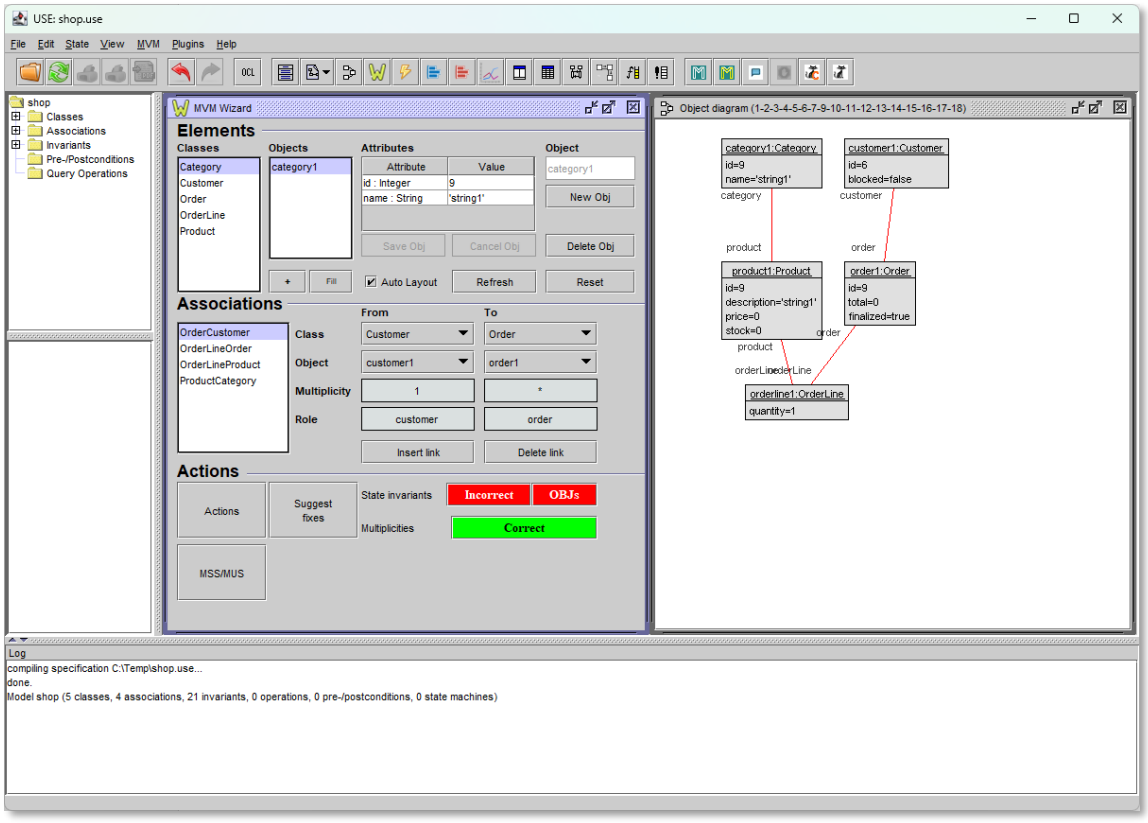
Open shop.use **specification**:



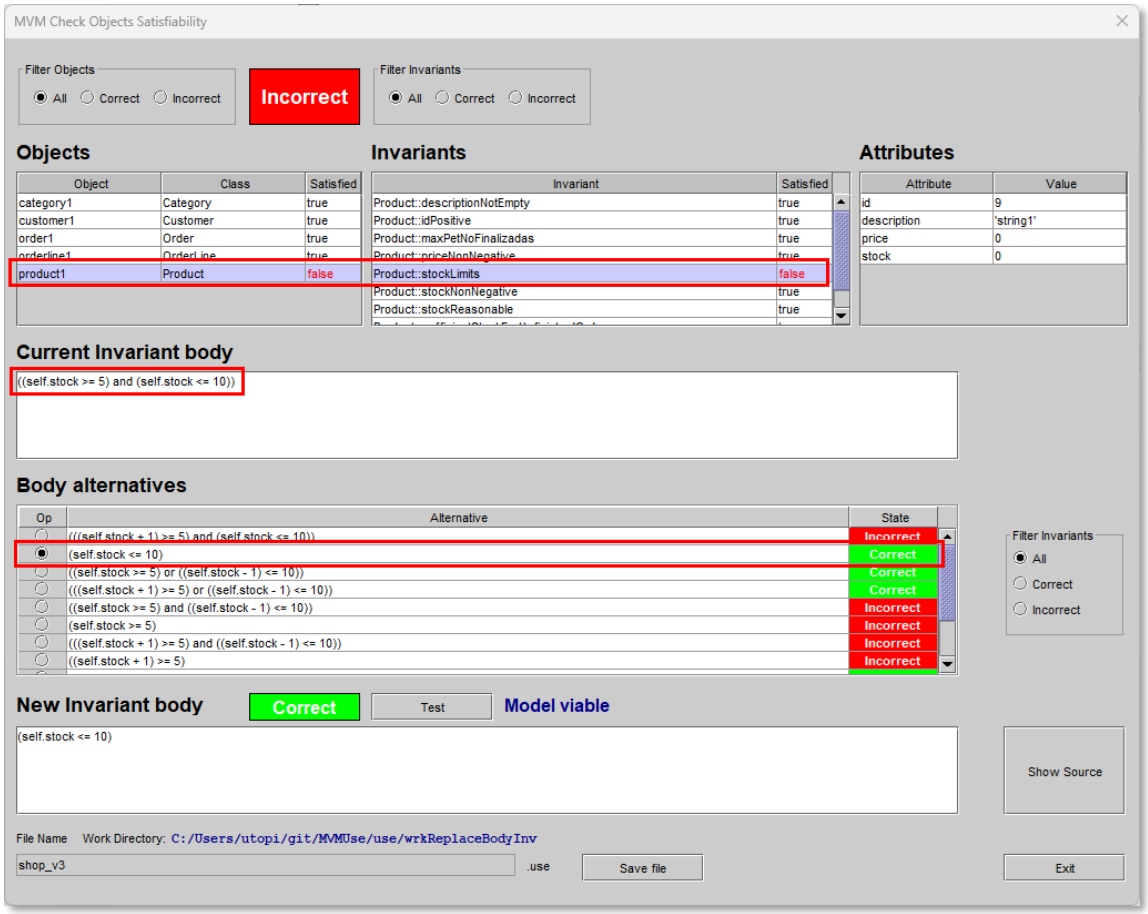
We run **Brute force**:



We instantiate with the first combination of **Best approximate solutions**:



Click on **OBJs** to see the invariants that fail:



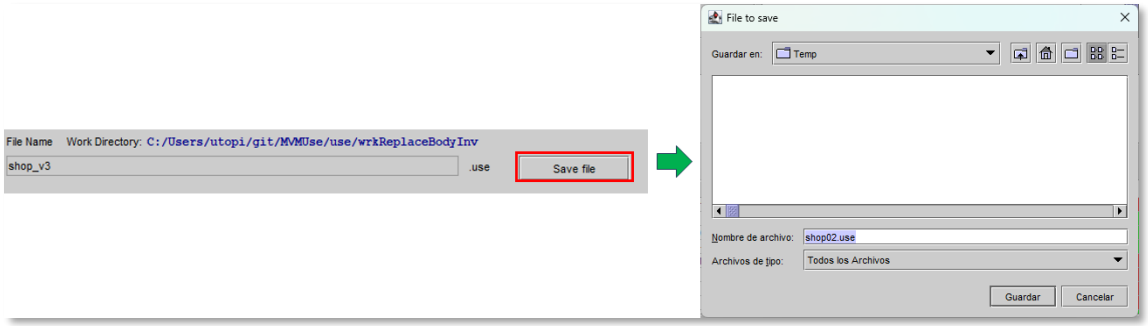
Select alternative for the invariant **`Product::stockLimits`** to change:

```
(self.stock >= 5 and self.stock <=10)
```

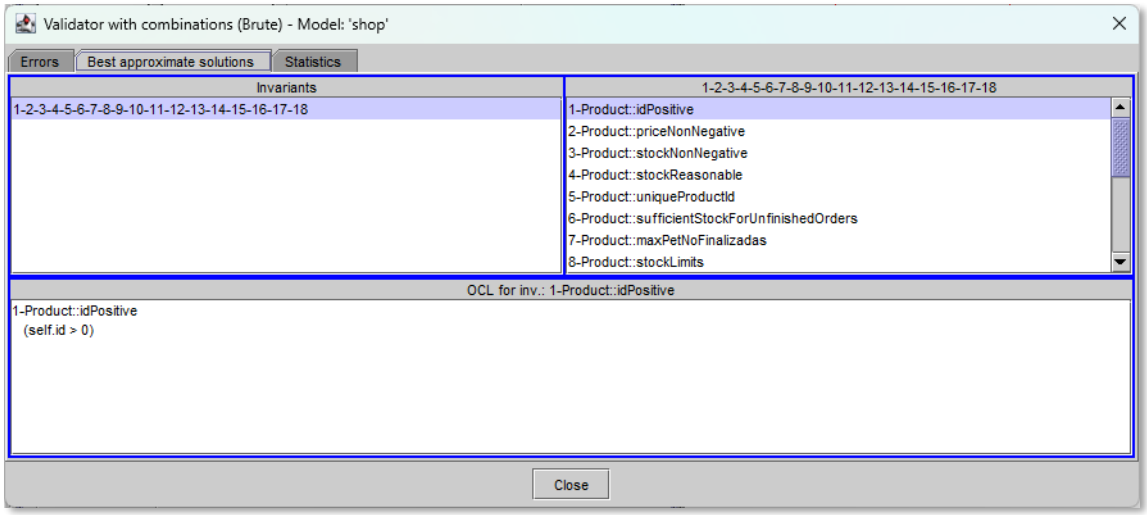
por

```
(self.stock <= 10)
```

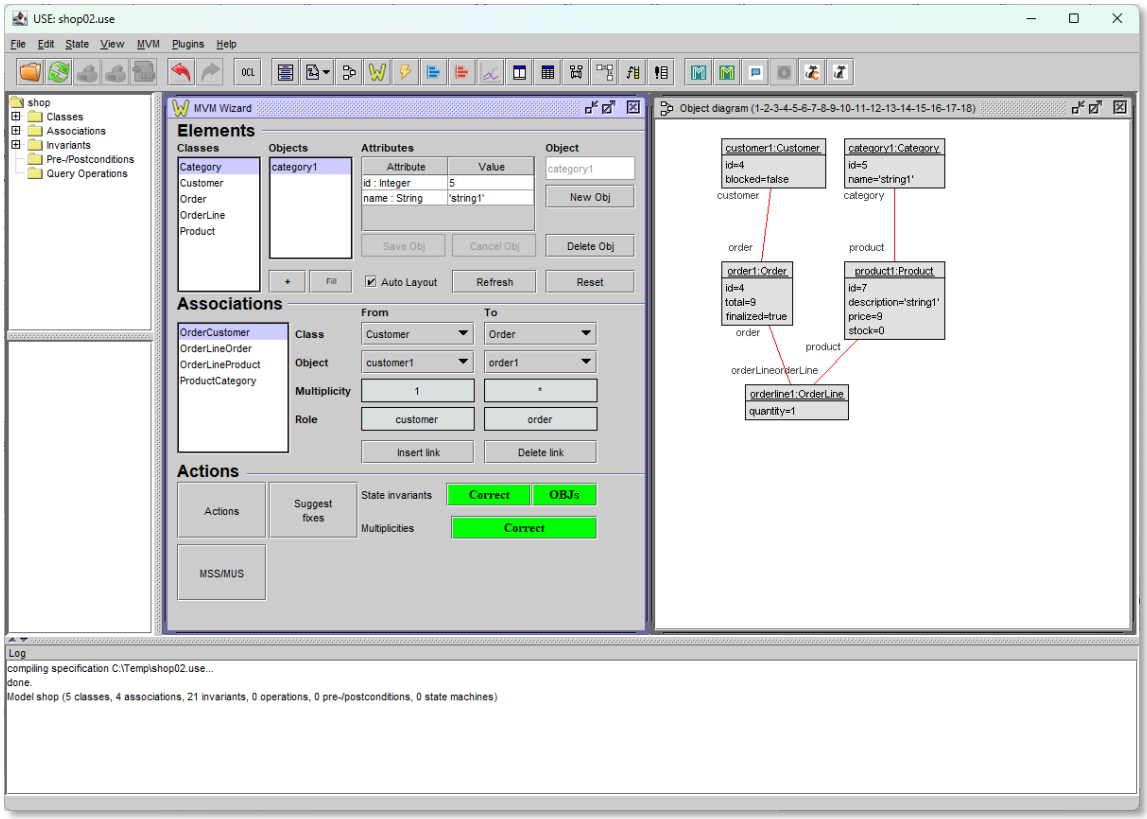
Salvamos fichero **shop02.use**:



Next, we open **shop02.use**, run **Brute force** and instantiate by double-clicking on the first combination of **Best approximate solutions**:

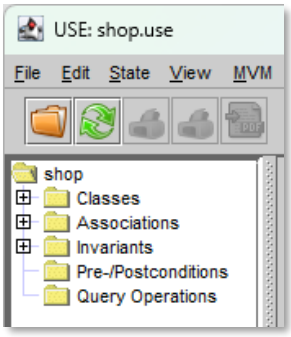


We observe that the instance is satisfactory:

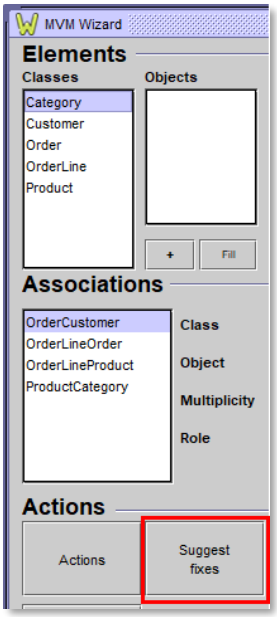


Ejemplo 5 - shop.use

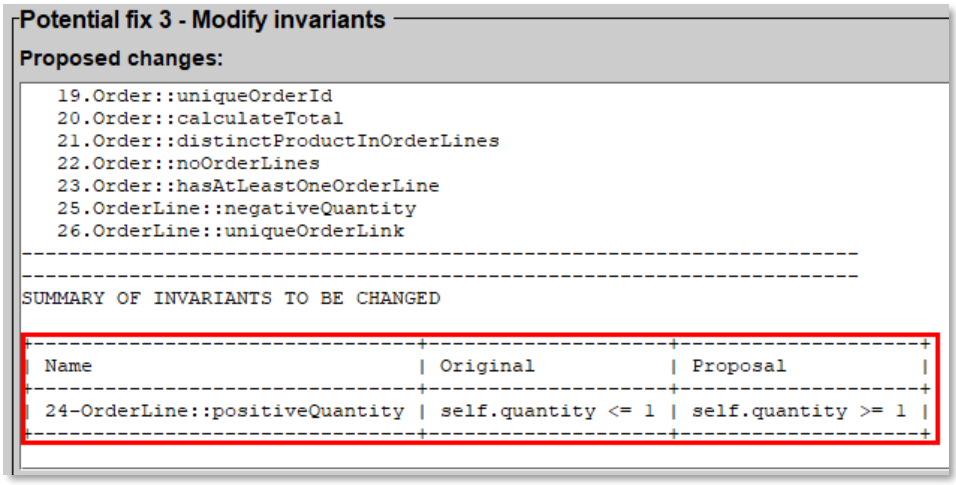
Open shop.use **specification**:



Open MVM Wizard and run **Suggest fixes**:



On the one hand, we see in **Proposed changes** that we propose to modify the invariant **`OrderLine::positiveQuantity`**:



Next, we copy the **shop.use** file to **shop02.use** and in the OrderLine**::positiveQuantity invariant** we change the following:

```
self.quantity <= 1
```

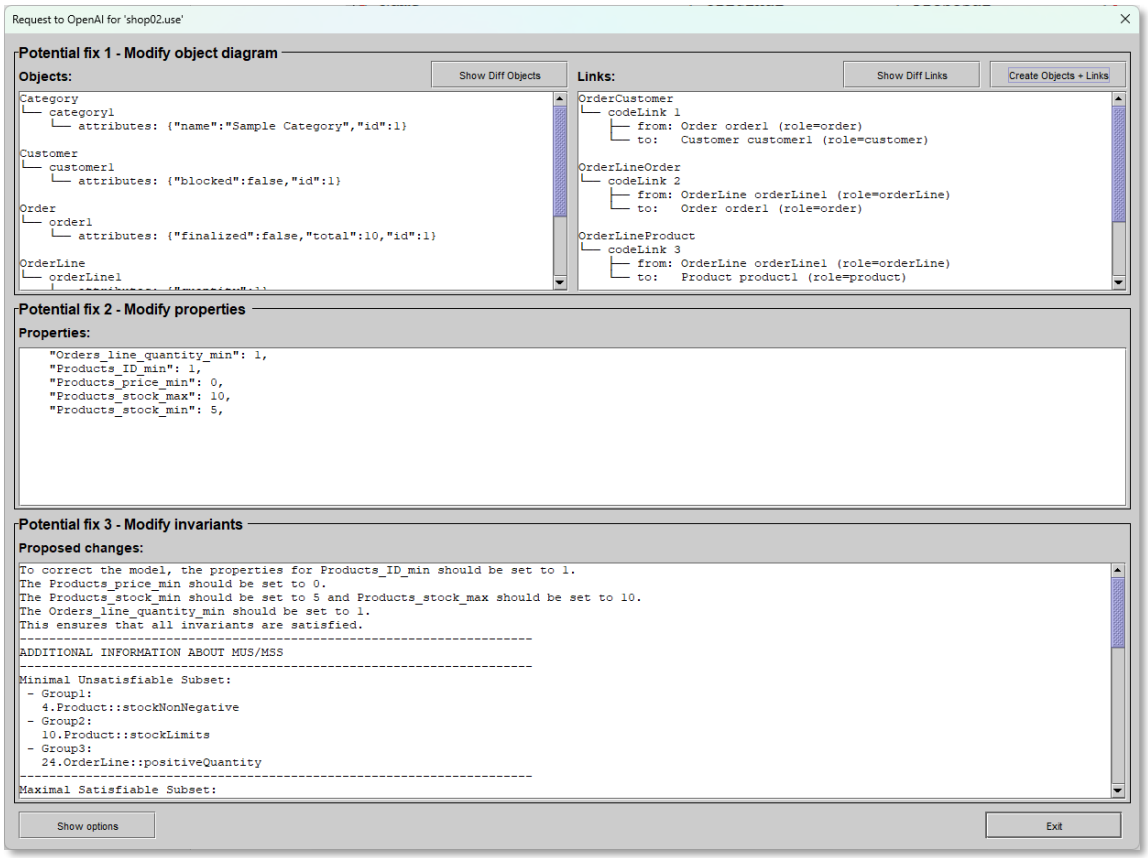
By

```
self.quantity >= 1
```

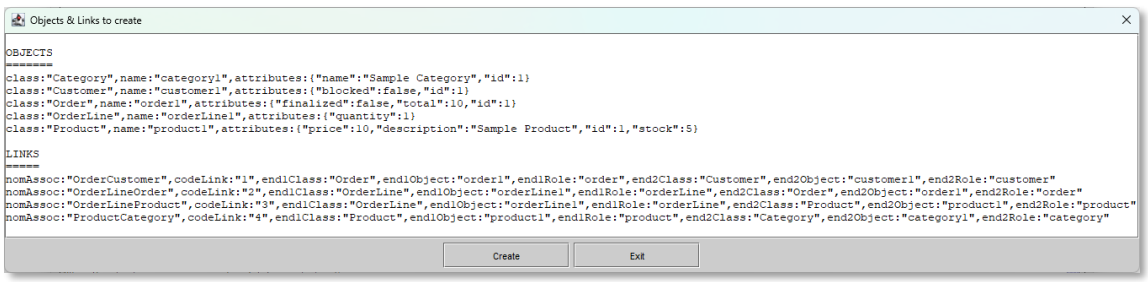
We also copy **shop.properties** to **shop02.properties**

Open **shop02.use** and open **MVM Wizard**.

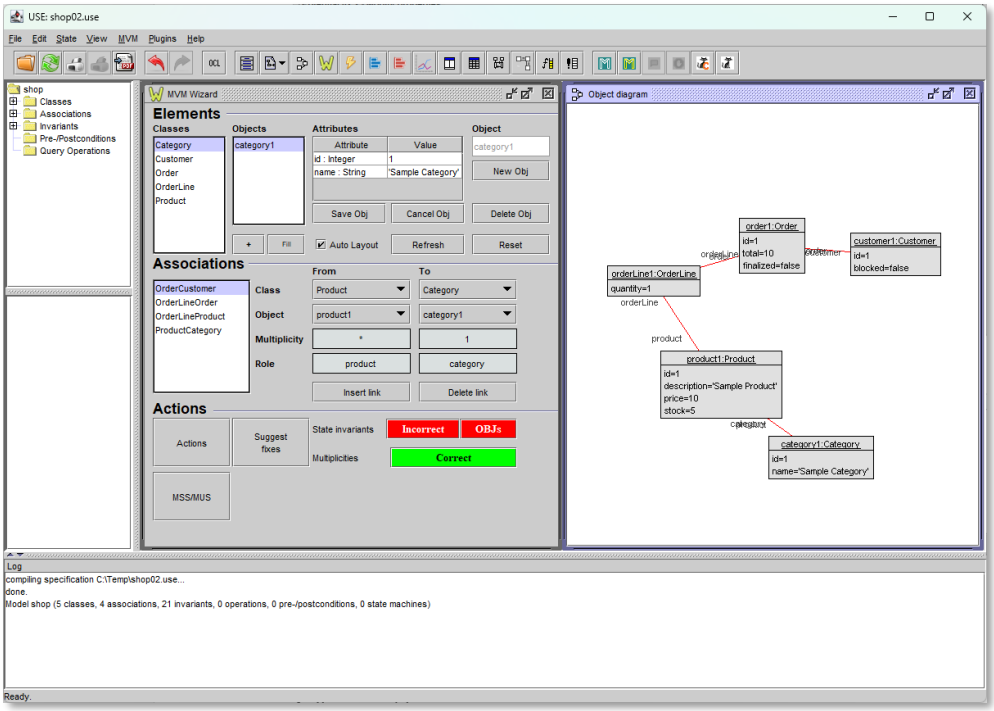
We run **Suggest fixes** and the following screen appears:



Click on **Create Objects + Links** and see the list of elements proposed for creation:

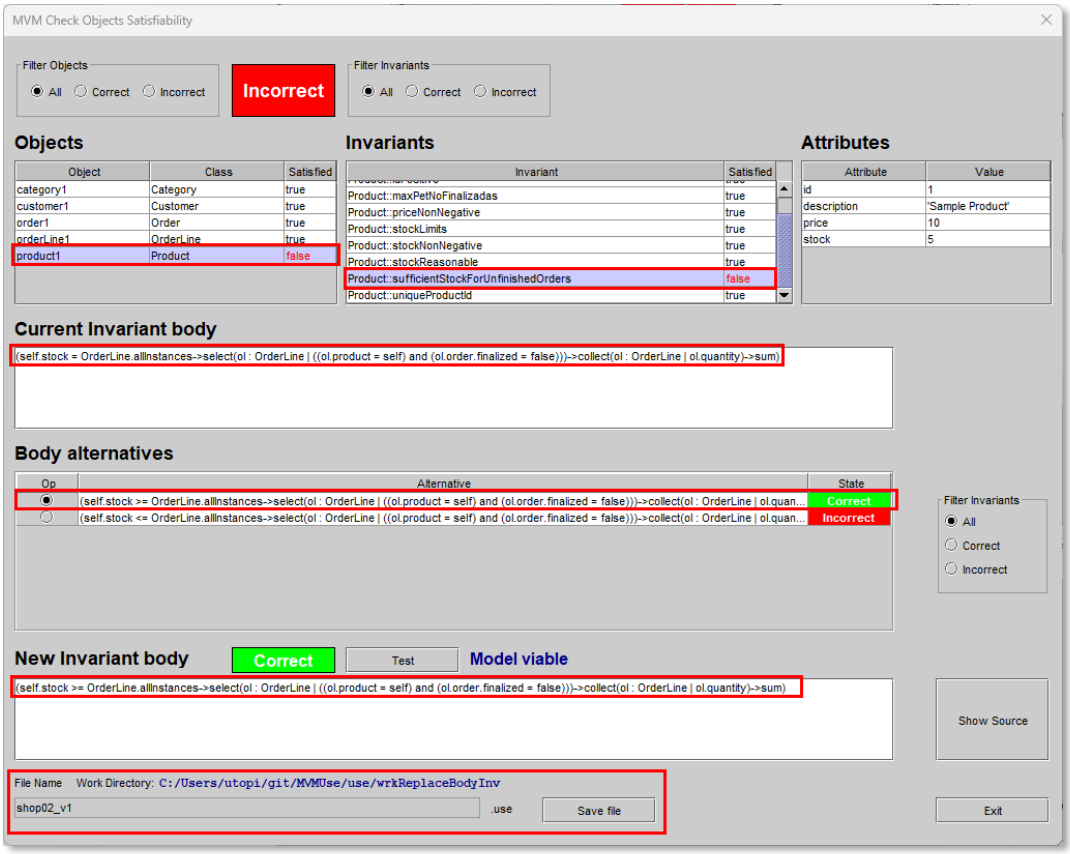


Click on **Create** and check instance created:



We click on **OBJs** and we see that the invariant fails:

```
Product::sufficientStockForUnfinishedOrders
```



We select the alternative that indicates '**Correct**' and apply change so that instead of:

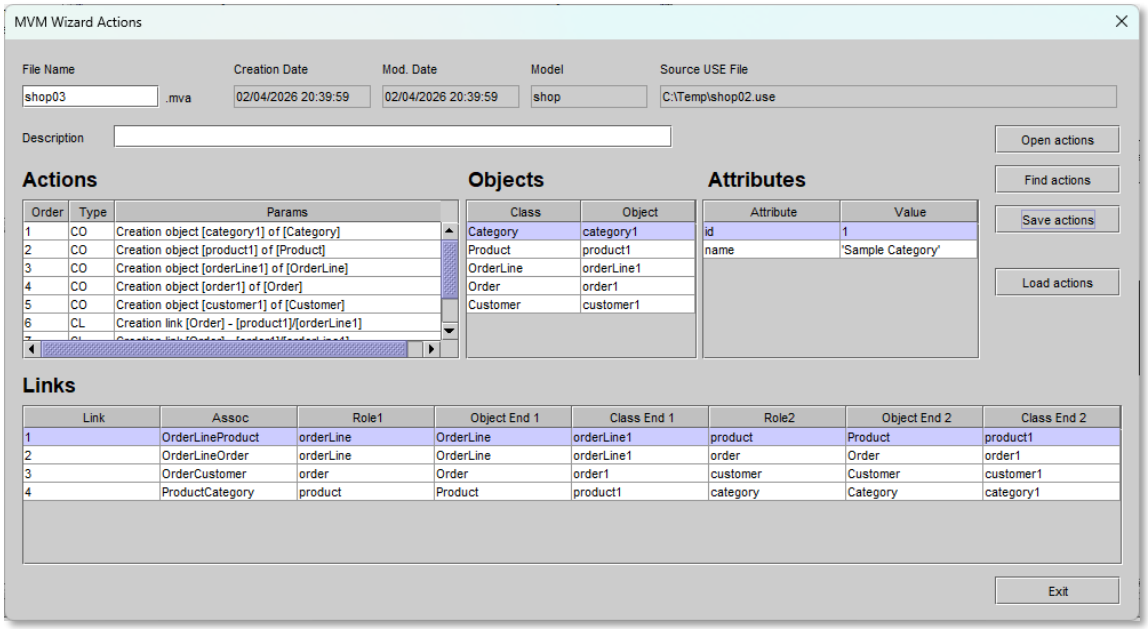
```
self.stock = OrderLine.allInstances()
     ->select(ol | ol.product = self and ol.order.finalized = false)
     ->collect(ol | ol.quantity)
     ->sum()
```

Stay:

```
(self.stock >= OrderLine.allInstances->select(ol : OrderLine |
((ol.product = self) and (ol.order.finalized = false)))->collect(ol
: OrderLine | ol.quantity)->sum)
```

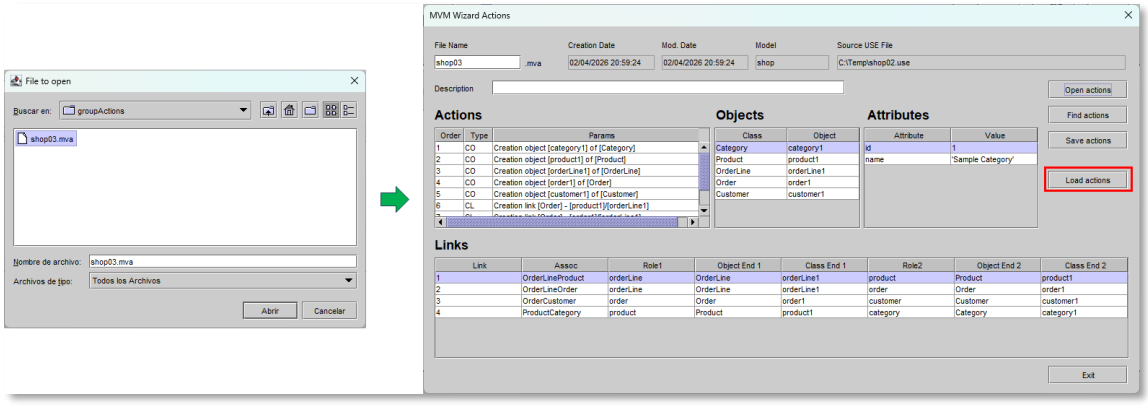
Save the file with the name **shop03.use** and exit the screen by pressing **Exit**.

We also save actions under the name **shop03.mva**:

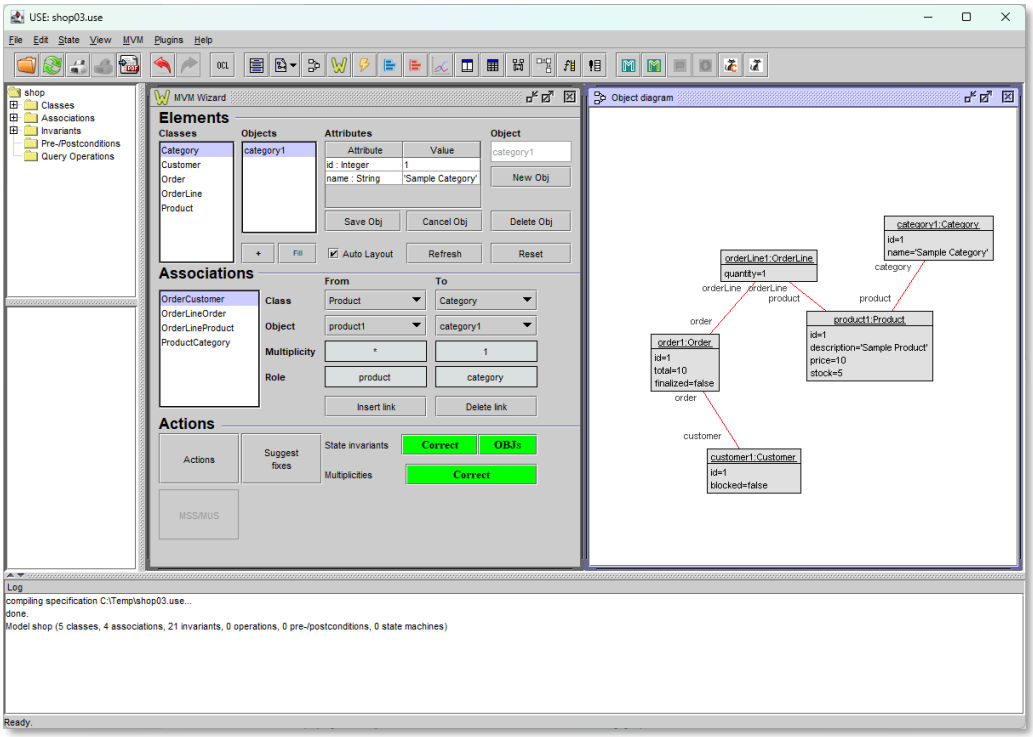


Press **Exit** and open **shop03.use**.

Open **MVM Wizard** and click on the **Actions** button to access **MVM Wizard Actions** and be able to load the **shop03.mva** sequence definition saved in the previous step:



Once we click on **Load Actions**, we will see the following:



ANNEXES

Example of Proposed changes

```
To correct the model, the age of Person must be within the range of 0 to 99.
The weight of Pet must be less than 4.
The invariant 'validAge' must be modified to 'self.age >= 0 and self.age <
99'.
The invariant 'validSmallerThanWeight' must be modified to 'self.weight <
4'.
--------------------------------------------------------------------------
ADDITIONAL INFORMATION ABOUT MUS/MSS
--------------------------------------------------------------------------
Minimal Unsatisfiable Subset:
 - Group1:
   6.validAge
 - Group2:
   7.validGreaterThanWeight
   8.validSmallerThanWeight
--------------------------------------------------------------------------
Maximal Satisfiable Subset:
 - Group1:
   1.validGreaterThanAge
   2.validSmallerThanNegAge
   3.numPets
   4.allWeightGreaterPets
   5.existsWeightGreaterPets
 - Group2:
   6.validAge
 - Group3:
   7.validGreaterThanWeight
   8.validSmallerThanWeight
--------------------------------------------------------------------------
SUMMARY OF INVARIANTS TO BE CHANGED

+------------+---------------------------------+----------------------------
-----+
| Name       | Original                        | Proposal
|
+------------+---------------------------------+----------------------------
-----+
| 6-validAge | self.age <= 0 and self.age > 99 | self.age >= 0 and self.age
< 99 |
+------------+---------------------------------+----------------------------
-----+
```

Request txt example

```
*Profile*
1. You are a software modeling assistant, expert on software design,
development and debugging.
2. You provide concise, concrete and actionable feedback about software
design errors.

*Task*
Given a UML class diagram annotated with OCL invariants that is
inconsistent, you will:
1. Identify the invariants that cause the inconsistency.
2. Review the properties file to see if the specified ranges are correct,
and if not, suggest values to change.
3. Modify ONLY the invariants that MUST be changed to make the model
satisfiable.
4. Calculate the MUS/MSS if this prompt does not provide it later.
5. Produce a JSON output with Properties, Objects, Links, Comment and
ChangedInvariants.
This explanation should be usable by a software engineer to locate and
correct the defects in the model.

*Inputs*
1. Model:

The UML model definition provided in the format of the USE tool (UML-based
Specification Environament) developed by the University of Bremen.

--------------------------------------------------------------------------
Model definition:
"""
model Animals
 class Person
 attributes
   age: Integer
   height: Integer
   weight: Integer
 end

class Pet
 attributes
   name: String
   species: String
   weight: Integer
 end

association BelongsTo between
  Person[*]
  Pet[*]
end
 constraints

context Person inv validGreaterThanAge:
  self.age > 3

context Person inv validSmallerThanNegAge:
  self.age < 8

context Person inv numPets:
   self.pet->size()>0

context Person inv allWeightGreaterPets:
   self.pet -> forAll(p|p.weight>5)

context Pet inv existsWeightGreaterPets:
```

```
   Pet.allInstances()->exists(p|p.weight>5)

-- Invariant that it was never fulfilled
context Person inv validAge:
   self.age <= 0 and self.age >99

-- One or the other will be valid but not both at the same time
context Pet inv validGreaterThanWeight:
  self.weight >= 0

context Pet inv validSmallerThanWeight:
  self.weight < 4




 """
-------------------------------------------------------------------------

2. Properties:
A textual description of the range of allowed values for each attribute in
the model and the range on the number of objects per class and the number of
links per association.

Properties:
"""
[default]

Integer_min = -10
Integer_max = 10

String_max = 10


# ---------------------------------------------------------------------------
Person
Person_min = 1
Person_max = 1

Person_age_min = -1
Person_age_max = -1
Person_height_min = -1
Person_height_max = -1
Person_weight_min = -1
Person_weight_max = -1

# BelongsTo (person:Person, pet:Pet) - - - - - - - - - - - - - - - - - - - -
- -
BelongsTo_min = 1
BelongsTo_max = 1

# ---------------------------------------------------------------------------
Pet
Pet_min = 1
Pet_max = 1

Pet_name_min = -1
Pet_name_max = -1
Pet_species_min = -1
Pet_species_max = -1
Pet_weight_min = -1
Pet_weight_max = -1
# ---------------------------------------------------------------------------
----
aggregationcyclefreeness = on
forbiddensharing = on

"""
```

```
---------------------------------------------------------------------

3. Invariants:
A summary of the invariants in the model, extracted from the USE definition
model, and assigning an integer index to each invariant that will be used to
refer to the invariant.

List of invariants:
"""
1-Person::validGreaterThanAge
2-Person::validSmallerThanNegAge
3-Person::numPets
4-Person::allWeightGreaterPets
5-Pet::existsWeightGreaterPets
6-Person::validAge
7-Pet::validGreaterThanWeight
8-Pet::validSmallerThanWeight"""
---------------------------------------------------------------------

4. OriginalInvariants:
A JSON array containing the exact original text of each invariant.

OriginalInvariants:
"""
[
  {
    "name": "Person::validGreaterThanAge",
    "text": "self.age > 3"
  },
  {
    "name": "Person::validSmallerThanNegAge",
    "text": "self.age < 8"
  },
  {
    "name": "Person::numPets",
    "text": "self.pet->size()>0"
  },
  {
    "name": "Person::allWeightGreaterPets",
    "text": "self.pet -> forAll(p|p.weight>5)"
  },
  {
    "name": "Pet::existsWeightGreaterPets",
    "text": "Pet.allInstances()->exists(p|p.weight>5) "
  },
  {
    "name": "Person::validAge",
    "text": "self.age <= 0 and self.age >99 "
  },
  {
    "name": "Pet::validGreaterThanWeight",
    "text": "self.weight >= 0"
  },
  {
    "name": "Pet::validSmallerThanWeight",
    "text": "self.weight < 4"
  }
]
"""
---------------------------------------------------------------------

5. Instance of the model:
An instance (a set of objects and links among objects of the model, ) that
should be satisfying all the textual invariants and the graphical UML
constraints such as the multiplicities of association ends.

List of objects:
"""
```

```
Object id=[1]|name=[person1]|class=[Person]|field=[age]
value=[4]|field=[height] value=[-1]|field=[weight] value=[-1]
Object id=[2]|name=[pet1]|class=[Pet]|field=[name]
value=['Buddy']|field=[species] value=['Dog']|field=[weight] value=[3]
"""

------------------------------------------------------------------------
List of links:
"""
codeLink="1" end1Class="Person" end1Object="person1" end1Role="person"
end2Class="Pet" end2Object="pet1" end2Role="pet" nomAssoc="BelongsTo"
"""

------------------------------------------------------------------------
*Outputs*

1. Explanation:
   1.1. A brief discussion of the inconsistency problems in the model.
   1.2. For each problem, the model should identify the invariants involved
and outline a potential way to solve the inconsistency.
   1.3. Do not provide suggestions like "this value should be valid".
   1.4. Instead, provide informative statements like "this value should be
larger", "this values should be within the following range", "the value of
attribute X should be different to the value of attribute Y", "this value
should be unique", etc.
   1.5. Each sentence must be on its own line.

2. Updated list of objects and links:
   2.1.1  If the input included an instance of the model, provide a modified
list of objects and links, with minimal changes, such that the corresponding
instance satisfies all invariants and graphical UML constraints.
   2.1.2 Once the list of objects has been compiled, review it again in case
any more need to be created and in case there are any classes without
objects.
   2.2.  If any object has an attribute whose value violates an invariant,
please suggest a new value for that attribute and include it in the list of
objects you provide at the end.
   2.3.  Please ensure that all objects involved in any link of any
association are part of the proposed object list.If an object does not exist
in the object list, do not use it in any link.

3. ChangedInvariants:
   A JSON array containing ONLY the invariants whose text is actually
modified.
   You MUST follow these rules:
     3.1. You MUST compare each proposed invariant with its corresponding
entry in OriginalInvariants.
     3.2. If the proposal is the same as the original (both textually and
semantically), it MUST NOT include that invariant..
     3.3. You MUST modify an invariant ONLY if it is impossible to satisfy
the model without modifying it.
     3.4. If the MUS contains invariants that CAN be satisfied by adjusting
objects or properties, you MUST NOT modify those invariants.
     3.5. If the MUS contains invariants that CANNOT be satisfied without
modifying them, you MUST generate a corrected version.
     3.6. The corrected version MUST be syntactically valid OCL.
     3.7. ChangedInvariants may be empty IF AND ONLY IF no invariant
requires modification.
     3.8. If you mention in the Explanation that an invariant must be
changed, you MUST include it in ChangedInvariants.
     3.9. Retain the numbering indicated in each invariant and indicate it
in the results table.

   Example format:
   "ChangedInvariants":[
     {
       "name":"1-validAge",
       "original":"self.age <= 0 and self.age > 99",
```

```
       "proposal":"self.age >= 0 and self.age < 99"
     }
   ]

*Remarks*
1. Please return only the JSON structure without any additional explanation,
since the result must be delivered to an application.
   Therefore, the JSON structure must contain the following parts:
   1.1 Properties: properties to be modified. The properties tag only refers
to the properties that exist in Animals.properties and they should review
the range limits to see if it is necessary to modify them to satisfy the
invariants.
   1.2 Objects: A list of objects with their corresponding values as if the
modified properties had already been applied.
       Please return the same field names indicated in the request
corresponding to each class.
       Please ensure that when defining objects, you normalize the output
using the tags 'class', 'name', and 'attributes'.
       If more objects are needed to satisfy the multiplicity of the
association links,especially those multiplicities that require an endpoint,
please indicate that these objects should be created and propose a sample in
the list of proposed resulting objects.
       If you don't need to change your values, don't change them.
       However, if there is a value that violates an invariant, it suggests
changing it.
   1.3 Links: For each link, specify the fields: codeLink, end1Class,
end1Object, end1Role, end2Class, end2Object, end2Role, nomAssoc.
       Example: codeLink="1" | end1Class="Person" end1Object="person1"
end1Role="person" end2Class="Pet" end2Object="pet1" end2Role="person"
nomAssoc="BelongsTo"
       For each association, respect its multiplicity. That is, in a
multiplicity of 4, it verifies that there are 4 links.
   1.4 Above all, make sure that all links of associations only use objects
indicated in the previous object list.
   1.5 Comment: Explanation of 'how to correct the model', taking into
account the properties that have been indicated for correction as if they
were already corrected.
       Focus only on modifying invariants (not objects or properties).
       The explanation should not be in the past tense but in the present or
future tense.
       The only tags to return should be: Properties, Objects, Links,
Comment and ChangedInvariants.
       Do not omit the creation of objects or links.
       If the ChangedInvariants tag contains modified invariants, don't
forget to include the invariant number provided in the initial list.
       Return ONLY valid JSON.
       Do not include markdown.
       If MUS exist, then ChangedInvariants must always exist.
       Return each sentence on a separate line, using a line break between
sentences.
       For example:
         Input:  Sentence1. Sentence2. Sentence3.
       Output:
         Sentence1.
         Sentence2.
         Sentence3.
   1.6 Include MUS and MSS in the Comment tag using EXACTLY this format:
------------------------------------------------------------------------
ADDITIONAL INFORMATION ABOUT MUS/MSS
------------------------------------------------------------------------
Minimal Unsatisfiable Subset:
 - Group1:
   1.invariantX
 - Group2:
   2.invariantY
   3.invariantZ
------------------------------------------------------------------------
Maximal Satisfiable Subset:
```

```
 - Group1:
   4.invariantA
   5.invariantB
   6.invariantC
 - Group2:
   7.invariantD
   8.invariantE
 - Group3:
   9.invariantF
  10.invariantG
-----------------------------------------------------------------------
```

JSON request example

```
{
  "response_format" : {
    "type" : "json_object"
  },
  "temperature" : 0,
  "messages" : [ {
    "role" : "user",
    "content" : "*Profile*\n1. You are a software modeling assistant, expert
on software design, development and debugging.\n2. You provide concise,
concrete and actionable feedback about software design
errors.\n\n*Task*\nGiven a UML class diagram annotated with OCL invariants
that is inconsistent, you will:\n1. Identify the invariants that cause the
inconsistency.\n2. Review the properties file to see if the specified ranges
are correct, and if not, suggest values to change.\n3. Modify ONLY the
invariants that MUST be changed to make the model satisfiable.\n4. Calculate
the MUS/MSS if this prompt does not provide it later.\n5. Produce a JSON
output with Properties, Objects, Links, Comment and ChangedInvariants.\nThis
explanation should be usable by a software engineer to locate and correct
the defects in the model.\n\n*Inputs*\n1. Model:\n\nThe UML model definition
provided in the format of the USE tool (UML-based Specification
Environament) developed by the University of Bremen.\n\n--------------------
-------------------------------------------------\nModel
definition:\n\"\"\"\nmodel Animals\r\n class Person\r\n attributes\r\n
age: Integer\r\n   height: Integer\r\n   weight: Integer\r\n end\r\n
\r\nclass Pet\r\n attributes\r\n   name: String\r\n   species: String   \r\n
weight: Integer\r\n end\r\n \r\nassociation BelongsTo between\r\n
Person[*]\r\n  Pet[*]\r\nend \r\n constraints\r\n\r\ncontext Person inv
validGreaterThanAge:\r\n  self.age > 3   \r\n  \r\ncontext Person inv
validSmallerThanNegAge:\r\n  self.age < 8     \r\n\r\ncontext Person inv
numPets:\r\n   self.pet->size()>0  \r\n\r\ncontext Person inv
allWeightGreaterPets: \r\n   self.pet -> forAll(p|p.weight>5)\r\n\r\ncontext
Pet inv existsWeightGreaterPets: \r\n   Pet.allInstances()-
>exists(p|p.weight>5)\r\n\r\n-- Invariant that it was never
fulfilled\r\ncontext Person inv validAge:\r\n   self.age <= 0 and self.age
>99     \r\n\r\n-- One or the other will be valid but not both at the same
time\r\ncontext Pet inv validGreaterThanWeight:\r\n  self.weight >= 0 \r\n
\r\ncontext Pet inv validSmallerThanWeight:\r\n  self.weight < 4    \r\n
\r\n\r\n\r\n\r\n \"\"\"\n-------------------------------------------------------
-------------------\n\n2. Properties:\nA textual description of the range of
allowed values for each attribute in the model and the range on the number
of objects per class and the number of links per
association.\n\nProperties:\n\"\"\"\n[default]\r\n\r\nInteger_min = -
10\r\nInteger_max = 10\r\n\r\nString_max = 10\r\n\r\n\r\n# ----------------
---------------------------------------------------- Person\r\nPerson_min
= 1\r\nPerson_max = 1\r\n\r\nPerson_age_min = -1\r\nPerson_age_max = -
1\r\nPerson_height_min = -1\r\nPerson_height_max = -1\r\nPerson_weight_min =
-1\r\nPerson_weight_max = -1\r\n\r\n# BelongsTo (person:Person, pet:Pet) - -
- - - - - - - - - - - - - - - - - - -\r\nBelongsTo_min =
1\r\nBelongsTo_max = 1\r\n\r\n# ------------------------------------------
```

```
------------------------------ Pet\r\nPet_min = 1\r\nPet_max =
1\r\n\r\nPet_name_min = -1\r\nPet_name_max = -1\r\nPet_species_min = -
1\r\nPet_species_max = -1\r\nPet_weight_min = -1\r\nPet_weight_max = -1\r\n#
------------------------------------------------------------------------
--\r\naggregationcyclefreeness = on\r\nforbiddensharing =
on\r\n\r\n\"\"\"\n------------------------------------------------------------
------------\n\n3. Invariants:\nA summary of the invariants in the model,
extracted from the USE definition model, and assigning an integer index to
each invariant that will be used to refer to the invariant.\n\nList of
invariants:\n\"\"\"\n1-Person::validGreaterThanAge\r\n2-
Person::validSmallerThanNegAge\r\n3-Person::numPets\r\n4-
Person::allWeightGreaterPets\r\n5-Pet::existsWeightGreaterPets\r\n6-
Person::validAge\r\n7-Pet::validGreaterThanWeight\r\n8-
Pet::validSmallerThanWeight\"\"\"\n----------------------------------------
----------------------------\n\n4. OriginalInvariants:\nA JSON array
containing the exact original text of each
invariant.\n\nOriginalInvariants:\n\"\"\"\n[\n  {\n    \"name\":
\"Person::validGreaterThanAge\",\n    \"text\": \"self.age > 3\"\n  },\n
{\n    \"name\": \"Person::validSmallerThanNegAge\",\n    \"text\":
\"self.age < 8\"\n  },\n  {\n    \"name\": \"Person::numPets\",\n
\"text\": \"self.pet->size()>0\"\n  },\n  {\n    \"name\":
\"Person::allWeightGreaterPets\",\n    \"text\": \"self.pet ->
forAll(p|p.weight>5)\"\n  },\n  {\n    \"name\":
\"Pet::existsWeightGreaterPets\",\n    \"text\": \"Pet.allInstances()-
>exists(p|p.weight>5) \"\n  },\n  {\n    \"name\": \"Person::validAge\",\n
\"text\": \"self.age <= 0 and self.age >99 \"\n  },\n  {\n    \"name\":
\"Pet::validGreaterThanWeight\",\n    \"text\": \"self.weight >= 0\"\n  },\n
{\n    \"name\": \"Pet::validSmallerThanWeight\",\n    \"text\":
\"self.weight < 4\"\n  }\n]\n\"\"\"\n----------------------------------------
------------------------------\n\n5. Instance of the model:\nAn instance (a
set of objects and links among objects of the model, ) that should be
satisfying all the textual invariants and the graphical UML constraints such
as the multiplicities of association ends.\n\nList of
objects:\n\"\"\"\nObject id=[1]|name=[person1]|class=[Person]|field=[age]
value=[4]|field=[height] value=[-1]|field=[weight] value=[-1]\r\nObject
id=[2]|name=[pet1]|class=[Pet]|field=[name] value=['Buddy']|field=[species]
value=['Dog']|field=[weight] value=[3]\n\"\"\"\n\n--------------------------
------------------------------------------\nList of
links:\n\"\"\"\ncodeLink=\"1\" end1Class=\"Person\" end1Object=\"person1\"
end1Role=\"person\" end2Class=\"Pet\" end2Object=\"pet1\" end2Role=\"pet\"
nomAssoc=\"BelongsTo\"\n\"\"\"\n\n----------------------------------------
----------------------------\n*Outputs*\n\n1. Explanation:\n   1.1. A brief
discussion of the inconsistency problems in the model.\n   1.2. For each
problem, the model should identify the invariants involved and outline a
potential way to solve the inconsistency.\n   1.3. Do not provide
suggestions like \"this value should be valid\".\n   1.4. Instead, provide
informative statements like \"this value should be larger\", \"this values
should be within the following range\", \"the value of attribute X should be
different to the value of attribute Y\", \"this value should be unique\",
etc.\n   1.5. Each sentence must be on its own line.\n\n2. Updated list of
objects and links:\n   2.1. If the input included an instance of the model,
provide a modified list of objects and links, with minimal changes, such
that the corresponding instance satisfies all invariants and graphical UML
constraints.\n   2.2. If any object has an attribute whose value violates an
invariant, please suggest a new value for that attribute and include it in
the list of objects you provide at the end.\n\n3. ChangedInvariants:\n   A
JSON array containing ONLY the invariants whose text is actually modified.\n
You MUST follow these rules:\n     3.1. You MUST compare each proposed
invariant with its corresponding entry in OriginalInvariants.\n     3.2. If
the proposal is the same as the original (both textually and semantically),
it MUST NOT include that invariant..\n     3.3. You MUST modify an invariant
ONLY if it is impossible to satisfy the model without modifying it.\n
3.4. If the MUS contains invariants that CAN be satisfied by adjusting
objects or properties, you MUST NOT modify those invariants.\n     3.5. If
the MUS contains invariants that CANNOT be satisfied without modifying them,
you MUST generate a corrected version.\n     3.6. The corrected version MUST
be syntactically valid OCL.\n     3.7. ChangedInvariants may be empty IF AND
ONLY IF no invariant requires modification.\n     3.8. If you mention in the
```

```
Explanation that an invariant must be changed, you MUST include it in
ChangedInvariants.\n     3.9. Retain the numbering indicated in each
invariant and indicate it in the results table.\n\n   Example format:\n
\"ChangedInvariants\":[\n     {\n       \"name\":\"1-validAge\",\n
\"original\":\"self.age <= 0 and self.age > 99\",\n
\"proposal\":\"self.age >= 0 and self.age < 99\"\n     }\n
]\n\n*Remarks*\n1. Please return only the JSON structure without any
additional explanation, since the result must be delivered to an
application.\n   Therefore, the JSON structure must contain the following
parts:\n   1.1 Properties: properties to be modified. The properties tag
only refers to the properties that exist in animals.properties and they
should review the range limits to see if it is necessary to modify them to
satisfy the invariants.\n   1.2 Objects: A list of objects with their
corresponding values as if the modified properties had already been
applied.\n       Please return the same field names indicated in the request
corresponding to each class.\n       Please ensure that when defining
objects, you normalize the output using the tags 'class', 'name', and
'attributes'.\n       If more objects are needed to satisfy the multiplicity
of the association links,especially those multiplicities that require an
endpoint, please indicate that these objects should be created and propose a
sample in the list of proposed resulting objects.\n       If you don't need
to change your values, don't change them.\n       However, if there is a
value that violates an invariant, it suggests changing it.   1.3 Links: For
each link, specify the fields: codeLink, end1Class, end1Object, end1Role,
end2Class, end2Object, end2Role, nomAssoc.\n       Example: codeLink=\"1\" |
end1Class=\"Person\" end1Object=\"person1\" end1Role=\"person\"
end2Class=\"Pet\" end2Object=\"pet1\" end2Role=\"person\"
nomAssoc=\"BelongsTo\"\n       For each association, respect its
multiplicity. That is, in a multiplicity of 4, it verifies that there are 4
links.\n   1.4 Comment: Explanation of 'how to correct the model', taking
into account the properties that have been indicated for correction as if
they were already corrected.\n       Focus only on modifying invariants (not
objects or properties).\n       The explanation should not be in the past
tense but in the present or future tense.\n       The only tags to return
should be: Properties, Objects, Links, Comment and ChangedInvariants.\n
Do not omit the creation of objects or links.\n       If the
ChangedInvariants tag contains modified invariants, don't forget to include
the invariant number provided in the initial list.\n       Return ONLY valid
JSON.\n       Do not include markdown.\n       If MUS exist, then
ChangedInvariants must always exist.\n       Return each sentence on a
separate line, using a line break between sentences.\n       For example:\n
Input:  Sentence1. Sentence2. Sentence3.\n       Output:\n
Sentence1.\n         Sentence2.\n         Sentence3.\n   1.5 Include MUS and
MSS in the Comment tag using EXACTLY this format:\n\n----------------------
-----------------------------------------\nADDITIONAL INFORMATION
ABOUT MUS/MSS\n-----------------------------------------------------------
---------\nMinimal Unsatisfiable Subset:\n - Group1:\n   1.invariantX\n -
Group2:\n   2.invariantY\n   3.invariantZ\n---------------------------------
-------------------------------------\nMaximal Satisfiable Subset:\n -
Group1:\n   4.invariantA\n   5.invariantB\n   6.invariantC\n - Group2:\n
7.invariantD\n   8.invariantE\n - Group3:\n   9.invariantF\n
10.invariantG\n-----------------------------------------------------------
---------\n"
  } ],
  "model" : "gpt-4o-mini"
}
```

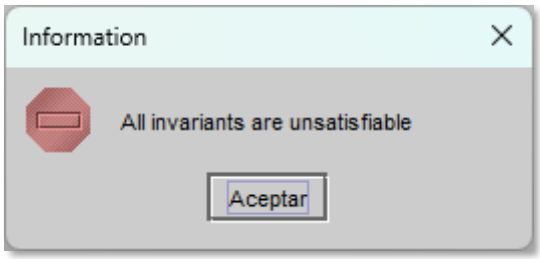
JSON result example


```
{
  "id": "chatcmpl-DRzrYUeqJ5K4yn16KuEVRd1JFJGZO",
  "object": "chat.completion",
  "created": 1775565028,
  "model": "gpt-4o-mini-2024-07-18",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "{\n  \"Properties\": {\n    \"Person_age_min\": 0,\n
\"Person_age_max\": 99,\n    \"Pet_weight_min\": 0,\n    \"Pet_weight_max\":
3\n  },\n  \"Objects\": [\n    {\n      \"class\": \"Person\",\n
\"name\": \"person1\",\n      \"attributes\": {\n        \"age\": 4,\n
\"height\": -1,\n        \"weight\": -1\n      }\n    },\n    {\n
\"class\": \"Pet\",\n      \"name\": \"pet1\",\n      \"attributes\": {\n
\"name\": \"Buddy\",\n        \"species\": \"Dog\",\n        \"weight\": 3\n
}\n    }\n  ],\n  \"Links\": [\n    {\n      \"codeLink\": \"1\",\n
\"end1Class\": \"Person\",\n      \"end1Object\": \"person1\",\n
\"end1Role\": \"person\",\n      \"end2Class\": \"Pet\",\n
\"end2Object\": \"pet1\",\n      \"end2Role\": \"pet\",\n      \"nomAssoc\":
\"BelongsTo\"\n    }\n  ],\n  \"Comment\": \"To correct the model, the age
of Person must be within the range of 0 to 99. The weight of Pet must be
less than 4. The invariant 'validAge' must be modified to 'self.age >= 0 and
self.age < 99'. The invariant 'validSmallerThanWeight' must be modified to
'self.weight < 4'.\\n\\n----------------------------------------------------
------------------\\nADDITIONAL INFORMATION ABOUT MUS/MSS\\n----------------
----------------------------------------------------\\nMinimal
Unsatisfiable Subset:\\n - Group1:\\n   6.validAge\\n - Group2:\\n
7.validGreaterThanWeight\\n   8.validSmallerThanWeight\\n--------------------
----------------------------------------------------\\nMaximal Satisfiable
Subset:\\n - Group1:\\n   1.validGreaterThanAge\\n
2.validSmallerThanNegAge\\n   3.numPets\\n   4.allWeightGreaterPets\\n
5.existsWeightGreaterPets\\n - Group2:\\n   6.validAge\\n - Group3:\\n
7.validGreaterThanWeight\\n   8.validSmallerThanWeight\",\n
\"ChangedInvariants\": [\n    {\n      \"name\": \"6-validAge\",\n
\"original\": \"self.age <= 0 and self.age > 99\",\n      \"proposal\":
\"self.age >= 0 and self.age < 99\"\n    },\n    {\n      \"name\": \"8-
validSmallerThanWeight\",\n      \"original\": \"self.weight < 4\",\n
\"proposal\": \"self.weight < 4\"\n    }\n  ]\n}",
        "refusal": null,
        "annotations": []
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 2515,
    "completion_tokens": 554,
    "total_tokens": 3069,
    "prompt_tokens_details": {
      "cached_tokens": 0,
      "audio_tokens": 0
    },
    "completion_tokens_details": {
      "reasoning_tokens": 0,
      "audio_tokens": 0,
      "accepted_prediction_tokens": 0,
      "rejected_prediction_tokens": 0
    }
  },
  "service_tier": "default",
```

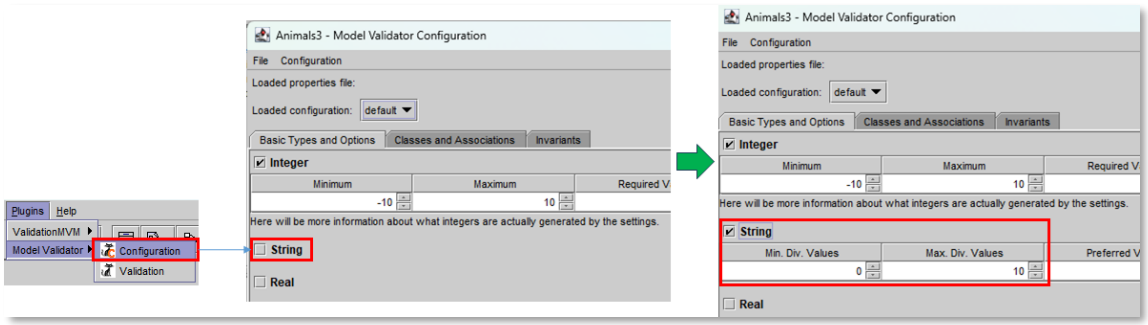
```
  "system_fingerprint": "fp_5e5eb987d2"
}
```

Possible problems in Test

All invariants are unsatisfiable **message appears**:



Review **Basic Types and Options**:

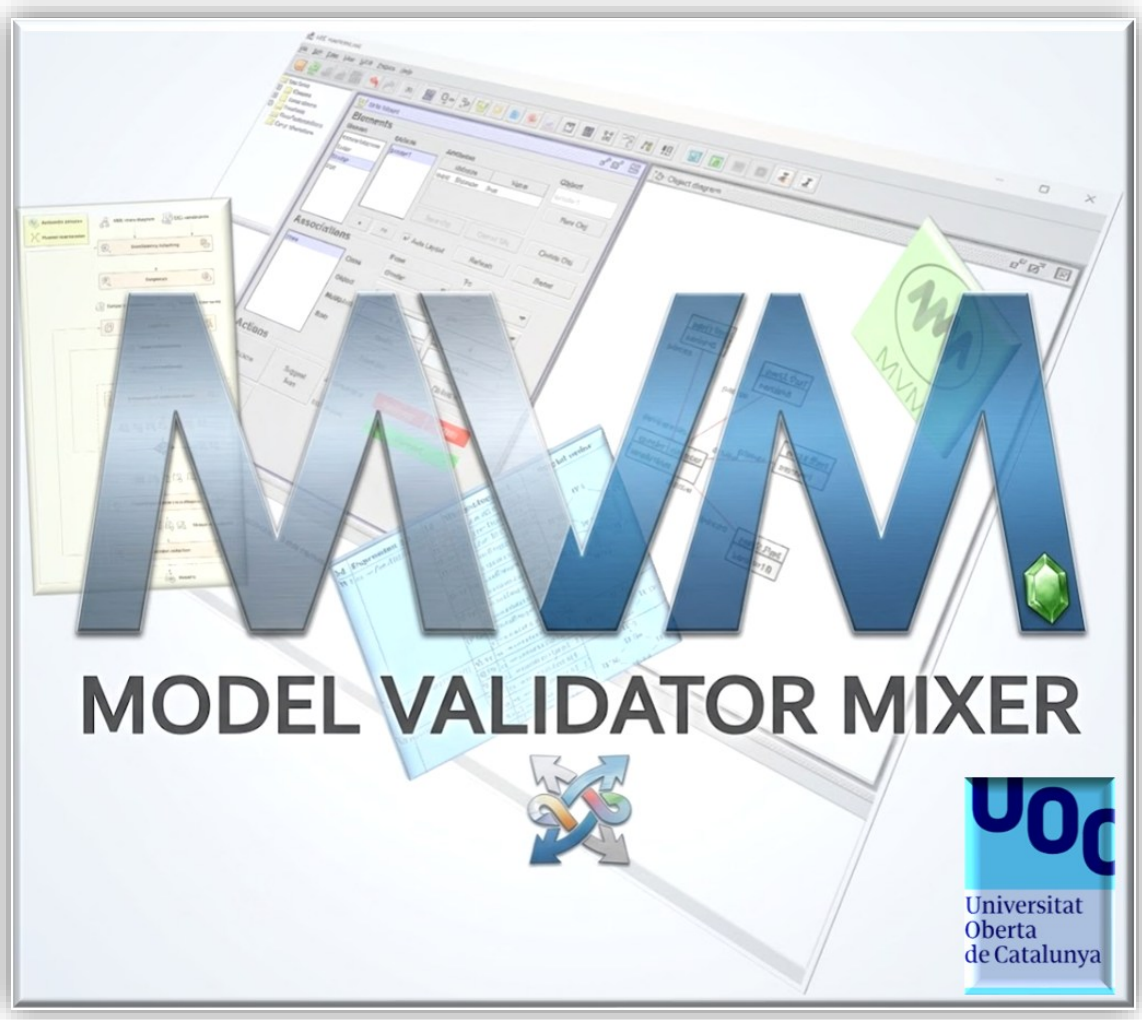


Let's make sure that the **type String** is checked in case the model uses String attributes.

Glossary

Bodyexpression	A primary expression that defines the body or logic of an operation or method.
Invariant	A condition, property, or logical assertion that must always be true at a specific point or throughout the lifecycle of a program, algorithm, or object.
MSS	Maximal Satisfiable Subset (Subconjunto Satisfacible Maximal)
MUS	Minimal Unsatisfiable Subset (Subconjunto Mínimo Insatisfacible).
MVM	Model Validator Mixer
OCL	Object Constraint Language. It is a formal language used to define business rules, constraints, and precise queries in UML models that graphical notation cannot express.
OpenAI	Artificial intelligence (AI) research and deployment company that aims to ensure that general AI (AGI) benefits all of humanity.
Satisfiability	(SAT) in software models is a computational and logical problem that seeks to determine whether there is an assignment of true or false values for the variables of a model, so that all the constraints or conditions defined are true simultaneously
Solver	Optimization tool designed to find the best possible solution (maximum or minimum) to a complex problem, subject to a set of constraints or limitations.
UML	The Unified Modeling Language (UML) is an international graphical standard used to visualize, specify, construct, and document the structure and behavior of complex software systems
USE	UML-based Specification Environment USE is a system for specifying information systems. A USE specification contains a textual description of a model using features present in UML class diagrams (classes, associations, etc.). Expressions written in the Object Constraint Language (OCL) are used to specify additional integrity constraints in the model.
Wizard	A wizard in software is a user interface that guides the user step by step to complete a complex or configuration task, simplifying technical processes without the need for advanced knowledge.

Table of Contents


MVM
MODEL VALIDATOR MIXER
Universitat
Oberta
de Catalunya